University of Warsaw
Faculty of Economic Sciences

Krzysztof Wojdalski
Album N^{o}: 310284

Microstructure-Aware Reinforcement Learning for High-Frequency Trading

Magister (master) degree thesis
Field of the study: Quantitative Finance

The thesis written under the supervision of
dr Pawel Sakowski
WNE UW

Warsaw, September 2026

Declaration of the supervisor

I hereby certify that the thesis submitted has been prepared under my supervision and I declare that it satisfies the requirements of submission in the proceedings for the award of a degree.

Date Signature of the Supervisor

Declaration of the author of the thesis

Aware of legal liability I certify that the thesis submitted has been prepared by myself and does not include information gathered contrary to the law.

I also declare that the thesis submitted has not been the subject of proceedings resulting in the award of a university degree.

Furthermore I certify that the submitted version of the thesis is identical with its attached electronic version.

Date Signature of the Author

Summary

A Twin Delayed DDPG (TD3) agent controls continuous exposure on tick-level order-book data from six Nasdaq equities, trained on three sessions and tested on a held-out fourth. Under frictionless mid-price execution it earns positive returns with low drawdown and a high profit factor. Independently trained policies stay active at 0 and 0.01 basis points but collapse to static positions at higher fees, showing cost sensitivity without a clean break-even; fixed-path bid/ask repricing reverses the result. Of three feature sets only the curated ten-feature microstructure state converged, leaving feature choice unresolved. The contribution is a reproducible pipeline from order-book features through Differential Sharpe Ratio reward design to out-of-sample evaluation.

Key words

reinforcement learning, high-frequency trading, order book, single-asset trading, TD3

(*Słowa kluczowe: uczenie ze wzmocnieniem, handel wysokiej częstotliwości, księga zleceń, handel pojedynczym aktywem, TD3*)

Field of the thesis (codes according to the Erasmus program)

Economics (14300)

Thematic classification

G11, G17, C63

The title of the thesis in Polish

Uczenie ze wzmocnieniem uwzględniające mikrostrukturę rynku w handlu wysokiej częstotliwości

Table of contents

1. INTRODUCTION

Much of the reinforcement learning literature in trading relies on low-frequency market representations such as daily or intraday OHLCV bars.[1] High-frequency trading, however, is governed by information embedded in the limit order book, where local liquidity, queue dynamics, spread changes, and event timing determine short-horizon risks. This thesis examines whether a deep RL agent can use order-book-derived features to make favourable trading decisions in a HFT setting.

High-frequency trading should therefore not be treated as a faster version of medium-frequency trading (MFT). The state observed by the agent is different. At short horizons, aggregated price bars omit information that is central to decision-making. Features derived from the order book provide a more detailed description of market conditions than candlesticks. These include depth imbalance, spread variation, order-flow pressure, and inter-event timing. A primary concern of this thesis is whether such microstructure-aware state representations improve the suitability of RL for this domain.

Reinforcement learning is a plausible modelling framework for this problem because trading is sequential and path dependent. Each action affects future exposure, portfolio evolution, and the set of subsequent decisions available to the agent. In high-frequency settings, this distinction matters.

Reinforcement learning has already been deployed in industry settings on limit order book data, but these applications typically address narrower problems such as optimal execution.[2]

Directional trading is a harder problem for several reasons. First, the reward signal at tick frequency is dominated by noise rather than price movement. Second, the LOB state must be transformed into features that carry predictive signal rather than merely describing liquidity conditions. Third, the agent must generalize across intraday regimes whose statistical properties can differ substantially from the training window.

This thesis investigates the use of RL, with particular focus on TD3, for high-frequency

[1] Abbreviations and domain-specific terms used throughout the thesis are defined in the glossary at the end of the document.

[2] An illustrative example is J.P. Morgan's LOXM system, described in contemporaneous reporting as an AI-driven execution tool for client orders rather than a stand-alone directional trading strategy (Noonan, 2017). In that execution setting, the limit order book is used to inform decisions about execution quality, fill likelihood, and spread-cost trade-offs, not to forecast short-horizon price direction.

single-asset trading based on order-book-derived information. The research is motivated not by a claim that artificial intelligence can universally solve trading, but by a narrower and testable question. Under controlled experimental conditions, can a continuous control agent using microstructure-aware features achieve superior out-of-sample performance relative to selected benchmarks? If so, does that performance survive realistic transaction costs?

Further constraints arise on both the execution and evaluation sides: transaction costs and turnover sensitivity can erase apparent profitability, and favorable in-sample results may instead reflect data leakage or unrealistic execution assumptions. The simulation is not a full execution engine. It omits queue-priority modelling, latency competition, and market impact, valuing the portfolio through a mid-price-like proxy and treating the agent as a price-taking participant that cannot move the book. Chapters 5 and 7 detail these limitations and the methodological safeguards (chronological evaluation, reward-return separation, robustness analysis) adopted in response. The contribution of the thesis is accordingly methodological and empirical: it evaluates RL agents in a microstructure-aware setting while making its simplifying assumptions explicit.

1.1. Scope and Objectives of the Research

1.1.1. Novelty of This Research

Building on the TD3/HFT/order-book focus introduced in Chapter 1, the novelty claim here is comparative. A large share of prior RL trading studies target lower-frequency inputs and evaluate algorithms such as Q-learning, DQN, or PPO primarily in bar-level settings, whereas this thesis aligns algorithm class, data granularity, and feature design jointly with high-frequency market microstructure. This algorithm-data-frequency combination remains less represented in prior work and defines the core novelty claim tested in the empirical chapters.

1.1.2. Hypothesis

The research is based on the following hypotheses, ordered so that each builds on the previous one; together they make the case that robustness analysis is a precondition for any credible economic claim:

1. A TD3 agent trained on order-book-derived high-frequency features learns a genuine, learnable out-of-sample risk-adjusted edge from limit-order-book microstructure, but the

economic magnitude of that edge is of the same order as the transaction cost required to trade on it: it is realised under frictionless mid-price execution and is not expected to survive realistic spread-crossing costs.

2. The empirical performance of the agent is materially sensitive to the transaction-cost assumption: the learned edge is positive under zero fee but changes sign once the per-trade cost reaches a fraction of a basis point. This is the scale of the bid-ask half-spread on these instruments.
3. Within the order-book-based trading framework studied here, extending the state representation from basic snapshot features to a broader microstructure-aware feature set leads to improved out-of-sample performance under identical training and evaluation conditions.
4. The reward objective materially shapes the learned policy: replacing a log-return reward with a Differential Sharpe Ratio reward changes both out-of-sample performance and the policy's directional exposure, under otherwise identical training and evaluation conditions.

1.1.3. Objectives of the Research

The primary research goal is to evaluate RL-based algorithms for high-frequency single-asset trading using order-book-derived data. The research objectives are structured in three interconnected phases:

1. **Design and Implementation**
    - Develop RL trading agents for high-frequency order-book trading applications
    - Construct a testing framework to evaluate agent performance
2. **Testing and Evaluation**
    - Assess agent performance on out-of-sample data against selected benchmarks with established and novel risk-return metrics, operationalizing the frictionless-execution component of Hypothesis 1 (its transaction-cost bound is operationalized under Hypothesis 2)
    - Analyze robustness under varying market conditions
3. **Analysis and Implications**

- Compare alternative order-book-derived feature sets within a common training and evaluation framework, operationalizing Hypothesis 3
- Assess the practical relevance and limitations of the approach under simplified execution assumptions
- Identify the binding constraints on out-of-sample performance (whether they originate in execution modeling simplifications, feature set coverage, or training data volume) to prioritize specific extensions for future work

1.2. Thesis Organization and Chapter Overview

The thesis proceeds from problem formulation and motivating literature to agent design and implementation, and finally to empirical evaluation.

Chapter 1 introduces the problem context, defines the research scope, and presents the methodological framework used throughout the thesis.

Chapter 2 reviews the literature most directly relevant to the present research question. It covers market microstructure evidence motivating order-book-derived features, the landscape of alternative machine learning paradigms and trading decision architectures, the structural case for reinforcement learning in trading, and prior RL trading applications with attention to action-space design, transaction-cost modeling, and evaluation rigor.

Chapter 3 develops the reinforcement learning foundations required for the thesis, with particular emphasis on the progression from earlier methods to TD3 and on the distinctions that matter for continuous-control trading problems.

Chapter 4 presents the design of the trading agent itself. It defines the continuous action space, the microstructure-aware state representation, the reward formulation, and the main policy and value-function design choices used in the empirical system.

Chapter 5 describes the implementation of the research pipeline, including the simulation architecture, data preparation, reproducible feature engineering, code structure, and the TD3-based training workflow used to produce reproducible experiments.

Chapter 6 reports the empirical results. It combines experiment snapshots, performance evaluation, statistical validation, and robustness assessment to separate promising outcomes from noise, overfitting, or fragile conclusions.

Chapter 7 synthesizes the findings, discusses the main limitations of the current framework,

and draws implications for future research and practical trading-system development.

2. LITERATURE REVIEW

This chapter surveys the academic foundations relevant to reinforcement learning for tick-level order-book trading. It first distinguishes the controlled simulation setting studied here from live high-frequency trading, then reviews the microstructure mechanisms that motivate the candidate state variables used in Chapter 4. The chapter then separates descriptive microstructure variables from tradable signals: a feature is useful only if it contains information available before the action, survives costs, and improves out-of-sample performance. The second half compares competing modeling approaches, introduces the reinforcement learning concepts most relevant to trading, and reviews prior RL trading applications through state representation, action-space design, reward definition, transaction-cost modeling, and validation rigor. The survey concludes by identifying the comparatively underexplored combination of LOB-derived features, continuous control, and microstructure-aware state design that this thesis studies.

2.1. Tick-Level Order Book Trading

Tick-level trading differs from bar-based trading because the decision unit is an order book event rather than a fixed calendar interval. A daily or minute-level strategy can often treat execution as a secondary implementation detail after the signal is estimated.[3] At tick frequency, execution conditions are part of the signal environment itself: the spread, queue sizes, recent order-flow pressure, and time since the last event all affect whether a position change is worth making.

This matters because expected price changes over short horizons are small relative to trading frictions. Crossing the spread, paying transaction costs, or changing exposure during a transient liquidity shock can dominate the gross directional signal. A tick-level agent must therefore condition not only on where the price may move, but also on whether the current liquidity state makes acting economically sensible.

The present thesis studies this problem in a controlled simulation, not in a live high-frequency trading environment. The simulator uses historical limit-order-book data and explicit cost assumptions to evaluate continuous position control. It does not model exchange latency,

[3] This generalization depends on the scale of trading. For investment funds, execution is rarely a negligible issue — quite the opposite. Large-scale execution costs, market impact, and implementation shortfall can dominate gross returns, making execution quality a primary concern.

queue priority, partial fills, strategic order cancellation, or the market impact of the agent's own orders. The literature reviewed below is therefore used to motivate state variables and evaluation discipline, not to claim full live-HFT realism.

The microstructure mechanisms discussed in this chapter manifest in observable empirical regularities at intraday and tick-level timescales.

At these timescales, financial markets exhibit structural patterns that are distinct from the macro-level anomalies studied in the asset pricing literature. They arise not from investor irrationality or information asymmetry in the fundamental sense, but from the mechanics of how orders interact in a limit order book. These mechanics include the strategic behavior of market makers, the clustering of institutional activity, and the discrete price grid imposed by exchange rules. Understanding these patterns is a prerequisite for designing a tick-level trading agent, since they define the environment in which the agent operates and motivate the feature set used here.

Bid-ask spreads induce negative serial correlation in transaction prices at very short horizons, as they bounce between bid and ask quotes (Roll, 1984). This microstructure noise distinguishes observed transaction prices from the latent efficient price and creates a mechanical pattern that matters for any tick-level decision rule. In this work, spread in basis points and the spread ratio feature directly encode the cost of crossing the bid-ask, conditioning the agent's position-sizing decisions on the current liquidity environment.

Intraday patterns in volume, volatility, and returns have been documented: a pronounced U-shape in trading activity, with elevated liquidity demand at market open and close and a quiet midday period (Harris, 1986; Wood et al., 1985). These patterns reflect institutional trading schedules and liquidity provision dynamics rather than information arrival. The hour-of-day sine and cosine encodings in the observation vector are motivated by this evidence because the agent should distinguish early-session conditions (wide spreads, high adverse selection) from midday periods (tighter spreads, lower activity) when sizing positions.

Prices cluster at round numbers and psychologically attractive levels, creating discontinuities in the order book that affect optimal order placement and execution (Harris, 1991). In a 10-level LOB, price clustering manifests as uneven spacing between price levels. This pattern is captured by the book convexity feature, which measures the curvature of the price ladder on each side. Awareness of price clustering is also relevant to the spread ratio feature, since clusters create temporary spread compression and expansion that deviates from the recent average.

These three findings collectively establish that short-horizon price dynamics are structured, not random. The bid-ask bounce motivates spread-aware features and reward design; intraday periodicity motivates temporal context features; price discreteness motivates curvature features of the order book. Rather than treating the limit order book as a transparent window onto an efficient price, the empirical evidence supports modeling it as a strategic environment with predictable mechanical regularities. This is precisely the setting in which a reinforcement learning agent can learn to act systematically.

2.2. Market Microstructure Mechanisms

Market microstructure is grounded in a theoretical literature that models the strategic interactions between informed traders, uninformed liquidity demanders, and market makers in a limit order book. Chapter 4 derives its features from three elements of each theoretical framework: the market mechanism, its observable proxy, and its limitation. Each framework motivates a candidate state variable, but none alone establishes profitability after costs.

Adverse selection and the spread decomposition. The bid-ask spread reflects a market maker's protection against trading with informed counterparties (Glosten & Milgrom, 1985). In their model, the market maker faces two types of order flow: uninformed liquidity traders, whose trades are uncorrelated with future price changes, and informed traders, who possess private information about fundamental value. Setting a positive spread allows the market maker to profit from uninformed flow and recoup losses on informed flow. Their contribution is to show that this adverse-selection channel alone is sufficient to generate a positive spread, in a model that assumes away inventory and order-processing costs entirely. Spreads observed in real markets are conventionally decomposed further, adding inventory-holding and order-processing components to the adverse-selection component isolated here.

This decomposition has a direct implication for trading signal design. A narrowing spread signals reduced adverse selection risk; a widening spread signals elevated informed activity. These measures, operationalized in Chapter 4 as spread in basis points and the spread ratio, are therefore motivated as candidate state variables by this theoretical framework.

The price impact of informed trading was formalized through the lambda parameter, which measures the sensitivity of price changes to order flow (Kyle, 1985). In Kyle's continuous-time model, the informed trader splits their order over time to maximize expected trading profit,

trading gradually so that each trade reveals as little as possible while remaining camouflaged by noise-trader flow, and the market maker sets prices to break even against order flow. The resulting relationship between order imbalance and price change is linear, with slope $\lambda = \sqrt{\Sigma_0}/(2\sigma_u)$: the ratio of the standard deviation of fundamental-value uncertainty to that of uninformed order flow, so price impact rises with information asymmetry and falls as noise trading supplies more camouflage. The order flow features in Chapter 4 (OFI and signed trade flow) can be understood as empirical proxies for Kyle's order imbalance variable, with the bid-ask slope features measuring the depth analogue of λ across levels of the order book. The limitation is that these proxies summarize observable pressure and depth, not the latent information set of the informed trader in Kyle's model.

Limit order book mechanics. The theory of the open limit order book as a competitive equilibrium was developed, showing that a full LOB can sustain itself as a market mechanism under adverse selection (Glosten, 1994). The key insight is that the price schedule embedded in the LOB (the sequence of bid and ask quotes at increasing depths) reflects the cumulative information content of orders at each quantity level. Because an order resting at depth executes only against a larger incoming trade, and larger trades are more likely to be information-driven, the price schedule must slope upward: deeper levels are compensated for bearing more adverse selection per unit, not less, alongside the uncertainty of whether they execute at all.

This theoretical result motivates the multi-level features used in Chapter 4. The book is not a single price but a price schedule. The shape of that profile (measured by depth ratios, slopes, and convexity features) may encode information about how informed the marginal liquidity provider expects the market to be at each quantity level.

Order flow and short-term price prediction. The most direct empirical and theoretical treatment of the relationship between LOB events and short-term price changes is found in the market microstructure literature (Cont et al., 2014). Their framework decomposes all LOB events into limit order arrivals, cancellations, and market orders, and defines order flow imbalance (OFI) as the net directional pressure these events exert on the best bid and ask queues. Using NYSE TAQ data for 50 randomly sampled S&P 500 stocks, they find a linear relationship between OFI and mid-price changes over ten-second intervals, with OFI explaining an average of 65% of short-term price variation (68% with a nonlinear correction); the authors note that OFI partly overlaps by construction with price-changing events, and that R^2 falls to roughly 35–60% when

that overlap is removed. Importantly, the relationship is contemporaneous: OFI at time t is correlated with price changes at time t, not a lagged effect, which is consistent with the efficient incorporation of order book information into prices.[4]

The theoretical channel is straightforward: when buy-side pressure dominates (positive OFI), market makers raise quotes to attract sell-side flow and reduce inventory risk, mechanically pushing the mid-price upward. The OFI feature in the feature design is included primarily as a state variable summarizing current order-flow pressure.

Top-of-book price adjustment. The weighted mid-price shifts the ordinary midpoint according to the displayed sizes at the best bid and ask. Stoikov distinguishes this static quantity from the model-based micro-price, which estimates a future midpoint conditional on the current midpoint, queue imbalance, and spread (Stoikov, 2018). This thesis implements only the static weighted mid-price, referred to here as the queue-weighted midpoint. The implementation retains the legacy feature name `hft_microprice`. Its divergence from the midpoint summarizes current top-of-book imbalance. It is not treated as a theoretically optimal forecast of the next transaction price. The measure also omits deeper queue dynamics, hidden liquidity, and the agent's execution priority.

Taken together, these four theoretical frameworks identify several mechanisms that may be relevant for state design in limit-order-book markets. The spread encodes adverse selection risk (Glosten & Milgrom, 1985); price impact and order imbalance are related through Kyle's lambda (Kyle, 1985); the shape of the full book encodes the information content at each depth level (Glosten, 1994); and OFI is the dominant contemporaneous predictor of short-term price changes (Cont et al., 2014). The queue-weighted midpoint summarizes the current top-of-book imbalance (Stoikov, 2018).

These mechanisms are not independent: they all reflect aspects of information aggregation through order flow, motivating the candidate feature families introduced in Chapter 4: spread-based, order-flow, depth-shape, and fair-value signals.

[4] OFI's relevance for the present thesis is narrower than the contemporaneous result implies: it serves as a compact state descriptor of current queue pressure at the time the agent acts, not a validated forecasting signal.

2.3. From Microstructure Signals to Tradable Decisions

LOB-derived variables may describe the market without forecasting it. A feature earns its place in the state space only if it survives transaction costs and improves held-out performance.

In the quantitative trading literature, predictive signals of this kind are commonly referred to as alphas: mathematical expressions over market data that encode a directional view on short-horizon returns (Tulchinsky, 2019). The microstructure features in this thesis's state space (order-flow imbalance, queue-weighted-midpoint divergence, spread regime, and book shape) occupy the same conceptual role, but are not used as standalone trading rules. Instead, they serve as inputs to an RL policy that learns, through experience, how to combine and act on them optimally. This replaces the hand-coded combination rule with a learned function while keeping the same evaluation discipline of out-of-sample performance under realistic costs.

2.4. Competing Modeling Approaches

Alternatives to reinforcement learning differ along two dimensions: learning paradigm and decision-system architecture. The relevant paradigms are supervised, unsupervised, semi-supervised, and reinforcement learning. Their architectural differences motivate the choice of reinforcement learning for the present problem.

2.4.1. Learning Paradigms

Supervised Learning. Supervised learning trains a model on labeled input-output pairs $(X_i, Y_i)_{i=1}^n$ to minimize a loss function $\mathcal{L}(f_\theta(X), Y)$ over a held-out sample. In trading, supervised approaches have been widely applied to return prediction, directional classification, and volatility forecasting. These are tasks where historical outcomes provide a natural training signal (Krauss et al., 2017; Zhang et al., 2019).

This includes modern LOB prediction models, which use deep architectures to map order book snapshots or event sequences into short-horizon price-movement labels. Such models are a serious benchmark class for extracting information from high-dimensional market data, especially when the objective is directional classification (Zhang et al., 2019). A strong supervised trading benchmark would not stop at an unconstrained return forecast. It would translate predicted returns or class probabilities into positions through a cost-aware rule, such as trading only when the

expected move exceeds the spread and fee threshold, scaling exposure by forecast confidence, or optimizing classification thresholds on a validation set for net return rather than raw accuracy. A second supervised variant would learn the action directly from labels constructed *ex post*: for example, the position that would have maximized next-horizon net return after costs, or a discretized buy/sell/hold target with an explicit no-trade region.

These alternatives are important because they close part of the gap between prediction and trading. They allow the supervised model to account for transaction costs, avoid low-conviction trades, and produce a policy-like mapping from market state to position. The limitation is therefore not that supervised learning is an inherently weak baseline. It is that the financial objective enters the pipeline indirectly: either through hand-designed thresholds and sizing rules applied after prediction, or through labels whose definition already embeds a particular holding horizon, cost model, and notion of *ex post* optimality.

This creates two sources of possible misalignment. First, a model that minimizes mean squared error, cross-entropy, or another statistical loss may not maximize risk-adjusted return after transaction costs, because predictive accuracy and trading profitability are not equivalent criteria (Moody & Saffell, 1998). Second, direct action labels are only as credible as the rule used to construct them. In a noisy and path-dependent environment, the *ex post* best one-step action need not be the action that maximizes longer-horizon portfolio utility once turnover, inventory persistence, and risk are considered.

This misalignment is most consequential at high frequencies, where turnover costs consume a large fraction of gross returns. At lower frequencies (daily or weekly rebalancing), the cost of turning over a position is small relative to the forecast signal, and well-specified supervised models remain competitive. Factor-based strategies in the Fama-French tradition, momentum screens, and lower-frequency deep-learning forecasting approaches operate at horizons where transaction costs are a second-order concern, so a predict-then-trade pipeline is not obviously handicapped in that setting. The advantage of direct optimization narrows as the cost-to-signal ratio decreases. At tick frequency, where the present thesis operates, this ratio is unfavorable for a predict-then-trade pipeline. The cost of crossing the spread on each position change can exceed the directional signal per step, making the alignment between reward and realized return a first-order design requirement. The motivation for reinforcement learning in this thesis should therefore be understood narrowly. TD3 is not adopted because supervised LOB prediction is

irrelevant, but because the empirical question studied here concerns continuous exposure control under a reward function that includes the consequences of position changes directly.

Unsupervised Learning. Unsupervised learning identifies structure in unlabeled data $\{X_i\}_{i=1}^n$ without a target variable. In trading research it is primarily used for market regime detection (clustering volatility or microstructure states), dimensionality reduction of large feature sets (PCA, autoencoders), and anomaly detection in order flow data.

These applications are useful as preprocessing or exploratory tools, but unsupervised learning does not produce trading decisions directly. It carries no reward signal and cannot optimize for a financial objective such as risk-adjusted return. In the present thesis, the regime-conditioning problem is addressed implicitly through engineered microstructure features (spread ratio, inter-event time, temporal encodings) that condition the agent's observation on the current market state without requiring an explicit unsupervised clustering step.

Semi-Supervised Learning. Semi-supervised learning combines a small labeled set with a larger unlabeled set, propagating label information through structural similarity in the data. It is useful when labeling is expensive or subjective. Medical annotation is the canonical application.

In financial trading, the approach is rarely adopted because the main bottleneck is not label scarcity but the absence of a stable labeling criterion: what constitutes an optimal action at a given market state depends on subsequent price evolution, which is noisy and non-stationary. Constructing pseudo-labels for unlabeled market observations requires either a strong prior on optimal behavior or a separate forecasting model. At that point, the problem has already been reformulated as supervised or reinforcement learning. This instability of financial labels is documented in the literature: fixed-horizon labeling schemes are path-dependent and regime-sensitive, which motivates the triple-barrier method as a more robust alternative. This is itself an acknowledgment that no universal labeling criterion exists across market conditions (López de Prado, 2018, ch. 3).

The same conclusion follows from a second perspective: the architecture of the decision system itself.

2.4.2. Decision-System Architectures

Rule-Based Trading Strategies. Rule-based strategies encode trading decisions as explicit conditional logic: position changes are triggered when observable market quantities cross pre-

defined thresholds. Common examples include moving-average crossover systems, momentum breakout rules, and mean-reversion strategies based on Bollinger Bands or RSI. In high-frequency settings, rule-based approaches include market-making strategies that post limit orders at fixed offsets from the mid-price and cancel them when inventory limits are reached.

The appeal of rule-based systems is transparency: the causal chain from market condition to action is explicit, auditable, and reproducible. They also impose no distributional assumptions on the data and can be deployed without a training phase.

The limitation is inflexibility. Rules are defined over a fixed set of observable conditions and optimized for a particular market regime. When it changes (for example, when a previously mean-reverting asset enters a trending period), fixed rules either require manual recalibration or continue generating losses until the regime reverts. This brittleness under non-stationarity is the primary motivation for learning-based approaches, which can update their behavior from experience rather than requiring explicit rule revision.

Indirect Optimization via Forecasts or Labels. An intermediate class of trading systems learns a surrogate target rather than optimizing trading performance directly. One variant employs supervised learning to generate price forecasts from market inputs and then feeds those forecasts into a separate decision rule. Another variant trains directly on labeled state-action pairs, imitating historical or hand-crafted trades without an explicit forecasting stage. This learns the action-mapping function by minimizing imitation loss against expert or rule-derived action labels, the standard behavior-cloning setup.

The limitation is the same in both cases: the optimization target is a surrogate rather than the trading objective itself. Forecast-based systems optimize statistical accuracy and then translate predictions into actions through a separate rules layer, discarding much of the contextual information present in the original state. Label-based systems optimize imitation error against actions derived from historical rules or expert behavior, which embeds a prior about what constitutes optimal behavior and does not guarantee maximizing risk-adjusted return.

Direct Optimization of Trading Performance. Direct optimization maps market states to actions and rewards them from realized returns net of transaction costs, eliminating the calibration gap between a forecasting objective and a separate decision rule. It does not eliminate modeling assumptions: the Differential Sharpe Ratio used here approximates risk-adjusted return, so its relationship with out-of-sample performance remains empirical. Chapter 4 develops this

distinction, explains the need for reward shaping at tick frequency, and separates the training objective from financial evaluation.

2.5. Reinforcement Learning for Trading

Reinforcement learning is a reasoned choice for algorithmic trading, not an arbitrary methodological one. The argument rests on three structural properties of the trading problem.

Unknown market dynamics. Model-based approaches require an explicit transition model. This is a specification of how the environment responds to actions. In financial markets, this model is unknown and non-stationary: price dynamics depend on the aggregate behavior of all participants, which changes continuously. Model-free RL methods learn directly from experience without requiring a transition model, making them a reasonable candidate for this setting (Sutton & Barto, 2018).

Off-policy learning and historical replay. Online experimentation in live markets is constrained by capital, risk, and regulatory requirements. The alternative is to learn from historical order book data. This is a fixed dataset of past transitions. Off-policy algorithms such as TD3 (Fujimoto et al., 2018) can train on stored experience rather than requiring fresh environment interaction at every update, making them sample-efficient in the data-limited financial setting. This should not be confused with a full offline-RL guarantee. Training from historical replay still raises distribution-shift and extrapolation risks when the learned policy selects actions that are poorly represented in the historical data.

Continuous position sizing. Trading decisions are naturally continuous: the agent should be able to express any degree of conviction through the size of its position, not just a binary buy/sell signal. Value-based methods such as DQN (Mnih et al., 2015) require a discrete action space and cannot represent fractional positions without reintroducing a grid approximation. Deterministic actor-critic methods operate natively in continuous action spaces and are therefore more appropriate for position-sizing tasks.

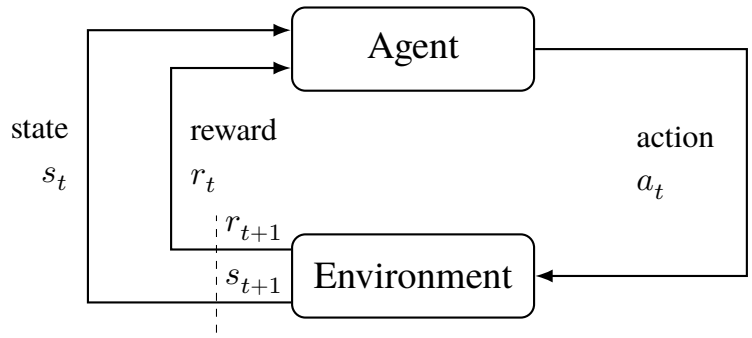
Together these three properties point toward a model-free, off-policy, continuous-action algorithm, which is the class to which TD3 belongs. This is a reasoned design choice, not a claim that TD3 is universally superior for trading. The MDP formulation remains approximate because market states are partially observable and non-stationary. Chapter 3 derives TD3's formal algorithmic properties. The broader RL literature informs the trading-specific design choices

made here.

2.6. Trading Problem Formulation

The finance literature reviewed here relies on RL concepts with specific consequences for trading. Chapter 3 provides their formal definitions, equations, and algorithm taxonomies; the standard reference for the framework itself is Sutton & Barto (2018).

Fig. 1. The agent-environment interaction loop in reinforcement learning.



Source: *author's own elaboration, based on Sutton & Barto (2018), 2nd ed.*

At each step the agent observes state s_t and reward r_t, selects action a_t, and receives the next state s_{t+1} and reward r_{t+1} from the environment.

Portfolio decisions are sequential and path-dependent. This makes the MDP framework (Bellman, 1957; Puterman, 1994) the formal language for trading agents. However, financial environments only approximately satisfy standard MDP assumptions. Market regime shifts, structural breaks, and partial observability all contribute to this approximation.

For this review, the most useful RL concepts are not the formal machinery of Chapter 3, but the dimensions along which applied trading papers differ. These dimensions determine whether results from one study are comparable to another and whether they are relevant to the problem addressed in this thesis.

The first dimension is the formulation of the trading problem itself: what is treated as the state, what counts as an action, and what scalar reward is used to evaluate behavior. In trading applications, these choices vary materially across studies. Some papers define the state using bar-level prices and technical indicators, while others incorporate current holdings, order-book information, or broader market context. Likewise, actions may represent a discrete buy/sell/hold decision or a continuous exposure level. Reward definitions also differ: some studies optimize raw

return, others net return after transaction costs, and others risk-adjusted criteria such as Sharpe-oriented objectives (Moody & Saffell, 1998). These modeling choices are not cosmetic. They change the economic meaning of the task the algorithm is solving, and any reward transformation must preserve the original task's optimal policy to remain valid (Ng et al., 1999).

The second dimension is action-space structure. Discrete-action methods such as Q-learning and its deep variants are naturally aligned with buy/sell/hold formulations (Mnih et al., 2015; Watkins, 1989). Continuous-control methods, by contrast, are designed for problems in which the agent chooses a portfolio weight or exposure level on a continuum. This distinction is central in the trading literature because it separates papers that treat trading as a classification-like decision problem from papers that treat it as a position-sizing problem. For the latter class, policy-gradient and actor-critic methods are usually the relevant algorithmic family (Sutton et al., 2000; Williams, 1992).

The third dimension is the training regime, especially the distinction between on-policy and off-policy learning. In financial applications this matters not only for sample efficiency but also for the practicality of learning from historical data. Off-policy methods can in principle reuse stored experience through replay-based updates (Lin, 1992), which is one reason they appear frequently in trading research where live experimentation is expensive or constrained. At the same time, the literature makes clear that training on historical market data does not eliminate instability, because financial environments remain noisy, non-stationary, and only approximately Markov (Deng et al., 2017; Tsitsiklis & Van Roy, 1997). Algorithm choice therefore cannot be separated from the validation protocol under which performance is assessed.

The fourth dimension is reward design. Financial RL studies differ not only in which algorithm they use, but in what behavior they incentivize. Rewards based on raw price changes can encourage excessive turnover or sensitivity to noise, whereas cost-aware or risk-adjusted rewards aim to align optimization more closely with economically meaningful outcomes. The Differential Sharpe Ratio is one influential example (Moody & Saffell, 1998), but the broader point is that reported performance is difficult to interpret without understanding the objective under which the policy was trained.

These distinctions provide the lens for reading the applied RL studies reviewed in the next section. Chapter 3 develops the formal reinforcement learning framework and the algorithmic lineage leading to TD3.

2.7. Applied RL Trading Evidence

RL trading research has progressed from theoretical proposals to applications across diverse markets. Five design dimensions organize the relevant evidence: state representation, action-space structure, reward definition, transaction-cost modeling, and validation protocol.

To keep conclusions methodologically defensible, this review distinguishes between: (i) proof-of-concept evidence from backtests under simplified assumptions, and (ii) stronger evidence that includes robust out-of-sample design, realistic cost modeling, and variance reporting across repeated runs. The chronology below is therefore secondary to the methodological comparison: reported performance is interpreted skeptically unless the study specifies what information the agent observes, how actions map to trades, whether costs are charged, and how out-of-sample performance is selected and reported.

2.7.1. Early Applications and Proof-of-Concept

Moody and Saffell present an early application of recurrent reinforcement learning to trading (Moody & Saffell, 1998). Their framework directly optimizes financial objectives, including profit, wealth, the Sharpe ratio, and the Differential Sharpe Ratio. They compare recurrent reinforcement learning with a Q-learning trader in a monthly S&P 500 index and Treasury-bill allocation problem. Both learned systems outperform buy-and-hold over the reported test period after a 0.5% transaction cost. The recurrent system achieves the higher Sharpe ratio. The paper refers to earlier work for comparisons with supervised learning; the experiment reported here does not test a supervised baseline.

A subsequent journal article develops the framework further and adds an intraday foreign-exchange application (Moody & Saffell, 2001). It models transaction costs and reports stronger simulated results for RRL than Q-learning. These historical experiments support feasibility, but they do not validate live trading performance.

2.7.2. From Discrete to Continuous Action Spaces

Early Q-learning studies tested asset allocation and daily equity trading. Neuneier allocates between the DAX and cash and reports higher terminal wealth than three benchmarks under a 0.2% transaction-cost assumption (Neuneier, 1998). However, the printed training period

overlaps the test period, so the comparison is not cleanly out of sample. Lee et al. divide KOSPI 200 trading into agents for buy timing, buy price, sell timing, and sell price (J. W. Lee et al., 2007). They report better profit and risk results than alternative systems. Training stops when monitored profit rates begin to deteriorate, making the stopping point part of model selection. These studies show that discrete actions can encode timing and execution decisions. For position sizing, however, discretization imposes a grid. Coarse grids limit precision, while fine grids increase action-space complexity.

Extending direct/recurrent RL to foreign-exchange trading (Dempster & Romahi, 2002; Gold, 2003) confirmed that mapping state directly into positions is viable, but performance varied substantially across currency pairs and system parameters. This is feasibility evidence, not a robust trading edge.

2.7.3. Deep Reinforcement Learning Era

The introduction of deep function approximators to RL trading extended applicability to high-dimensional state spaces.

DDPG was applied to 30 Dow Jones equities using daily prices, holdings, and cash as state variables (Liu et al., 2018). The agent continued training during the nominal trading period, so the reported results do not represent a frozen-policy evaluation. A fuzzy-representation autoencoder was combined with direct reinforcement learning for futures trading (Deng et al., 2017). Deng et al. test several architecture settings, but neither study reports variation across random seeds. Henderson et al. later documented sensitivity to seeds, implementations, and hyperparameters on standard continuous-control benchmarks (Henderson et al., 2018). Their study did not examine financial rewards.

The EIIE framework included portfolio-vector memory (PVM) and CNN, RNN, and LSTM policy variants for cryptocurrency portfolio management (Jiang et al., 2017). Its fully differentiable end-to-end design incorporated previous portfolio weights into the state and penalized transaction costs explicitly. The three variants outperformed a uniform constant rebalanced portfolio and a suite of published online portfolio-selection algorithms in backtests on 11 coins plus Bitcoin as cash. The policies continued learning during these backtests, so the results were adaptive-policy simulations rather than frozen-policy evaluations.

An earlier cryptocurrency portfolio study used a convolutional network to map historical

prices directly into continuous portfolio weights under a deterministic policy-gradient-style objective (Jiang & Liang, 2017). Its 30-minute Poloniex backtests are relevant because they treat trading as direct portfolio-action optimization, but the assumptions of immediate execution and no market impact remain far from tick-level order-book trading.

DDPG, PPO, and Policy Gradient (PG) were compared for China A-share portfolio management, finding, counterintuitively, that the simpler PG algorithm outperformed both actor-critic methods (Z. Liang et al., 2018). The proposed adversarial training method improved average daily return and Sharpe ratio in backtests, with the PG-based agent outperforming a uniform constant rebalanced portfolio baseline.

A focused TD3 study on single-asset trading for AMZN and BTC used daily data and a continuous action space in $[-1, 1]$ (Majidi et al., 2024). Its minimal state representation (lagged close-return windows rather than large technical-indicator sets) reported improved return and Sharpe ratio for TD3 versus discrete variants and buy-and-hold, though results were mixed across assets. This study is best interpreted as evidence that continuous action sizing can be beneficial under realistic frictions, not as definitive proof of robust alpha across market regimes.

Kabbani and Duman evaluate a TD3 framework using account state, price variables, technical indicators, and news sentiment (Kabbani & Duman, 2022). In their first experiment, returns and Sharpe ratios increase as indicators and sentiment are added. These results are averaged across five seed-varying runs, but the experiment uses training data only. A second ten-stock experiment reports a Sharpe ratio of 2.68 during a nominal test period. The agent continues learning during that period, and no seed distribution is reported for this result. The simulator deducts percentage commissions but assumes zero slippage and market impact. The study is therefore best interpreted as a proof of concept rather than a frozen-policy out-of-sample performance estimate.

Two further studies mark boundary cases outside this thesis's LOB-level, single-asset scope. An ensemble PPO/A2C/DDPG strategy on Dow Jones constituents (Yang et al., 2020) is a useful actor-critic comparison in a liquid equity universe, but its state representation, trading frequency, and execution assumptions differ from millisecond-level LOB trading. AlphaStock (Wang et al., 2019) combines a Sharpe-oriented objective with history-state and cross-asset attention, contributing mainly through representation and interpretability in stock selection. This supports the claim that state representation matters materially in RL trading, but does not address

order-book-driven single-asset exposure control.

A volatility-scaled deep RL trading strategy tested across 50 liquid futures contracts spanning commodities, equity indices, fixed income, and FX, benchmarked against classical time-series momentum, was proposed (Zhang et al., 2020). Its result, that reward and risk-scaling choices materially affect realized performance, reinforces that algorithm choice cannot be separated from state representation, action design, reward definition, transaction costs, and evaluation protocol.

2.7.4. Comparative Evidence and Limitations

Algorithm comparisons and reproducibility. A later move toward reproducible DRL trading pipelines is illustrated by an implementation of several standard algorithms, including DQN, DDPG, PPO, SAC, A2C, and TD3, within common market environments and backtesting workflows (Liu et al., 2020). Its relevance is infrastructural rather than evidentiary: it shows that applied trading studies increasingly require standardized environments, comparable baselines, and explicit market-friction assumptions before algorithm rankings can be interpreted.

These comparisons do not establish a universal ranking of reinforcement learning and traditional strategies. Results depend on the market, sample period, baselines, transaction costs, and evaluation protocol. Traditional strategies remain useful as interpretable baselines. Any advantage attributed to reinforcement learning should therefore be tested under matched conditions.

Overfitting and generalization. In cryptocurrency markets, published deep RL trading strategies were shown to often fail to generalize out-of-sample due to backtest overfitting, motivating a probability-of-overfitting estimator to reject overfit agents before deployment (Gort et al., 2022). The implication is that evaluation methodology (strict temporal splits, multi-seed reporting, realistic cost assumptions) matters as much as algorithm choice.

Reward specification. Reward shaping is one of the central design choices in DRL trading, as a critical survey of cryptocurrency DRL systems documents (Millea, 2021). Misspecified rewards can lead to metric gaming, excessive turnover, ignoring tail risks, or misalignment with actual investor preferences. The Differential Sharpe Ratio (Moody & Saffell, 1998) addresses some of these concerns by targeting risk-adjusted return directly, but does not fully resolve the tension between reward simplicity and the richness of real trading objectives.

The literature reviewed here converges on design lessons rather than a headline result. Several reviewed studies formulate position sizing as a continuous action and apply actor-critic methods (Kabbani & Duman, 2022; Z. Liang et al., 2018; Majidi et al., 2024). These examples show that continuous controls are compatible with portfolio allocation. They do not establish a general preference or superiority over discrete alternatives.

Risk-adjusted reward signals, especially Differential Sharpe Ratio formulations, appear promising for out-of-sample generalization relative to raw return objectives, provided they penalize costs and avoid encouraging excessive turnover. Explicit transaction-cost modeling is a prerequisite for any realistic performance claim at the frequencies considered here, and such claims remain highly sensitive to the evaluation protocol. Validation must use held-out periods, frozen policies, and repeated runs where stochastic training is material. Without these elements, a positive backtest is best interpreted as a proof of concept rather than evidence of a robust trading edge.

Within the representative literature reviewed here, a comparatively underexplored design combination is the joint use of order-book-derived state variables, continuous-control RL, and an explicit microstructure-oriented state design. Much of the applied literature relies primarily on OHLCV or other aggregated price representations, while studies using deeper limit-order-book information remain fewer and methodologically heterogeneous. The present thesis is positioned within that less represented part of the literature by constructing the agent's observation vector from 10-level order book data and by evaluating a continuous-control agent under the modeling assumptions stated in later chapters.

2.8. Implications for This Thesis

The literature reviewed in this chapter motivates the thesis design along five dimensions. First, market microstructure theory and evidence support the use of LOB-derived state variables rather than relying only on bar-level price summaries. Spread, imbalance, order-flow, fair-value, and intraday-regime features each correspond to a distinct mechanism discussed in the microstructure literature.

Second, the reviewed evidence also limits interpretation: these are candidate state variables, not guaranteed trading signals, tested empirically in later chapters.

Third, the comparison with supervised and rule-based approaches motivates direct op-

timization of trading decisions. Forecasting models can be informative, but a separate rule is still required to translate predictions into exposure. In this thesis, the agent instead chooses a continuous portfolio position directly, making the action space part of the optimization problem.

Fourth, the applied RL literature motivates the use of continuous-control actor-critic methods and risk-aware reward design, while also requiring caution about reported performance. Prior studies show that RL trading results are sensitive to transaction costs, benchmark choice, random seeds, and validation protocol (Gort et al., 2022; Henderson et al., 2018; Z. Liang et al., 2018). For that reason, the later empirical chapters separate training reward from financial evaluation metrics and assess the learned policy on held-out data.

Finally, the contribution of the thesis follows from the combination of these design choices: a microstructure-aware state representation, continuous exposure control, transaction-cost-aware evaluation, and explicit robustness assessment within a controlled single-asset HFT simulation. The objective is not to claim a deployable trading system, but to test whether this particular design combination can produce credible out-of-sample evidence under the stated simulation assumptions.

3. REINFORCEMENT LEARNING

The formal RL framework comprises states, actions, rewards, policies, and value functions. Financial environments make each component challenging; Chapter 4 specifies the features, reward, and network architectures used here. Two algorithmic distinctions (model-free versus model-based and on-policy versus off-policy) then frame the progression from early value-based methods to TD3 for continuous-control trading.

3.1. Components of a Reinforcement Learning System

Reinforcement learning systems address sequential decision-making problems by selecting actions that maximize cumulative discounted future rewards. This subsection defines the core components using trading as the primary illustrative context, following the standard treatment of reinforcement learning (Sutton & Barto, 2018).

Formally, the trading problem can be described as an approximate Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition kernel, $R(s, a, s')$ is the reward function, and γ is the discount factor. In financial markets, this formulation is necessarily an approximation. The agent observes engineered order-book features rather than the full market state, so the empirical problem is better understood as a partially observed decision problem represented through an approximate MDP. Relevant information may be omitted because of hidden liquidity, unobserved order-flow history, latent intraday regimes, queue priority, and other market participants' private decisions. The transition process is also affected by non-stationarity and intraday seasonality. The state representation used in this thesis should therefore be understood as an attempt to construct a useful decision state under partial observability, not as proof that the market itself is fully Markovian.

Environment. The environment defines the space of possible states and the rules governing transitions between them. In the trading setting studied here, the environment is the simulated order-book market. At each time step, it receives a portfolio-exposure action from the agent, updates the portfolio value according to price changes and transaction costs, and returns the new state and reward. The environment encapsulates everything outside the agent's direct control (price dynamics, spread, and execution frictions represented by the simulator) under a set of explicit simulation assumptions.

State. The state s_t is a snapshot of the environment at time t that contains sufficient information for the agent to select an action. In the HFT setting, the state is a vector of engineered microstructure features derived from the limit order book: imbalance measures, spread, order flow signals, and temporal encodings. States can be terminal, ending an episode, or non-terminal, continuing the interaction sequence. The Markov property requires that the optimal action depend only on s_t and not on the full history; the feature engineering in Chapter 4 is designed to approximate this property.

Chapter 4.2 discusses how specific microstructure features are selected and normalized to construct a useful decision state under this partial observability.

Action. The action a_t is the decision the agent takes given state s_t. In this thesis the action space is continuous: $a_t \in [-1, 1]$, representing the target portfolio exposure from maximum short to maximum long. The continuous formulation is important because it allows the agent to express varying degrees of conviction through position size, rather than being restricted to a binary buy/sell decision.

Policy. The policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ maps states to actions. A deterministic policy is used: given the same state, the policy always produces the same action. A deterministic policy is chosen here because the evaluation objective is a single exposure decision per state, and consistency is important for interpretability and execution in deployed trading systems. During training, exploration noise is added to the policy output to encourage discovery of profitable behaviors; at evaluation time the policy is used without noise.

The choice between deterministic and stochastic policies involves a trade-off that is particularly salient in trading environments. Stochastic policies can provide more flexible exploration and robustness to uncertainty, but they require selecting between sampling or taking the mode at deployment time. Deterministic policies eliminate this ambiguity but require explicit exploration mechanisms. Chapter 4.5 compares both approaches (PPO's stochastic policy vs. DDPG/TD3's deterministic policy) in the context of the trading problem.

Reward. The reward r_t is a scalar signal returned by the environment after the agent takes action a_t in state s_t. It provides the only learning signal available to the agent — there are no labels, no supervisor. The reward function is therefore a critical design choice: it determines what the agent actually optimizes. In trading, rewards are typically derived from portfolio returns, possibly adjusted for risk and transaction costs. The specific reward formulation used here is

developed in Chapter 4.

Reward misspecification is a fundamental risk in trading applications. A naive reward function that optimizes raw returns without accounting for risk, transaction costs, or tail risk can lead to pathological behaviors such as excessive turnover, overconcentration in volatile positions, or ignoring rare but catastrophic events. The Differential Sharpe Ratio formulation used in Chapter 4.3 addresses this by providing a risk-adjusted learning signal that penalizes volatility while remaining recursively computable for online learning.

Value Function. The value function $V^\pi(s)$ estimates the expected cumulative discounted reward starting from state s under policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s \right] \tag{1}$$

where:

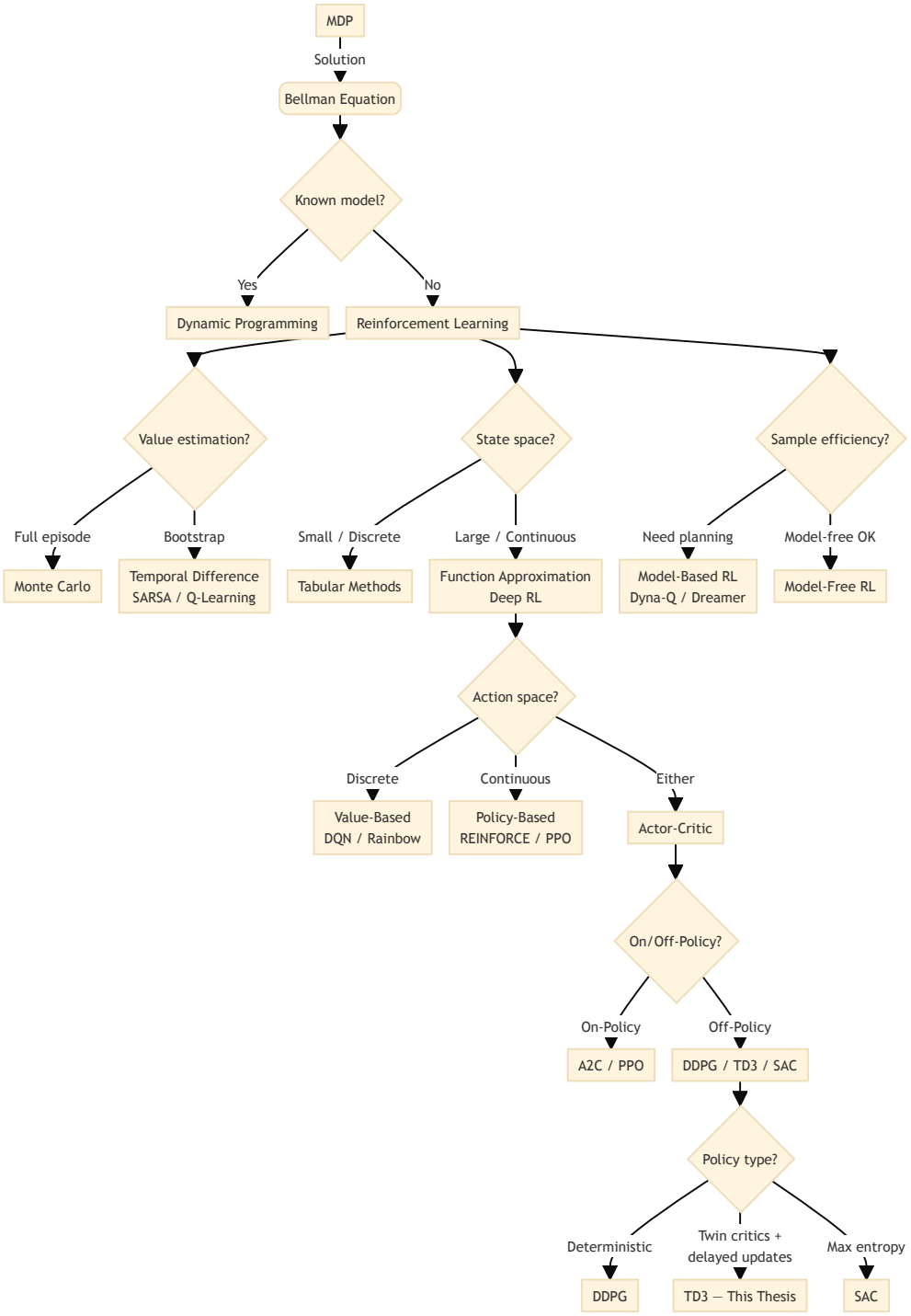
- $V^\pi(s)$ — expected cumulative discounted return starting from state s under policy π
- $\gamma \in [0, 1]$ — discount factor
- r_{t+k} — reward received k steps after t
- $\mathbb{E}_\pi$ — expectation over trajectories generated by following π from $s_t = s$

The value function allows the agent to reason about long-run consequences rather than only immediate rewards. This is a necessary capability in trading, where the cost of a position change today affects future returns through transaction costs and portfolio path dependence. The action-value function $Q^\pi(s, a)$ extends this to state-action pairs and forms the basis of the critic networks in actor-critic algorithms. This cumulative discounted reward term appears explicitly in the RL objective in Equation 16.

3.2. Classifying RL Algorithms

The diagram below maps the reinforcement learning landscape from the Bellman equation through the key algorithmic relaxations that lead to TD3. It is intended as an orientation before the detailed treatment that follows.

Fig. 2. RL algorithm taxonomy from Bellman equation to TD3



Source: Author's own diagram, based on (Sutton & Barto, 2018).[5]

[5] This taxonomy is deliberately simplified, not a precise classification. Categories have fuzzy boundaries: PPO appears under both "Policy-Based" and "Actor-Critic" because it combines a stochastic policy with a value-based critic; DDPG, TD3, and SAC share an "Off-Policy" branch despite fundamental differences in policy type and

3.2.1. Model-free vs Model-based

Reinforcement learning algorithms divide into two broad families depending on whether they require a model of the environment.

Model-based methods learn or are given a transition model $P(s' \mid s, a)$ and use it to plan ahead, computing value functions by rolling out simulated trajectories rather than purely from observed data. Dyna-Q and AlphaZero are canonical examples. When the model is accurate, planning is data-efficient; when the model is wrong, planning amplifies errors. In financial markets, transition dynamics are unknown and non-stationary, making a reliable model practically unattainable.

Model-free methods learn directly from sampled transitions without building a transition model. They are less sample-efficient, and they still rely on the sampled experience being informative for the target environment and on the function approximator generalizing usefully across states and actions. TD3 belongs to this family.

3.2.2. Value-Based, Policy-Based, and Actor-Critic

A second dimension classifies algorithms by what they learn and optimize.

Value-based methods learn a value function and derive a policy implicitly by acting greedily with respect to it. Q-learning and its deep variant DQN are prototypical examples (Mnih et al., 2015). They work well in discrete action spaces but struggle when actions are continuous, because the greedy step $\arg\max_a Q(s, a)$ becomes intractable over an uncountable set.

Policy-based methods directly parameterize and optimize the policy π_θ. REINFORCE updates parameters by gradient ascent on expected return (Williams, 1992):

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}_{\pi_\theta} \left[\sum_t r_t \right] \tag{2}$$

where:

- θ — policy parameters
- $\alpha > 0$ — learning rate
- ∇_θ — gradient with respect to θ

objective. Readers should not infer mutually exclusive algorithmic families from the crisp boundaries shown here.

- r_t — reward at step t

Policy gradients handle continuous actions naturally but suffer from high variance, requiring many samples to obtain a reliable gradient signal. This variance problem motivates the actor-critic architecture, which introduces a critic network to provide lower-variance value estimates.

Actor-critic methods combine both: an actor μ_ϕ selects actions and a critic $Q_\theta(s, a)$ evaluates them, using the critic's lower-variance estimates to guide actor updates. The full actor-critic derivation and TD3's specific extensions are given in Section 3.3.

3.2.3. On-Policy vs Off-Policy

On-policy methods evaluate and improve the same policy used to collect experience. SARSA is a canonical example, updating with:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \tag{3}$$

where:

- $Q(s, a)$ — action-value function
- $\alpha > 0$ — learning rate
- r_t — reward received after taking action a_t in state s_t
- $\gamma \in [0, 1]$ — discount factor
- a_{t+1} — next action drawn from the current policy

On-policy methods tend to be stable but wasteful: each sample is used once and then discarded.

Off-policy methods decouple the data-collection policy (the behavior policy) from the policy being improved (the target policy). Q-learning updates with the greedy maximum:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \tag{4}$$

where:

- $\max_a Q(s_{t+1}, a)$ — greedy value of the next state, taking the maximum over all possible actions a

- remaining symbols are as in Equation 3

This is contrasted with SARSA's update in Equation 3, where the next action is sampled from the current policy rather than taking the maximum. This separation allows experience from a replay buffer, collected under earlier or exploratory policies, to be reused many times, regardless of how a_{t+1} was actually selected. In a financial simulator, however, replay does not create counterfactual observations of how the market would have evolved under actions not actually taken. It is valid only under the maintained simulation assumption that the agent is price-taking and that its exposure choice affects portfolio wealth but not the subsequent order-book path. TD3 is off-policy and uses a replay buffer to reuse collected transitions under this simulator design; the implications for sample efficiency and execution assumptions in financial settings are discussed in Section 3.3.

3.3. Actor-Critic Methods for Continuous Control

Reinforcement learning algorithms have advanced through conceptual breaks, each addressing a limitation of its predecessor. The lineage leading to TD3 isolates the properties relevant to the continuous-action, single-asset trading problem formulated in Chapter 4.

3.3.1. Value-Based Methods and Their Limitations in Continuous Control

The foundational algorithms of the field (Q-learning (Watkins & Dayan, 1992) and its on-policy variant SARSA (Sutton & Barto, 2018)) operate over tabular representations of state-action value functions. Both methods converge to the optimal policy under mild conditions, but only when the state-action space is small enough to enumerate explicitly. In a trading context with continuous price-derived features, the state space is effectively infinite, and a tabular representation is computationally infeasible.

Deep Q-Networks (DQN) (Mnih et al., 2015) resolved the scalability problem by replacing the lookup table with a deep neural network, enabling generalization across states. Two architectural innovations were critical: experience replay, which decorrelates training samples by storing and randomly resampling past transitions, and a periodically frozen target network, which stabilizes the Bellman update by preventing the target from shifting at every gradient step. DQN achieved human-level control across a wide range of tasks, but it inherits a structural

constraint from Q-learning: it requires a discrete action space, because the max operator in the Bellman target presupposes that all actions can be enumerated. For a trading agent that outputs a continuous portfolio weight in $[-1, 1]$, this is not an acceptable limitation. Discretizing the action space into a finite grid resolves the representation issue formally, but it reintroduces the granularity problem. A coarse grid loses the benefits of continuous control, while a fine grid makes the maximization step computationally expensive and recovers the curse of dimensionality in higher-dimensional portfolio settings.

3.3.2. Policy Gradient Methods and the Actor-Critic Architecture

A distinct class of algorithms avoids the maximization problem by parameterizing the policy directly and optimizing it through gradient ascent on expected cumulative reward. REINFORCE introduced this approach (Williams, 1992), and the policy gradient theorem was established in the general function approximation setting (Sutton et al., 2000). The gradient takes the form:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t] \tag{5}$$

where:

- $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_t r_t]$ — expected return objective
- $\pi_\theta(a_t \mid s_t)$ — probability of action a_t under the current policy
- $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ — discounted return from step t
- $\nabla_\theta \log \pi_\theta$ — score function (log-policy gradient)

This generalizes the simpler form shown in Equation 2, which presents the parameter update rule directly rather than deriving it from the gradient of the objective. While REINFORCE applies naturally to continuous action spaces through a parameterized distribution (for example, a Gaussian policy), it suffers from high variance in the gradient estimate. This variance arises because G_t is computed from a full episode trajectory, making it sensitive to trajectory length and reward scale.

The actor-critic architecture reduces this variance by replacing G_t with a learned advantage estimate $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, maintained by a separate critic network. The actor updates the policy in the direction of higher-valued actions, while the critic provides a lower-

variance baseline. This separation is the architectural foundation of all subsequent continuous-action algorithms, including TD3, which instantiates this architecture with deterministic policies and twin critics as described in Equation 21 and Equation 20.

3.3.3. DDPG: Deterministic Policies for Continuous Control

The deterministic policy gradient (DPG) theorem was established, showing that for a deterministic policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ the policy gradient takes the form (Silver et al., 2014):[6]

$$\nabla_\phi J(\phi) = \mathbb{E}_s \Big[\nabla_a Q_\theta(s,a)\big|_{a=\mu_\phi(s)} \cdot \nabla_\phi \mu_\phi(s) \Big] \tag{6}$$

where:

- $J(\phi)$ — expected return
- $Q_\theta(s,a)$ — critic (action-value function) parameterized by θ
- $\mu_\phi(s)$ — deterministic actor
- $\nabla_a Q_\theta$ — gradient evaluated at the action produced by the current policy

This deterministic gradient is a specialization of the stochastic policy gradient in Equation 5 for policies that output a single action rather than a distribution. TD3 uses this gradient form as shown in Equation 21.

Deep Deterministic Policy Gradient (DDPG) was then introduced (Lillicrap et al., 2016), applying the DPG theorem (Silver et al., 2014) to deep neural network function approximators and replacing the tabular actor and critic with multi-layer networks. Rather than sampling an action from a distribution, the actor outputs a single action $\mu_\phi(s)$ directly, making the gradient computation tractable over a continuous action space without the intractable arg max required by value-based methods.

DDPG inherits the experience replay and target network mechanisms from DQN, making it an off-policy algorithm and therefore sample-efficient. However, three failure modes in DDPG were identified (Fujimoto et al., 2018). Independent reproducibility work separately documents that DDPG's benchmark performance is highly sensitive to random seed, codebase,

[6] The original DPG paper uses θ for the actor and w for the critic (Silver et al., 2014); the DDPG paper uses θ^μ and θ^Q (Lillicrap et al., 2016). This thesis adopts the TD3 convention throughout (ϕ for the actor and θ for the critic(s)) to maintain consistency with the TD3 equations in Chapter 4 (Fujimoto et al., 2018).

and hyperparameter choice (Henderson et al., 2018). First, the critic tends to overestimate action values because actor optimization can exploit positive approximation errors in the critic, and bootstrapped targets can propagate those overestimated values through subsequent updates. Overestimated Q-values then push the policy toward actions that exploit the critic's errors rather than the true reward signal. Second, the algorithm is sensitive to the relative frequency of actor and critic updates: updating the actor before the critic has converged amplifies the overestimation problem. Third, a deterministic policy can overfit to narrow peaks in the value estimate, so that small errors in the critic's value landscape are exploited as if they were real returns. In financial environments, where rewards are noisy and non-stationary, one would expect these instabilities to be particularly acute. The critic must separate genuine market signal from approximation artifacts, and overestimated Q-values provide a corrupted learning signal under conditions where the true reward is already weakly informative.

3.3.4. PPO: On-Policy Actor-Critic with Clipped Objectives

Proximal Policy Optimization (PPO), an on-policy algorithm that balances the size of each policy update against training stability, was introduced in (Schulman et al., 2017). PPO optimizes a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{7}$$

where:

- $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ — probability ratio between the updated and old policies[7]
- $\hat{A}_t$ — advantage estimate
- ϵ — clipping hyperparameter (typically 0.2) limiting how far the new policy may deviate from the old one in a single update

This contrasts with TD3's approach to policy updates, which uses the deterministic gradient in Equation 6 and Equation 21 without explicit clipping constraints. PPO's clipping mechanism provides stability through objective constraints, while TD3 achieves stability through target smoothing and delayed updates.

[7] The original PPO formulation uses $r_t(\theta)$ for this ratio (Schulman et al., 2017), but that symbol is reserved throughout this thesis for the RL reward signal. $\rho_t(\theta)$ is used here to avoid the conflict.

The clipping mechanism prevents excessively large policy updates that would collapse performance, without the second-order machinery of Trust Region Policy Optimization (TRPO) (Schulman et al., 2015), which pursues the same goal through an explicit trust-region constraint. PPO is correspondingly simpler to implement and empirically more sample-efficient than TRPO, while remaining more stable than an unconstrained policy gradient (Sutton et al., 2000). Unlike TD3 and DDPG, PPO is on-policy: it optimizes against the most recent policy and does not use a replay buffer. This trades sample efficiency for stability, which can be advantageous in environments where the value function is difficult to approximate.

PPO also maintains a stochastic policy rather than the deterministic policies used by DDPG and TD3. At each state, the policy outputs a distribution over actions (typically Gaussian), and actions are sampled from this distribution. Exploration is handled naturally through the entropy of the action distribution, which can be explicitly encouraged through an entropy bonus term in the objective:

$$L^{\text{CLIP+ENT}}(\theta) = L^{\text{CLIP}}(\theta) + c_1 \mathcal{H}(\pi_\theta) \tag{8}$$

where:

- $L^{\text{CLIP}}(\theta)$ — clipped surrogate objective from Equation 7
- $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta(a \mid s)]$ — policy entropy
- $c_1 > 0$ — coefficient controlling the strength of the exploration bonus

This entropy-based exploration contrasts with the additive action noise used by DDPG and TD3 in Equation 12.

For trading applications, PPO's stochastic policy provides built-in exploration without requiring explicit noise injection during training, and its clipped objective helps avoid catastrophic performance drops that can occur in high-variance financial environments. However, its on-policy nature means each transition is used only once before being discarded, which requires more simulator transitions for comparable learning. In the present thesis, where the training window is fixed to a specific historical period, this makes on-policy methods less efficient than replay-based alternatives at extracting signal from a bounded simulation budget.

3.3.5. TD3: Addressing the Instability of Deterministic Actor-Critic Methods

TD3 was introduced as a direct response to DDPG's failure modes, proposing three targeted mechanisms (Fujimoto et al., 2018).

Clipped double Q-learning. Two independent critic networks Q_{θ_1} and Q_{θ_2} are maintained, and the Bellman target is constructed using the minimum of their estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1}) \tag{9}$$

where:

- r_t — immediate reward
- γ — discount factor
- $Q_{\theta'_i}$ — two target critic networks with slowly updated parameters θ'_i
- s_{t+1} — next state
- $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$ — target action with smoothing noise ϵ_{smooth}

This differs from the standard Q-learning target in Equation 4 through two key modifications: (1) the minimum operator over two critics reduces overestimation bias, and (2) the deterministic target action $\tilde{a}'$ replaces the maximum over actions. The TD3 instantiation in Chapter 4.4 uses the same formulation, shown in Equation 20.

Taking the minimum provides a conservative estimate of target value and substantially reduces overestimation bias. This is analogous to the original double Q-learning correction (Hasselt, 2010), adapted to the deterministic actor-critic setting; the deep variant, Double DQN (Hasselt et al., 2016), was found ineffective in this setting (Fujimoto et al., 2018).

Delayed policy updates. The actor and target networks are updated less frequently than the critics (typically once for every two critic updates). This allows the value estimates to stabilize before they are used to compute the policy gradient, breaking the feedback loop between an unstable critic and an erroneous actor update.

Target policy smoothing. When computing Bellman targets, noise is added to the target action $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$, with ϵ_{smooth} clipped to a small interval. This prevents the critics from fitting sharp value peaks caused by function approximation, acting as a regularizer on the value landscape.

Together, these three modifications produce an algorithm that is substantially more reliable than DDPG on standard continuous control benchmarks (Fujimoto et al., 2018). Evidence from trading-adjacent settings is more limited and heterogeneous. Several deep RL algorithms were compared in asset-holding tasks, with A2C achieving the highest cumulative reward and TD3 and DDPG exhibiting more balanced, lower-turnover trading behaviour (Mohammadshafie et al., 2024); TD3 was also evaluated in cryptocurrency trading with explicit attention to backtest overfitting (Gort et al., 2022). These results should not be read as establishing a general ranking: they span different asset classes, reward formulations, and evaluation protocols. They do suggest, however, that TD3's stability mechanisms are not systematically harmful in financial settings. This matters for the present thesis for a specific reason: the agent is trained in a price-taking historical-market simulator where each transition can be replayed because the agent's action changes portfolio wealth but not the subsequent order-book path. Replay therefore improves computational use of the simulated experience, but only under this execution assumption. It should not be interpreted as evidence that the agent has observed all counterfactual market reactions to alternative trades.

TD3 is selected on the basis of two constraints imposed by the thesis design and one preference that follows from them.

The action space is continuous: the agent outputs a target portfolio exposure in $[-1, 1]$. This eliminates the DQN family entirely, since a discretized action grid would make position sizing a modelling artifact rather than a learned decision. The viable candidates are therefore restricted to policy-gradient and actor-critic algorithms. The training window is also bounded by a fixed historical period, with live data collection outside the thesis's scope. Off-policy algorithms that reuse transitions through a replay buffer extract more signal from each simulated step than on-policy methods, which disadvantages PPO relative to DDPG and TD3.

Given these constraints, the viable candidates are DDPG, TD3, and SAC (Haarnoja et al., 2018). DDPG and TD3 are preferred over SAC because they use natively deterministic policies. This matches the evaluation objective of a single exposure decision per state and avoids the choice between sampling from or taking the mode of a stochastic distribution at deployment.

SAC is excluded on methodological scope grounds rather than irrelevance. Its entropy-temperature coefficient introduces an additional objective dimension (trading off return against policy entropy) that is absent from the return-only formulation used here. This entropy-augmented

framework has been applied to trading problems in the literature (Liu et al., 2020). Comparing SAC to TD3 would require disentangling whether any improvement comes from the maximum-entropy objective or from the off-policy replay both algorithms share, a separate comparison outside this thesis's focus on value-estimation corrections for deterministic policies. SAC remains a promising direction for future work, particularly its online variants for regime shifts in non-stationary markets.

Between DDPG and TD3, TD3 is preferred because it was introduced precisely to correct DDPG's failure modes (critic overestimation, policy-update instability from premature actor updates, and overfitting to narrow peaks in the value estimate). All three are expected to be acute when rewards are noisy and the critic must separate genuine market signal from approximation artifacts. TD3 is therefore not chosen as a generally superior trading method, but as the algorithm whose structural properties (off-policy replay, deterministic actor, explicit overestimation corrections) most directly match this thesis's simulation design.

This choice should be read together with the trading constraints discussed in Chapter 2, which motivate continuous exposure control, off-policy learning from historical data, and caution about transaction costs, non-stationarity, and partial observability. TD3 satisfies the first two through deterministic continuous actions and replay-based learning; it does not solve the remaining market-design problems, which the empirical chapters address separately.

3.3.6. Algorithm Comparison: PPO, DDPG, and TD3 for Trading

The preceding sections establish how PPO, DDPG, and TD3 differ in learning paradigm, policy type, and stability mechanism, and why TD3 is selected under this thesis's constraints. Table 1 in Chapter 4 summarizes these differences at the implementation level (actor output, exploration noise, target smoothing, and update frequency) as configured for the controlled comparison in Chapter 6, where all three algorithms are evaluated on identical trading tasks.

4. DESIGN OF THE TRADING AGENT

The RL framework from Chapter 3 defines seven components of the single-asset HFT agent: action space, state representation, reward function, value function, policy, discount factor, and step size. The trading problem's structure and constraints motivate each choice.

4.1. Action Space

The action space defines the set of permissible decisions available to the trading agent at each time step. This design choice fundamentally shapes the agent's behavior and determines the granularity of control over portfolio positions.

4.1.1. Continuous vs Discrete Actions

Trading systems can be formulated with either discrete or continuous action spaces:

Discrete action spaces restrict the agent to a finite set of categorical decisions. The two most common formulations are the three-action set $\{-1, 0, 1\}$ representing short, neutral, and long positions, and the two-action set $\{0, 1\}$ representing only short and long positions without a neutral option.

Continuous action spaces allow the agent to output any value within a specified range, enabling smooth portfolio allocation decisions.

4.1.2. Implementation: Box Portfolio Action Space

In this implementation, the action space is defined as a continuous interval:

$$\mathcal{A} = [-1, 1] \tag{10}$$

where the action $a_t \in \mathcal{A}$ at time t represents the target portfolio allocation for the traded asset. The semantic interpretation is:

- $a_t = 1.0$: Maximum long exposure (fully invested long position)
- $a_t = 0.0$: Neutral position (no exposure, cash equivalent)
- $a_t = -1.0$: Maximum short exposure (fully invested short position)
- $a_t \in (0, 1)$: Partial long position scaled by magnitude

- $a_t \in (-1, 0)$: Partial short position scaled by magnitude

This continuous action space is required for deterministic policy gradient methods like DDPG and TD3, which are designed for problems where the agent outputs a single action rather than choosing from a discrete set. The policy that maps states to actions in this space is formalized in Equation 23.

This formulation supports short-selling and is a standard simplification in single-asset RL trading environments applied to equity data. The simulation treats long and short positions as cost-symmetric: the transaction cost term $c \cdot |a_t - a_{t-1}|$ applies identically regardless of direction. In practice, short positions in these equities incur securities-lending borrow costs that are not captured here; for large, liquid general-collateral names such as these, documented borrow fees are on the order of tens of basis points per year on the notional short (D'Avolio, 2002). For strictly intra-session episodes, the per-tick borrow cost is negligible: even at a deliberately conservative 50 bps annualized and a 10-second average hold, the cost is on the order of 10^{-7} of notional. The cost-symmetry assumption is therefore inconsequential provided episodes do not span overnight. Where the agent holds short positions across sessions, reported returns represent an upper bound on achievable short-side performance. The continuous nature enables the agent to express varying degrees of conviction: for example, $a_t = 0.3$ indicates a moderately bullish stance, while $a_t = -0.8$ signals strong bearish conviction.

4.1.3. Action Constraints and Transaction Costs

The action space is bounded to prevent unrealistic leverage. In practice, unbounded actions could lead to positions exceeding available capital or margin requirements. The $[-1, 1]$ range implicitly assumes unit leverage without margin multipliers.

Transaction costs are applied proportionally to position changes. If the agent transitions from a_{t-1} to a_t, the turnover magnitude is $|a_t - a_{t-1}|$, and fees are calculated as:

$$\text{fee}_t = c \cdot |a_t - a_{t-1}| \cdot \text{notional value} \tag{11}$$

where:

- c — transaction cost rate (trading fees per unit of notional turnover), matching the penalty term in Equation 15

This cost term appears in the reward function as the penalty $c \cdot |a_t - a_{t-1}|$ shown in Equation 15. The agent must learn to balance expected returns from position adjustments against this turnover cost. This incentivizes the agent to minimize unnecessary portfolio rebalancing, as frequent adjustments erode returns. During training, the RL algorithm must learn to balance the potential gains from reallocation against the friction costs incurred.

4.1.4. Action Smoothing and Exploration Noise

During training, exploration is introduced through additive Gaussian noise:

$$a_t^{\text{noisy}} = \text{clip}(a_t + \epsilon, -1, 1) \tag{12}$$

where:

- $\epsilon \sim \mathcal{N}(0, \sigma^2)$ — Gaussian exploration noise
- σ — exploration noise standard deviation[8]

DDPG and TD3 use this mechanism during data collection; at evaluation they apply the unperturbed deterministic policy. PPO instead obtains action diversity from its stochastic policy and entropy bonus (Equation 8). Clipping keeps the exploratory actions within the valid bounds.

TD3 separately applies target policy smoothing during critic updates, as specified with the policy implementation in Section 4.5.1.1.

4.2. State Space

The state space defines the informational representation available to the agent at each decision point. At each time step t, the agent receives an observation vector $s_t \in \mathcal{S}$ constructed from the current state of the limit order book (LOB). The design of this observation is the primary engineering decision in the empirical pipeline: the agent can only exploit structure that is present in its input.

[8] The main experiment uses $\sigma = 0.3$. This value is at the upper end of the 0.1–0.3 range commonly used in continuous-action TD3 applications and was chosen to maintain sufficient action diversity in the low-signal tick-level setting.

4.2.1. Design Requirements

An effective state representation for high-frequency trading must satisfy several criteria simultaneously.

Approximation of the Markov property. The TD3 algorithm assumes that s_t contains sufficient information such that the optimal action depends only on the current state, not on the full history. Raw price levels violate this assumption because their unconditional distribution is non-stationary; a price of 180 means something very different depending on recent trajectory. Engineered features (returns, imbalances, normalized spreads) are designed to be more stationary and to encode the local market state rather than the absolute price level.

Informativeness about short-term price direction. For the reward signal to be learnable, the observation must contain features that are predictive of near-term mid-price changes. The selection is therefore grounded in market microstructure theory, which identifies specific LOB quantities (imbalances, order flow, spread) as the empirically dominant predictors of short-term price movement.

Dimensionality discipline. Each additional observation dimension enlarges the input space that the policy and critic networks must generalize over. Features are included only when they carry a distinct theoretical mechanism not already captured by other features in the set.

Scope and timescale consistency. The observation is restricted to LOB-derived features. Bar-based signals such as OHLCV aggregates or technical indicators require a fixed aggregation window, imposing a coarser timescale than the tick-level decisions being made. A one-second bar collapses dozens of individual order-book events into a single price summary, discarding the queue dynamics, imbalance shifts, and spread fluctuations that constitute the decision-relevant information at this resolution. Cross-asset signals such as VIX or index returns face a related problem. They update at frequencies far below the inter-event times of the constituent tick streams, contributing no new information between consecutive book updates. Aligning multiple tick streams at sub-second precision also introduces synchronization risk, since a VIX observation and a LOB snapshot sharing a timestamp to the second are not the same market moment.

Beyond these practical constraints, restricting the state to single-asset LOB features is a deliberate experimental choice. The thesis tests whether microstructure-aware features alone are sufficient for a viable agent, since adding cross-asset signals would answer a different question

and prevent clean attribution of any observed performance to LOB structure. Augmenting the observation with synchronized cross-asset regime indicators once baseline LOB-only performance is established is a natural extension left for future work (Chapter 7).

4.2.2. Feature Groups and Selection Rationale

The feature registry organises the LOB candidate pool into five groups, each targeting a distinct microstructure mechanism. **Imbalance features** measure the relative weight of resting liquidity on each side of the book. This is computed both at individual levels (levels 0–2) and in a three-level volume-weighted aggregate, together with order count imbalance, which is sensitive to fragmentation that a single large institutional order would mask (Biais et al., 1995; Cao et al., 2009). **Top-of-book price-adjustment features** summarize displayed queue imbalance relative to the midpoint. The central measure is the queue-weighted midpoint (Stoikov, 2018), implemented under the legacy feature name `hft_microprice`. It is complemented by its divergence from the midpoint and a volume-weighted midpoint skew across three levels. **Spread and cost features** capture adverse selection risk through the bid-ask spread in basis points (Glosten & Milgrom, 1985), its ratio to a 50-event rolling mean, book convexity on each side (Bouchaud et al., 2002), and depth slopes that proxy market impact per unit of volume (Kyle, 1985). **Order flow features** measure net directional pressure from LOB events between consecutive snapshots: order flow imbalance (Cont et al., 2014) and its 50-event rolling sum, queue depletion on each side, signed trade flow from the tape (C. M. C. Lee & Ready, 1991), and odd-lot trade ratio and imbalance. Odd-lot activity separates retail and algorithmic small-lot activity from institutional flow, signalling changes in adverse selection risk.[9] **Regime and temporal features** condition the agent on activity rate via log inter-event time (Engle, 2000), on price momentum via mid-price acceleration, and on the intraday seasonal pattern via cyclical sine and cosine hour encodings (Wood et al., 1985). The final selected TD3 configuration uses a reduced subset of this candidate pool: best-level book pressure, three-level order-book imbalance, best-level order-count imbalance, the queue-weighted midpoint and its divergence (under legacy

[9] Odd-lot activity as a retail/institutional separator is a distinct empirical question from the price-improvement-based retail-identification method (Boehmer et al., 2021); the latter is not direct evidence for the odd-lot mechanism used here. Odd-lot share is computed from the February 25–27, 2026 MBP-10 dataset across AAPL, MSFT, TSLA, META, AMZN, and AVGO and should not be interpreted as a long-run population statistic. See Chapter 5.1 for the MBP-10 format definition.

`microprice` feature names), bid and ask depth slopes, instantaneous OFI, 50-event rolling OFI, and 50-event signed trade flow. The observation vector is additionally extended at runtime with the agent's current portfolio exposure $a_{t-1} \in [-1, 1]$, making the state partially Markovian with respect to inventory and allowing the agent to condition position changes on the transaction cost they would incur. This feature is excluded from z-score normalization because its bimodal distribution, concentrating near ± 1 under the continuous action space, makes standardization counterproductive relative to the raw bounded signal.

The candidate pool from which these groups were drawn contained over forty computable statistics. Features eliminated by the IC/ICIR filter or excluded on redundancy grounds include VPIN (Easley et al., 2012), cancel-to-trade ratio, trade arrival rate, OFI autocorrelation, large trade ratio, depth ratio, and price vamp. VPIN proved collinear with rolling OFI after conditional screening. Cancel-to-trade ratio and trade arrival rate carried insufficient incremental ICIR beyond order count imbalance. OFI autocorrelation's predictive content was subsumed by the rolling OFI window. Large trade ratio was redundant with the odd-lot decomposition. Depth ratio and price vamp did not survive the theory-constrained Stage 1 screen. Exact formulas, citations, parameter values, and model-inclusion status for both selected and excluded features are catalogued in Appendix A.

4.2.3. Observation Space and Normalization

The complete observation vector $s_t \in \mathbb{R}^d$ is formed by concatenating all engineered features. The main selected-feature TD3 experiment uses ten static LOB-derived features plus the runtime position feature, giving $d = 11$.

The ten static features selected in the main TD3 configuration are standardized with causal running statistics:

$$\tilde{x}_{i,t} = \frac{x_{i,t} - \mu_{i,t}}{\sqrt{\sigma^2_{i,t} + \varepsilon}} \tag{13}$$

where:

- $x_{i,t}$ — raw value of feature i at step t
- $\mu_{i,t}$ — running mean estimated from observations up to t
- $\sigma^2_{i,t}$ — running variance estimated from observations up to t

- $\varepsilon > 0$ — small constant for numerical stability

The mean $\mu_{i,t}$ and variance $\sigma^2_{i,t}$ are updated online from observations available up to timestep t within the current chronological split and trading session. The running statistics are reset at detected session boundaries, preventing previous-day market conditions from contaminating the opening representation of the next session. When cyclical encodings are used in other feature specifications, they are already bounded in $[-1, 1]$ and require no normalization, as with the position feature described above.

The formal observation space is:

$$\mathcal{S} = \mathbb{R}^d \tag{14}$$

In practice, after standardization, feature values concentrate in $[-3, 3]$ under approximate Gaussian assumptions. The policy and value networks must generalize to out-of-distribution observations that arise during stress episodes or regime shifts not present in the training sample.

4.2.4. On the Exclusion of Raw LOB Columns

Raw price and size columns (that is, the unprocessed bid and ask prices and sizes at each level) were deliberately excluded from the observation vector. Prior work in the deep learning tradition feeds the raw LOB matrix directly to the network (Zhang et al., 2019), allowing convolutional or recurrent architectures to discover structure from data without hand-engineered intermediaries. This is a coherent approach when the model itself is the feature extractor.

In the present setup, however, comprehensive microstructure features with established theoretical mechanisms are the primary input to the agent. Including raw columns alongside them introduces redundancy without adding an interpretable signal. Normalized bid sizes at each level carry substantially the same information as the imbalance features derived from them. Normalized absolute prices at tick frequency are near-meaningless once the local market state is already encoded through queue-weighted-midpoint divergence, OFI, and spread measures. Mixing both paradigms would imply that the engineered features are insufficient, which is inconsistent with the selection rationale described above. The observation vector is therefore composed entirely of theoretically motivated derived features.

4.2.5. Feature Selection Process

Feature selection follows a two-stage procedure that combines theoretical constraints with empirical ranking.

Stage 1: Theory-constrained candidate pool. The LOB feature registry contains over forty computable statistics. The candidate pool is restricted to features that represent a distinct mechanism identified in the market microstructure literature. Each of the five groups (imbalance, fair value, spread, order flow, and regime) corresponds to a specific theoretical construct: queue pressure (Biais et al., 1995; Cao et al., 2009), top-of-book price adjustment (Stoikov, 2018), adverse selection cost (Glosten & Milgrom, 1985; Kyle, 1985), directional flow (Cont et al., 2014; C. M. C. Lee & Ready, 1991), and intraday regime context (Engle, 2000; Wood et al., 1985). Features not grounded in this literature (raw price and size columns, or purely data-driven aggregates without a microstructure interpretation) are excluded from consideration regardless of their statistical properties. This stage prevents data-mining by ensuring that every candidate has a prior economic justification before any data is examined.

Stage 2: IC/ICIR ranking within the theory-constrained pool. Within each group, multiple implementations of the same theoretical concept exist: OFI at different rolling horizons, spread ratios at different windows, imbalance at different depth levels. Theory cannot resolve which implementations are informative or which combinations are redundant; this is an empirical question. Each candidate is scored by its Information Coefficient Information Ratio (ICIR). ICIR is the mean IC divided by the standard deviation of IC over rolling windows of the training split, where IC is the Spearman rank correlation between the feature and the forward log-return proxy. In the pooled multi-symbol design, scoring is performed independently for each of the 18 training files (6 securities × 3 trading days). The per-file scores are then averaged across files before the redundancy filter is applied, so the final ranking reflects cross-symbol, cross-day average predictive content rather than a single symbol's idiosyncratic regime. Scoring and the conditional IC redundancy filter both operate exclusively on the training files so that the evaluation day (March 2, 2026) remains clean for model comparison. The validation IC is computed separately and reported as an out-of-sample stability check but plays no role in selection. Redundancy within the selected set is controlled via a conditional IC filter: after each feature is selected, its linear signal is regressed out of remaining candidates, and ICIR is recomputed on the residuals. A candidate

is included only if it retains incremental predictive content beyond what the already-selected features collectively capture. The full procedure is described in Appendix F.

The result is a state space whose composition is jointly justified: every feature is grounded in theory (Stage 1) and empirically non-redundant within the theory-constrained pool (Stage 2). The two stages address different questions. Theory answers: which mechanisms belong in the state space? IC/ICIR answers: among the available implementations of those mechanisms, which combinations contribute distinct signal?

The selection is operationalized through a registry-driven feature pipeline. Feature names resolve to computation functions at runtime; adding, removing, or reordering features therefore changes the experimental specification without rewriting the feature implementations. The active feature set for the main experiment is the selected causal LOB set summarized in Appendix A: three imbalance features, two fair-value features, two depth-slope features, three order-flow features, and the runtime position feature. Regime and temporal features remain part of the implemented candidate registry but are not used in the selected TD3 specification.

4.3. Reward Function

The reward function determines what the agent actually optimizes. The thesis implements direct optimization of trading performance: the agent observes the market state, selects a continuous portfolio exposure $a_t \in [-1, 1]$, and receives a scalar reward reflecting realized return net of transaction costs. A baseline formulation of this reward is:

$$r_t = a_{t-1} \cdot \frac{P_t - P_{t-1}}{P_{t-1}} - c \cdot |a_t - a_{t-1}| \tag{15}$$

where:

- a_{t-1} — portfolio exposure held during the period
- P_t — price at the end of the period
- P_{t-1} — price at the beginning of the period
- $c \cdot |a_t - a_{t-1}|$ — transaction cost proportional to the size of the position change

The agent learns a deterministic policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ to maximize expected cumulative reward:

$$\max_{\phi} \mathbb{E}_{\mu_\phi} \left[\sum_{t=1}^{T} \gamma^{t-1} r_t \right] \tag{16}$$

where:

- ϕ — actor (policy) parameters being optimized
- T — episode horizon
- $\gamma \in [0, 1]$ — discount factor
- r_t — reward received at step t

This objective matches the standard RL formulation introduced in Equation 1, with the cumulative discounted reward as the quantity being optimized.

In trading, two properties are essential for the reward to work well in practice. The signal should be risk-adjusted so the agent does not maximize returns by taking excessive volatility, and it should be computable recursively so that no look-ahead into future returns is required at each training step. The baseline reward requires refinement at high frequency. The Differential Sharpe Ratio addresses both requirements.

4.3.1. Execution Model and the Signal–Friction Decomposition

Equation 15 separates into a **signal term**, $a_{t-1}\,(P_t - P_{t-1})/P_{t-1}$, and a **friction term**, $c\,|a_t - a_{t-1}|$ (the proportional fee of Equation 11). Throughout this thesis P_t is the **mid-price**, not an executable bid or ask quote, in keeping with the mid-price execution assumption stated in Section 5.2. Baseline runs set $c = 0$, so a buy and a sell settle at the same mid, no spread is crossed, and the friction term vanishes. The reward is then the signal term alone.

The state includes a queue-weighted midpoint and its divergence from the ordinary midpoint. The implementation retains the legacy `microprice` feature name. Because the queue-weighted midpoint is a convex combination of the best bid and ask, its divergence satisfies $|\text{QWM}(t) - \text{mid}(t)| \leq (\text{ask}(t) - \text{bid}(t))/2$ at every tick. This exact algebraic result bounds the instantaneous scale of one input feature. It does not bound a future mid-price change, the relation learned from the full state, the policy's holding period, or the strategy's profit. Consequently, it cannot establish a theoretical ceiling on the edge available to the agent.

The transaction-cost sweep in Table 6 varies c. Policies trained at 0 and 0.01 basis points trade actively. At 0.1 and 1 basis points, the policies instead hold a constant, saturated position.

The sweep therefore shows sensitivity to the cost assumption, but it does not identify a clean break-even fee for a functioning policy. This result qualifies Hypothesis 1. Fixed-path bid/ask repricing provides separate evidence that the frictionless action path does not survive spread-crossing costs (Section 7.2). Retraining under explicit bid/ask execution remains necessary.

4.3.2. Log-Return

The simplest reward is the single-step log-return:

$$r_t = \log \frac{P_t}{P_{t-1}} \cdot a_{t-1} \tag{17}$$

where:

- P_t — mid-price at the end of the current step
- P_{t-1} — mid-price at the previous step
- $a_{t-1} \in [-1, 1]$ — portfolio exposure held during the period

This is the first term of Equation 15 without the transaction cost penalty. At tick frequency, this signal is dominated by microstructure noise. Individual price changes on an MBP-10 order book (defined in Chapter 5.1) are a fraction of the spread. The agent cannot distinguish genuine directional movement from quote flickering in a single step.

Training on raw log-return produces highly variable gradient estimates. It also tends to reward high-turnover behavior regardless of risk.

A related alternative is the Sharpe ratio computed over a fixed rolling window. This approach requires storing the full window of past returns at each step. It also introduces a sharp discontinuity when old returns drop out of it.[10]

Neither property is desirable in an online RL setting.

[10] When the oldest return in a fixed window is unusually large or small, its removal produces an abrupt shift in the estimated mean and variance (and therefore in the Sharpe ratio) that is unrelated to the current action. The agent cannot distinguish this accounting artifact from a genuine change in performance, introducing spurious gradient variance. Exponential weighting in the DSR avoids this because old returns decay continuously rather than dropping off a cliff.

4.3.3. Differential Sharpe Ratio

The Differential Sharpe Ratio (Moody & Saffell, 1998) resolves both issues. It is defined as the first-order approximation of how the current portfolio log-return R_t changes the exponentially weighted Sharpe ratio:

$$D_t = \frac{B_{t-1}\,\Delta A_t - \frac{1}{2}A_{t-1}\,\Delta B_t}{\left(B_{t-1} - A_{t-1}^2 + \varepsilon\right)^{3/2}} \tag{18}$$

where:

- A_{t-1} — exponentially weighted estimate of the first moment of returns (defined in Equation 19)
- B_{t-1} — exponentially weighted estimate of the second moment of returns (defined in Equation 19)
- $\Delta A_t = R_t - A_{t-1}$ — one-step increment of A
- $\Delta B_t = R_t^2 - B_{t-1}$ — one-step increment of B
- $(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}$ — running variance raised to the power $3/2$, stabilized by a small constant ε

This replaces the simple log-return in Equation 17 with a risk-adjusted measure that penalizes volatility. When DSR training is active, $r_t := D_t$ in Equation 16. The DSR reward signal addresses the high-variance problem that affects raw returns, though it does not eliminate the noise concerns at tick frequency.

A_t and B_t are exponentially weighted estimates of the first and second moments of returns:

$$A_t = \eta R_t + (1-\eta)\,A_{t-1}, \qquad B_t = \eta R_t^2 + (1-\eta)\,B_{t-1} \tag{19}$$

where:

- $R_t = \log(P_t/P_{t-1}) \cdot a_{t-1}$ — portfolio log-return at step t[11]
- $\eta \in (0, 1]$ — exponential smoothing parameter

[11] In the implementation, R_t is computed as $\log(\text{NLV}_t/\text{NLV}_{t-1})$, where NLV_t is the net liquidation value of the portfolio. This is equivalent to the formula above when transaction costs are zero. When costs are non-zero, the NLV-based return absorbs transaction costs implicitly, making the implementation more conservative than the analytic expression suggests.

- A_t — running mean of returns
- B_t — running mean of squared returns

At $t = 0$, $A_0 = B_0 = 0$.

The smoothing parameter η controls the effective memory of the estimator: larger η makes the estimate more reactive to recent returns, smaller η averages over a longer history. The effective window length is approximately $1/\eta$ steps. In the empirical experiments, $\eta = 0.01$, giving an effective memory of roughly 100 ticks — long enough to smooth microstructure noise while short enough to adapt to intraday regime shifts.

The DSR has two properties that make it well-suited to this setting. First, it penalizes return volatility intrinsically. A sequence of returns with the same mean but higher variance produces a lower DSR. This discourages the agent from achieving returns through high-variance position flipping.

Second, the DSR is recursively computable from A_{t-1} and B_{t-1} alone. This means no fixed window, no look-ahead, and no discontinuity as old observations expire.

A shaped reward such as DSR is used to stabilize learning. However, it is not equivalent to the financial performance metrics reported in Chapter 6.

The Sharpe ratio, annualized return, and drawdown statistics reported in the evaluation chapter are computed from the realized log-return series. These come from the deterministic policy rollout. They are not computed from the cumulative DSR accumulated during training.

Conflating the two would invalidate the empirical comparison. For example, treating a high training DSR as evidence of a strong Sharpe ratio on the test set would be incorrect.

The DSR approach has four limitations. First, it assumes the first and second moments of returns are approximately stationary over the effective memory window ($\approx 1/\eta$ steps). Intraday trading is well documented to violate this through market opening effects, scheduled announcements, and regime shifts (Cont et al., 2014; Engle, 2000), and exponential weighting only partially compensates. Second, like other variance-based measures, it penalizes upside and downside volatility symmetrically, whereas traders typically care more about downside risk. The Sortino ratio, the Calmar ratio, or utility-based rewards would align better with that preference, but are non-recursive or computationally intractable at tick frequency. Third, it can become unstable when the denominator $(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}$ is small (early in training or

during low-variance regimes), amplifying microstructure noise into extreme reward values. The implementation clamps D_t to $[-10, 10]$, following standard reward-clipping practice (Mnih et al., 2015).[12] Fourth, because the running moments A_t, B_t are not part of the observation, the reward at time t depends on a hidden trajectory of past returns. TD3 samples replayed transitions out of order, so the same state-action pair can yield different rewards depending on when it was collected, violating the stationarity its Bellman backup assumes. This effect is strongest early in each episode, before the EMA moments stabilize after $\approx 1/\eta$ steps. Augmenting the observation with (A_t, B_t) would restore the Markov property and is left for future work.

The DSR was chosen despite these limitations for two reasons. First, its recursive formulation makes it uniquely suitable for online RL, where alternatives such as fixed-window Sharpe or utility-based rewards require storing unbounded history or solving optimization problems at each step. Second, while variance is an imperfect proxy for risk, it provides a tractable baseline that discourages the most pathological high-turnover behaviors. Whether an alternative reward formulation performs better is a question for future work, addressed in Chapter 7.

4.4. Value Function

The state-value function $V^\pi(s)$ and action-value function $Q^\pi(s, a)$ give the expected cumulative discounted return under policy π. TD3 implements the Q-function through design choices that distinguish it from a vanilla actor-critic.

4.4.1. Twin Critics

TD3 maintains two independent Q-networks Q_{θ_1} and Q_{θ_2}. The Bellman target is computed using the minimum of the two estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1}) \tag{20}$$

where:

- r_t — immediate reward
- γ — discount factor

[12] In the DSR scenario, the reward scale is $\kappa = 1$. At tick frequency the empirical variance term dominates $\varepsilon = 10^{-16}$ by several orders of magnitude, so the resulting signal falls naturally in a well-conditioned $\mathcal{O}(1)$ range for the Adam optimiser and Huber critic loss, and no ad-hoc reward rescaling is required.

- $Q_{\theta_i'}$ — two target critic networks with slowly updated parameters θ_i'
- s_{t+1} — next state
- $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$ — smoothed target action with $\epsilon_{\text{smooth}} \sim \mathcal{N}(0, \sigma^2)$ clipped to a small interval

Taking the minimum counteracts the overestimation bias that arises when bootstrapped targets are computed from the same network being updated. This failure mode is documented in DDPG and addressed here following the TD3 formulation (Fujimoto et al., 2018). This formulation extends the standard Q-learning target in Equation 4 by (1) using twin critics to reduce overestimation and (2) employing a deterministic target policy rather than taking a maximum over actions. The TD3 target was first introduced in Chapter 3 in Equation 9.

Both critics are trained by minimizing the TD error against the shared target y_t. The actor is updated using only the first critic's gradient:

$$\nabla_\phi J(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\Big[\nabla_a Q_{\theta_1}(s,a)\big|_{a=\mu_\phi(s)} \cdot \nabla_\phi \mu_\phi(s)\Big] \tag{21}$$

where:

- ϕ — actor (policy) parameters being updated
- $J(\phi)$ — expected return
- $\mathcal{D}$ — replay buffer
- $Q_{\theta_1}(s,a)$ — first critic evaluated at the current policy action $\mu_\phi(s)$
- $\nabla_\phi \mu_\phi(s)$ — Jacobian of the policy with respect to its parameters

This is the deterministic policy gradient theorem (Silver et al., 2014), first presented in the general form in Equation 6. The gradient chain through the critic provides a lower-variance estimate of how policy parameters should change compared to the REINFORCE gradient in Equation 5.

The second critic contributes only through the conservative Bellman target, not directly to the policy gradient.

4.4.2. Loss Function

The critics are trained with Smooth L1 (Huber) loss rather than MSE:

$$\mathcal{L}(\delta) = \begin{cases} 0.5\,\delta^2, & |\delta| \leq 1, \\ |\delta| - 0.5, & \text{otherwise.} \end{cases} \tag{22}$$

where:

- $\delta = y_t - Q_\theta(s_t, a_t)$ — TD error measuring the discrepancy between predicted and bootstrapped value estimates

The linear tail limits the influence of large TD errors that occur during volatile market episodes, providing robustness that MSE does not. This loss function is applied to the TD error derived from the Bellman target in Equation 20.

4.5. Policy

The policy maps the current state to an action. The three algorithms compared here (PPO, DDPG, and TD3) differ in a fundamental property of this mapping. PPO maintains a probability distribution over actions and samples from it, while DDPG and TD3 produce a single deterministic action for each state. This distinction has direct consequences for how each algorithm explores during training, how it behaves at evaluation time, and what the policy gradient looks like. The subsections below describe each architecture, followed by a table that makes the design choices explicit.

4.5.1. Deterministic Policy Architecture

DDPG and TD3 implement a deterministic policy:

$$\mu_\phi : \mathcal{S} \to \mathcal{A}, \qquad a_t = \mu_\phi(s_t) \tag{23}$$

where:

- $\mathcal{S} = \mathbb{R}^{11}$ — observation space for the selected-feature TD3 configuration
- $\mathcal{A} = [-1, 1]$ — continuous action space
- ϕ — network parameters
- s_t — state at step t

- a_t — resulting deterministic action

The policy is a multi-layer perceptron. The input is the $d = 11$-dimensional state vector. The network passes this through two hidden layers and produces a scalar action:

$$s_t \in \mathbb{R}^{11} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^{1} \xrightarrow{\text{Tanh}} [-1,\, 1] \tag{24}$$

Hidden layers use ReLU activations, following the DQN architecture (Mnih et al., 2015); ReLU is generally preferred over saturating activations to mitigate vanishing gradients in deep networks. The output layer applies no activation before the Tanh squashing, which maps the pre-activation scalar to $(-1, 1)$. Tanh is preferred over a hard clip because it is differentiable everywhere: the policy gradient passes through the output non-linearity without discontinuity, and the smooth saturation near the action boundaries provides an implicit regularizer against extreme position changes.

The exported network widths are also recorded in the reproducibility specification in Appendix B.

4.5.1.1. DDPG and TD3 Exploration

During data collection, both algorithms apply the Gaussian exploration rule in Equation 12; the selected TD3 configuration uses $\sigma = 0.3$. At evaluation they instead use the unperturbed action $a_t^{\text{eval}} = \mu_\phi(s_t)$.

TD3 additionally applies independent target-policy smoothing only when computing Bellman targets (Equation 20; Equation 9). Its smoothing noise $\epsilon_{\text{smooth}} \sim \mathcal{N}(0, 0.2^2)$ is clipped to $[-0.3, 0.3]$, preventing the critics from fitting sharp value peaks. DDPG does not apply this regularizer.

Target actor. Both algorithms maintain a slowly updated copy of the actor, $\mu_{\phi'}$, used only when computing Bellman targets:

$$\phi' \leftarrow \tau\, \phi + (1 - \tau)\, \phi', \qquad \tau = 0.005 \tag{25}$$

where:

- ϕ — online actor parameters

- ϕ' — target actor parameters
- $\tau = 0.005$ — soft-update coefficient controlling how quickly the target tracks the online network

The slow rate ensures the bootstrap target shifts gradually, even while the online actor receives more aggressive gradient updates. The target actor is never used for data collection or evaluation.

Architectural difference between DDPG and TD3. The actor networks for DDPG and TD3 are identical. The differences between the two algorithms lie entirely in the critic and training procedure: TD3 maintains twin critics and uses their minimum for conservative value targets, and delays actor updates to one per every two critic updates. DDPG uses a single critic and updates actor and critic at the same frequency. This makes DDPG the natural control for TD3: it isolates the effect of clipped double Q-learning, delayed policy updates, and target policy smoothing while holding the policy architecture constant.

4.5.2. Stochastic Policy: PPO

PPO maintains a stochastic policy that outputs a probability distribution over actions at each state:

$$\pi_\theta(\cdot \mid s_t) = \text{TanhNormal}(\mu_\theta(s_t),\, \sigma_\theta(s_t)) \tag{26}$$

where:

- $\mu_\theta(s_t)$ — state-dependent mean produced by the actor network
- $\sigma_\theta(s_t) > 0$ — state-dependent standard deviation produced by the actor network
- TanhNormal — Gaussian distribution whose samples are squashed through tanh to lie in $(-1, 1)$

The actor network shares the same hidden-layer structure as the deterministic actors (two layers of width [128, 64]) but its output head differs. The final linear layer produces $2n_{\text{act}} = 2$ values, which are split into a mean μ and a raw scale parameter $\log \sigma$ by a parameter extractor. The scale is transformed to a positive standard deviation via a softplus non-linearity, and a TanhNormal distribution is constructed:

$$s_t \in \mathbb{R}^{11} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{Tanh}} \mathbb{R}^{64} \xrightarrow{\text{Tanh}} \mathbb{R}^{2} \xrightarrow{\text{split}} (\mu_\theta(s_t),\ \log \sigma_\theta(s_t)) \tag{27}$$

The resulting $(\mu_\theta(s_t), \log \sigma_\theta(s_t))$ pair parameterizes the TanhNormal policy distribution defined in Equation 26 above.

The PPO actor uses Tanh activations in the hidden layers, following standard practice for actors with stochastic output heads where bounded hidden representations improve numerical stability of the log-probability computation. The actor contains approximately 9,900 parameters (the output dimension doubles the final layer relative to the deterministic actors).

Exploration. Exploration is handled naturally through the entropy of the action distribution. An entropy bonus $c_1 = 0.01$ is added to the PPO objective to encourage the policy to maintain action diversity during training. No explicit noise injection is required.

Evaluation. At evaluation time, stochastic sampling is replaced by the mode of the TanhNormal distribution, which is $\tanh(\mu_\theta(s_t))$. This yields a deterministic action for each state, directly comparable to the DDPG and TD3 evaluation policies.

Value network. Unlike DDPG and TD3, whose critics estimate $Q(s, a)$, PPO's critic estimates the state value $V_\phi(s)$.[13] The input is therefore the state vector alone (11 dimensions), not the state-action concatenation. The critic architecture is otherwise identical: two hidden layers of width [128, 64] with Tanh activations, followed by a linear output layer producing a scalar value estimate.

4.5.3. Controlled Comparison Design

All three algorithms are trained with the same state representation, reward function, data split, random seed, learning rates, and weight decay. The network capacity is equalized: both actor and value networks use hidden dimensions [128, 64] across all three algorithms. This ensures that differences in empirical performance in Chapter 6 are attributable to the algorithmic properties (policy type, update rule, and stability mechanisms) and not to differences in model capacity or input features.

The table below summarizes the policy-level design choices for each algorithm in the controlled comparison.

[13] In the PPO context, ϕ parameterises the value network V_ϕ, whereas for DDPG and TD3 it parameterises the actor μ_ϕ. The PPO actor is parameterised by θ throughout.

Table 1. Policy design choices for each algorithm in the controlled comparison.

Property	**PPO**	**DDPG**	**TD3**
Policy type	Stochastic	Deterministic	Deterministic
Actor output	TanhNormal(μ, σ)	$\mu_\phi(s) \in [-1, 1]$	$\mu_\phi(s) \in [-1, 1]$
Hidden dims	[128, 64]	[128, 64]	[128, 64]
Hidden activation	Tanh	ReLU	ReLU
Output activation	— (distribution)	Tanh	Tanh
Exploration	Entropy bonus	Additive Gaussian	Additive noise
	$c_1 = 0.01$	noise	$\sigma = 0.3$
Target smoothing	None	None	$\sigma = 0.2$, clip ± 0.3
Evaluation action	$\tanh(\mu_\theta(s_t))$	$\mu_\phi(s_t)$	$\mu_\phi(s_t)$
Critic input	s only	(s, a)	(s, a)
Number of critics	1	1	2 (twin)
Actor update freq.	Every epoch	Every step	Every 2 critic steps

Source: Author's own elaboration.

4.6. Discount Factor

The main TD3 HFT experiment uses a discount factor of $\gamma = 0.9$. Chapter 3 defines its role in the value function.

With $\gamma = 0.9$ the effective planning horizon is approximately $1/(1 - \gamma) = 10$ steps. On the event-time HFT data used here, this short horizon intentionally emphasizes local order-book dynamics rather than distant price movements. This is appropriate for a Differential Sharpe Ratio reward computed from noisy tick-level portfolio changes, where long bootstrap horizons can make critic targets unstable.

$\gamma = 0.9$ is held constant for the main selected-feature TD3 specification to isolate the effect of the state representation and reward design. Sensitivity to γ remains a natural direction for future work, particularly whether higher values such as 0.95–0.995 improve performance when the objective is less local than the present HFT setup.

4.7. Optimization Hyperparameters

4.7.1. Learning Rates

Both actor and critic use Adam (Kingma & Ba, 2014) with a learning rate of $\alpha = 0.0001$. This is deliberately conservative relative to the 3×10^{-4} default common in continuous-control benchmarks. Preliminary experiments with more aggressive rates produced Q-value explosions and policy collapse. These failure modes are consistent with the high noise-to-signal ratio of tick-frequency rewards and the sensitivity of bootstrapped TD targets to large gradient steps. Stability was prioritized over sample efficiency.

Actor and critic use the same nominal rate, but TD3's policy delay (every 2 critic updates) effectively halves the actor's update frequency, providing additional stability without a separate hyperparameter.

4.7.2. Weight Decay

L2 weight decay of $\lambda = 2 \times 10^{-6}$ is applied to the critic networks, while the actor uses no explicit weight decay. The critic regularizer is intentionally small, sufficient to discourage memorization of training-period noise without restricting the network's capacity to represent nonlinear market dynamics. No further regularization (dropout, batch normalization) is used, keeping the architecture parsimonious.

4.7.3. Scope

The reported learning rate and weight decay define the experimental settings used throughout the experiments. They were chosen to maintain stable critic targets and avoid policy collapse in preliminary runs, which makes them appropriate for the present high-frequency training setup. A broader hyperparameter study could refine these values further, but that analysis lies beyond the scope of the current empirical comparison.

5. IMPLEMENTATION OF THE TRADING AGENT

The experimental pipeline implements the Chapter 4 design through its simulation architecture, data preparation, code organization, and TD3 training workflow.

5.1. Simulation Architecture

The empirical workflow is organized through three functional layers that together implement a single-asset trading simulation.

Market data and feature construction. The agent observes states derived from high-frequency order-book data across six Nasdaq-listed equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO). Raw order-book information is transformed into a vector of microstructure features. This layer is responsible for loading and validating the market dataset, preserving chronological ordering, constructing derived features, separating learning features from the price series used for portfolio valuation, and creating train, validation, and test splits without leakage (see Section 5.3.4). Reinforcement learning in financial settings is highly sensitive to timestamp errors and inconsistencies between engineered features and the price series used by the simulation.

Decision layer. The RL agent maps the current state representation to a bounded continuous action interpreted as target portfolio exposure. The agent learns action selection, turnover, and reward trade-offs jointly through environmental feedback. There is no separate forecasting step or labeled-action prior. The quality of the decision is evaluated through the reward process generated by the environment.

Execution and portfolio accounting. The agent's chosen action is translated into a portfolio update under explicit assumptions about pricing, transaction costs, and liquidity. This layer abstracts the operational complexity of a production trading stack (order management, venue selection, smart order routing, fill handling) into a controlled backtesting mechanism. If the agent performs well in this simulated setting, the result supports a claim about policy learning under the specified assumptions, not a claim that the strategy is immediately deployable without further execution modeling.

Simulation engine. The portfolio accounting layer is built on `tradingenv` (XAI Asset Management, 2024), an open-source event-driven market simulator developed by XAI Asset Management that implements the Gymnasium API, the Farama Foundation's maintained succes-

sor to OpenAI Gym. The library models a broker with configurable transaction fees, handles continuous portfolio rebalancing, and records a full trade history for post-episode analysis. It was integrated into the training pipeline via a custom wrapper (`StreamingTradingEnvXY`) that streams fixed-length episode windows from memory-mapped data files, keeping peak memory proportional to episode length rather than dataset size. One performance modification was applied to the library. The duplicate-timestamp check in the internal `TrackRecord._checkpoint` method was overridden in a subclass to use an $O(1)$ dictionary lookup instead of an $O(n)$ linear list scan, which would otherwise degrade step time quadratically over long evaluation episodes.

5.2. Simulation Assumptions

The following assumptions govern the trading simulation throughout all experiments.

Zero market impact and no order-book interaction. Agent trades are assumed not to move quoted prices, consume displayed depth, or alter subsequent order-book states. The environment therefore treats the agent as price-taking rather than market-shaping. This is defensible only while position sizes remain small relative to displayed order-book depth, which holds for the MBP-10 dataset used here (defined in Chapter 5.1)[14] That size argument covers price impact from order size; it does not cover whether the agent's own trading feeds back into the order-flow features it conditions on. Section 7.2 (Section 7.2.1) returns to this.

Mid-price execution. Trades are valued at the mid-price rather than at executable bid/ask quotes. This avoids explicit spread-crossing and simplifies reward accounting, but it makes the reported backtest returns optimistic for the evaluated action path. Real execution may cross the spread and incur maker/taker fees, queue uncertainty, partial fills, and slippage. Section 4.3 (Section 4.3.1) explains this assumption, while Chapter 6 measures how the small per-step edge responds to modeled transaction costs.

Zero latency. Observations are assumed to be acted on immediately, with no transport, decision, or matching-engine delay between state observation and execution. This is appropriate for a study focused on strategy learning rather than low-latency execution engineering. In production HFT systems, such delays can invalidate short-lived signals; Section 7.2 (Section 7.2.1) quantifies this against the dataset's event cadence.

[14] All six instruments (AAPL, MSFT, TSLA, META, AMZN, and AVGO) are highly liquid Nasdaq-listed blue-chips with tight spreads, deep displayed depth, and continuous two-sided quoting. This ensures that simulated agent trades represent a negligible fraction of resting book volume at any level.

Bounded continuous exposure. Position constraints are enforced via the continuous action interval $[-1, 1]$, preventing unbounded leverage.

Transaction fee. A configurable fee parameter is applied per position change. Baseline runs use a no-fee setting; sensitivity experiments vary this parameter.

Narrow data scope. Training uses a three-day window (February 25–27, 2026) across the six instruments, and validation and testing use the two halves of a single held-out session (March 2, 2026). This scope means the agent has only encountered one volatility regime, one spread environment, and one pattern of intraday seasonality. Results should be interpreted as a proof of concept rather than as evidence of generalizability across market conditions or time periods.

These assumptions collectively make the simulation tractable and the results interpretable as learning-dynamics benchmarks rather than deployment-ready performance estimates. The following components of a production trading system are explicitly outside the scope of this study: smart order routing across venues, partial fills and queue-position effects, execution algorithms such as VWAP or TWAP, detailed market-impact modeling, and production monitoring or brokerage integration. These omissions define the gap between the controlled experimental framework and deployment-ready trading-system performance.

5.3. Data Preparation

5.3.1. Asset Selection

The experiments use equity data from six Nasdaq-listed blue-chip instruments: AAPL (Apple Inc.), MSFT (Microsoft Corp.), TSLA (Tesla Inc.), META (Meta Platforms Inc.), AMZN (Amazon.com Inc.), and AVGO (Broadcom Inc.). All six are among the most liquid equities in US markets, with tight spreads, deep displayed depth, and continuous two-sided quoting throughout the trading session. The selection criterion was high but not extreme liquidity: instruments were chosen to ensure defensible price-taking assumptions and rich microstructure signals. Names whose order-book event density on the Nasdaq ITCH feed would be disproportionately inflated by factors unrelated to underlying economic activity were avoided. This liquidity profile ensures that the price-taking assumption embedded in the simulation (that agent trades do not move quoted prices) is empirically defensible. It also ensures that the microstructure features derived from the order book reflect genuine supply and demand rather than thin-market artifacts.

Blue-chip selection also minimizes borrow costs, supporting the action space's cost-symmetric short-selling treatment (Section 4.1): loan supply is abundant for these highly liquid large-caps, keeping the symmetric-cost assumption negligible relative to transaction fees. Had the study focused on smaller-cap or illiquid instruments, borrow costs could reach 5–10% or more, rendering that assumption invalid.

The choice to train a single pooled model across multiple instruments, rather than a separate model per instrument, is a deliberate design decision. Securities within the same asset class that trade on the same exchange, under the same tick-size regime, and with comparable order-flow dynamics share a common microstructure regime. The residual cross-sectional variation (differences in volatility level, beta, or sector exposure) is dominated by this shared structure. Within such a homogeneous class, LOB feature distributions are comparable after standard normalization, making it feasible for a single policy network to learn market-wide order-book dynamics rather than instrument-specific quirks. Pooling across N securities multiplies the effective training sample by a factor of N, reducing the risk of overfitting to idiosyncratic patterns in any single stock's order-flow sequence. It also encourages the model to capture universal microstructure signals (spread dynamics, order-flow imbalance, and depth asymmetries) that transfer across equities within the same class. The feature pipeline supports this directly: all inputs are z-score normalized per security. The microstructure features in the state space (order-flow imbalance, relative spread, depth slope, queue imbalance) are inherently relative measures that carry the same semantic meaning regardless of the underlying price level or absolute volume.

A further consideration is access symmetry. Because all six instruments trade on a centralized, regulated exchange with public pre-trade transparency, all market participants observe the same order book. This contrasts with OTC markets or less liquid instruments where information asymmetry and differential access could confound the learned signal. The goal is to isolate the contribution of the learning algorithm and feature engineering, not to exploit a structural information advantage.

The data were obtained from Nasdaq via the DataBento API, using the MBP-10 (Market By Price, 10 levels) feed. This is a hybrid order book format that sits between L2 (aggregated depth) and L3 (order-by-order) data, providing depth-of-book snapshots at up to ten price levels. It additionally records the order count at each level via `side_ct_xx` fields (e.g., `bid_ct_00`

for the number of orders at the best bid).[15] This granularity is necessary for constructing the microstructure features described in Chapter 4 and listed in Appendix A.

5.3.2. Input Format

The pipeline consumes time-indexed LOB snapshot data. Each row represents an order-book event: a `timestamp` for ordering and temporal features; bid/ask price, resting size, and order count at each book level `NN` (00-09, feeding spread, imbalance, depth, and queue features respectively); and `action`, `side`, and `size` describing the event itself (used for signed trade-flow features). An auxiliary `close` series is used only for reward and portfolio valuation. Unless stated otherwise, all timestamps in this thesis are reported in UTC and all monetary values in US dollars.

Table 2 shows four consecutive events from the AAPL feed at market open on 25 February 2026. The three events span roughly 9 milliseconds, illustrating the sub-millisecond cadence typical of the feed. Each row advances the state of the order book. The first event is a resting limit-order addition on the bid side at a non-top-of-book level. The second is a trade print that consumes displayed size at the best ask. The third and fourth are additions that refresh the ask side after the trade.

Table 2. Four consecutive AAPL MBP-10 events, 25 February 2026.

Times-tamp	Action	Side	Price	Size	Best Bid	Best Ask	Bid Sz	Ask Sz
14:30:00.016	A	B	271.20	200	271.45	271.85	500	756
14:30:00.024	T	N	271.65	21	271.45	271.85	500	756
14:30:00.024	A	A	271.79	140	271.45	271.79	500	140
14:30:00.025	A	A	271.79	100	271.45	271.79	500	240

Source: Author's own elaboration based on DataBento Nasdaq MBP-10 data, AAPL, 25 February 2026.

Action: A = limit-order addition, T = trade print. Side: B = bid, A = ask, N = no aggressor side (trade report). Best Bid/Ask and Bid Sz/Ask Sz reflect the level-0 (top-of-book) snapshot

[15] MBP-10 schema documentation: https://databento.com/docs/schemas-and-data-formats/mbp-10 For a technical reference on market data quality and L3 order book processing, see Nasdaq TotalView-ITCH specification: https://www.nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/NQTVITCHSpecification.pdf

after each event is applied.

5.3.3. Preprocessing and Splitting

Raw data is validated for chronological order, required column presence, non-negative prices, and non-crossed best quotes before any feature engineering. This stage is conservative: its purpose is correctness, not statistical filtering.

Before feature engineering, each daily file is filtered to retain only events that modify at least one of the five shallowest book levels (levels 0–4). The full MBP-10 feed delivers updates at all ten price levels. Every microstructure feature in the state space draws exclusively on levels 0–4: order-flow imbalance, depth slopes, queue sizes, bid/ask slopes, and related signals are all defined relative to the top-of-book region. Events that update only levels 5–9 are therefore invisible to the feature computation; they produce consecutive observations with an identical observable state, inflating episode length and adding computational overhead without contributing any learnable signal. Including them would also implicitly penalise the agent for changes in book depth that it has no mechanism to detect or respond to. The filter is applied immediately after chronological validation and before any feature transformation, so the feature pipeline never encounters these no-signal events.

The dataset is organized as one parquet file per security per trading day, yielding 24 files in total (6 symbols × 4 trading days). The four days cover February 25–27, 2026 (days 1–3, training) and March 2, 2026 (day 4, evaluation). All files are filtered to US regular market hours (09:30–16:00 ET). Raw event counts and inter-event timing per symbol and split, before the level-0–4 filter is applied, are reported in Appendix, Table 12. Event density varies substantially across symbols (from roughly 190,000 to 7.1 million events per split), and this variation carries through to the post-filter split sizes summarized below.

Training data. The 18 files from days 1–3 are used exclusively for training. Each daily file is treated as a complete, independent training unit: the agent encounters all order-book events from that file in one episode. During training, the 18 files are served in uniformly shuffled random order across episodes, so the agent never sees a fixed day-order sequence and cannot exploit calendar artifacts.

Validation and test data. The 6 day-4 files (one per symbol) are held out entirely from training. Each file is split 50/50 at its chronological midpoint into a validation half and a test

half; the six validation halves and six test halves are concatenated separately to form the pooled validation and test sets. This design ensures that evaluation data covers the same six securities but originates from a genuinely unseen trading session, providing a clean out-of-sample window. The validation split supports model comparison and development diagnostics; the test split is reserved for final out-of-sample reporting and does not influence any model selection decisions.

The four-day window spanning 25 February to 2 March 2026 is a recognized limitation of the current empirical design. The agent's learned behavior has not been exposed to different volatility regimes, spread environments, or intraday seasonality patterns beyond this specific period, and the generalizability of the results is bounded accordingly.

Feature pipeline fitting. All ten features in the selected set use online, strictly-causal normalization: the statistics that standardize the observation at step t are accumulated from that session's own events up to $t-1$ only (Section 5.3.4). They also reset at each session break. For this feature set the separate fitting step over the concatenated training days is therefore inert (its output is never consulted at transform time), and the evaluation day is normalized entirely by its own expanding within-session statistics. The causality guarantee is consequently stronger than a train-fit-then-freeze scheme would provide: training-period data cannot leak into the evaluation representation because it does not enter it at all. This was confirmed directly. Normalizing the AAPL evaluation day with a pipeline fitted on a different security (TSLA) reproduces the AAPL-fitted output bit for bit. A prefix-invariance check (normalizing the first 200,000 versus the first 400,000 rows) leaves the overlapping rows unchanged.

5.3.4. Feature Normalization and Causality Preservation

Normalization of microstructure features presents a methodological challenge for sequential learning: the conventional approach of fitting a standard scaler on the full training dataset and applying it globally introduces look-ahead bias. Each observation's normalized value would incorporate statistical information from future timesteps, violating the causal structure that a trading agent must respect in deployment. An agent at step t can only condition on information available at time t. Allowing its inputs to be scaled using the mean and variance of the entire training set (including steps $t+1, t+2, \ldots$) would, in effect, provide the agent with knowledge it cannot access.

To preserve causality, this study employs a **cumulative running normalization** scheme.

For each feature, the mean and variance used to standardize the observation at step t are updated incrementally from the mean and variance at step $t-1$, following Welford's online algorithm (Welford, 1962). This requires no buffer of past observations and no pass over future data, updates in $O(1)$ per step, and remains numerically stable even for the near-empty denominators of early-session observations.

The resulting normalized value is the single-feature case of Equation 13 (Chapter 4), with the feature index i suppressed and the running mean and variance written as $\bar{x}_t, \sigma_t^2$ rather than $\mu_{i,t}, \sigma_{i,t}^2$:

$$\tilde{x}_t = \frac{x_t - \bar{x}_t}{\sqrt{\sigma_t^2 + \varepsilon}} \tag{28}$$

Because $\bar{x}_t$ and σ_t^2 incorporate only observations $1, \ldots, t$, and x_t is available before the action at step t is chosen, this transformation never uses future information.

5.3.4.1. Session-Aware Normalization

The continuous-time nature of equity markets introduces regime discontinuities at session boundaries. Overnight and weekend price gaps, changes in order-flow patterns between the opening auction and continuous trading, and differences in liquidity across intraday periods all represent shifts in the underlying data-generating process. A naive cumulative normalization that spans these boundaries would allow statistics from the previous session to contaminate the current session's representation. For example, a 5% overnight price decline would make the opening trade appear as an extreme outlier when normalized against statistics that include the previous day's stable prices, even though such gaps are expected in continuous-time markets.

To address this, running statistics are **reset at session boundaries**. A session break is defined as a time gap exceeding one hour, which distinguishes short intraday interruptions (e.g., lunch recess) from genuine overnight or weekend closures. At each boundary, $\bar{x}$ and σ^2 are reinitialized to their default values ($\bar{x} = 0, \sigma^2 = 1$), and the accumulation process begins anew for the next session. This prevents the overnight gap from distorting the morning's normalization statistics and ensures that each trading session's features are scaled relative to that session's own empirical distribution.

5.3.4.2. Event-Time vs. Continuous-Time Weighting

An additional design choice concerns whether to weight observations by event count or by elapsed time. In high-frequency trading, events cluster during periods of high market activity: a burst of order-book updates over one second may contain 100 events, while a quiet 10-second interval may contain only one. An event-based normalization (the default in many standard RL libraries) would assign equal weight to each of the 100 clustered events and the one sparse event, effectively over-representing the high-activity period in the statistical summary. Continuous-time markets, however, are not sensitive to the number of events *per se* but to the duration over which each price or quote persists.

This implementation also supports an optional **time-weighted normalization** mode, where each observation is weighted by the elapsed time since the previous event rather than counted equally. The running mean and variance then reflect a time-average of the feature's trajectory instead of an average across discrete events. For the main experiments, event-based weighting is used as the default, matching the standard approach in the RL literature (e.g., Raffin et al., 2021; E. Liang et al., 2018); sensitivity analysis with time-weighted normalization is a direction for future work.

5.3.4.3. Session-Scoped Statistics and Leakage Prevention

For causal running normalization, statistics are maintained **independently per security** and are **reset at each detected trading-session boundary**; they are not reset at the boundaries between the training, validation, and test subsets. Because the accumulator only moves forward in time and never uses observations after the current step, there is no look-ahead: statistics from the test period cannot influence the training representation, and a later observation cannot influence an earlier one. One consequence should be stated plainly. The validation and test sets are the two chronological halves of a single trading session, split at its midpoint. The running statistics are not reset at that midpoint, so the test half's normalization is warm-started by the accumulated statistics of the validation half. This is backward-looking and therefore not leakage in the look-ahead sense, but it does mean the validation and test halves are not statistically independent samples and should not be described as separated by the normalizer. The per-symbol scoping is retained for its original purpose: each security's features are normalized relative to its own microstructure rather than to magnitudes that differ across price levels and liquidity profiles.

Cyclical temporal encodings (hour-of-day sine and cosine) are already bounded in $[-1, 1]$

and are not subject to z-score normalization. Raw price columns used for portfolio accounting are kept in their original units and are never replaced by normalized values. This separation ensures that the observation features passed to the learning algorithm are statistically well-behaved, while the pricing inputs used for reward and valuation computation retain economic interpretability and correct accounting semantics.

5.3.4.4. Transformed Feature Example

The causal running normalization in Equation 28 converts each raw order-book event into the microstructure features consumed by the policy. A twelve-event AAPL window, reported in Appendix, Table 13, illustrates this pipeline end to end: raw best bid/ask prices and sizes alongside the corresponding normalized features. Three properties stand out. **Book Pressure** and **Order Imbalance** capture instantaneous liquidity and directional pressure: positive when bid depth dominates, negative when a bid-side refresh shifts pressure toward the ask (visible at event 12 in the appendix table). **Queue-Weighted-Midpoint Deviation** tracks the static top-of-book adjustment away from the ordinary midpoint. **OFI** captures signed net trade flow over a short window and spikes sharply when displayed depth is consumed or replenished. These four features are a representative subset; the full feature set additionally includes depth slopes, queue depletion rates, VPIN, and rolling order-flow statistics.

5.3.5. Feature–Return Correlations

Table 16 reports Pearson and Spearman rank correlations between each engineered feature and the one-step-ahead log mid-price return, computed over the full streamed training set (six instruments, three sessions, roughly 18.7 million event pairs). Pearson detects linear association; Spearman detects any monotone relationship, including non-linear but rank-preserving ones.

The top-of-book imbalance features carry a modest but genuine marginal signal. Best-level order-count imbalance has a Spearman correlation of 0.26 with the next mid-price move, and best-level book pressure, queue-weighted-midpoint divergence, and instantaneous OFI each sit near 0.17. These values are stable across all six instruments (per-symbol Spearman between 0.14 and 0.33 for order-count imbalance) and carry the sign expected when queue imbalance predicts which side of the book depletes first, the mechanism set out in Chapter 2 (Cont et al., 2014; Stoikov, 2018). The matching Pearson correlations are smaller (≤ 0.16), so the relationship is

monotone rather than linear. Every other feature – fair-value level, spread regime, book convexity, trade intensity, order-flow autocorrelation, and time encodings – has $|r| < 0.02$ on both measures in isolation.

Two things follow for the choice of function approximator. First, even the strongest single feature accounts for under seven per cent of the rank variance in the next return (0.26^2), so no feature is a standalone rule; a usable edge has to come from combining them. Second, a linear predictor – logistic or ridge regression, or any weighted sum of features – would capture only the linear component of the imbalance features and would impose one additive, monotone response per feature across every market regime. The policy instead needs the non-linear shape of the imbalance response and its interaction with spread, depth, and order-flow state, which is what a neural-network approximator can represent. The full per-feature breakdown is in Appendix, Table 16.

5.3.6. Feature and Price Column Separation

A strict separation is maintained between the observation features passed to the learning algorithm and the price inputs used by the portfolio simulation. The learning features are normalized and engineered; the pricing columns are economically interpretable and used only for reward and valuation computation. Mixing these two roles can produce invalid reward calculations or cause normalized prices to appear in financial metrics.

In the current implementation, the auxiliary pricing series used for portfolio valuation plays the role of a mid-price proxy rather than an executable bid/ask quote. This simplifies reward accounting but makes the resulting backtest performance optimistic relative to realistic spread-crossing execution.

5.3.7. Dataset Split Summary

Table 3 summarises the row counts and timestamp ranges of the training, validation, and test subsets for the pooled six-asset dataset used in the primary experiment. Row counts reflect the number of order-book events that pass the level-0–4 filter after chronological validation. No split is truncated: `train_size`, `validation_size` and `test_size` are all unset, so training uses every qualifying event from the three training days across the six instruments, and the single evaluation day is split at its midpoint into validation and test. Counts are reported per split across

all six instruments pooled, not per instrument.

Table 3. Training, validation, and test split sizes and timestamp ranges for the pooled six-asset LOB dataset.

Split	Events	First timestamp	Last timestamp	Mean Δt	Median Δt
train	18,707,348	2026-02-25 14:30:00	2026-02-27 20:59:59	8.52 ms	14.6 µs
val	3,673,436	2026-03-02 14:30:00	2026-03-02 16:53:17	8.87 ms	25.8 µs
test	3,673,439	2026-03-02 16:53:17	2026-03-02 20:59:59	15.27 ms	33.3 µs

Source: Author's own calculations based on DataBento Nasdaq MBP-10 data.

5.3.8. Feature Statistics

For most engineered features, means near zero and standard deviations near one confirm that causal running normalisation operates as intended. The order-flow and book-shape features are markedly heavy-tailed: rare aggressive orders drive pre-clipping z-scores beyond ± 500 and push excess kurtosis into the thousands for the OFI family, but the environment bounds the values received by the policy to $[-5, 5]$. The full distributional breakdown, computed over the complete streamed training set (six instruments, three sessions, roughly 18.7 million events), is reported in Appendix, Table 15.

5.4. Implementation and Training Pipeline

5.4.1. Experiment Specification Design

The full source code for this thesis is publicly available online (Wojdalski, 2026). Each experiment is fully specified by a structured set of inputs covering the dataset, feature subset, reward type, environment settings, and optimization hyperparameters. No source-code changes are required to switch between experimental variants. A pre-training validation stage checks path availability, feature definitions, split feasibility, and column compatibility before any computation begins, preventing wasted runs from invalid experimental specifications. A condensed summary of the main specification is provided in Appendix B. A high-level overview of the full pipeline is provided in Appendix E, spanning data acquisition and offline feature research through preparation,

pre-training configuration checks, training, and post-training evaluation. The complete machine-readable diagram is maintained alongside the source code in `docs/workflow.d2`.

5.4.2. Feature Pipeline

Feature engineering is registry-driven: feature names resolve to computation functions, allowing feature-set composition without editing training code. This design is central to systematic feature subset comparisons. Normalization parameters are estimated on the training split only for non-causal scalers. For the main HFT experiment, features use causal running normalization: each split is transformed chronologically, and the normalization state is reset at detected session boundaries. This preserves causal integrity across both time and the chronological split.

5.4.3. Training Loop

The processed training split is used to construct the trading environment with a continuous action space $a_t \in [-1, 1]$ representing target portfolio exposure. Training follows the TD3 procedure described in Chapter 3, with comparable implementations for DDPG and PPO for experimental comparison. All three algorithms share a common training infrastructure: experience collection, network updates, and evaluation. The primary differences are in the update rules (deterministic policy gradient for DDPG/TD3, clipped surrogate for PPO) and replay buffer usage (off-policy for DDPG/TD3, on-policy for PPO).

For DDPG and TD3, experience is collected into a replay buffer and the critic is updated from sampled mini-batches via the deterministic policy gradient. Target networks track the main networks through soft updates with coefficient τ, and exploration noise is applied during training and removed at evaluation time. For PPO, training follows the on-policy paradigm: batches are collected using the current policy, multiple epochs of updates are performed on each batch, and the clipped surrogate objective constrains policy changes. The stochastic policy handles exploration through entropy bonuses rather than explicit noise injection.

Algorithm 1 Common Experiment Setup

Require: Experiment specification (dataset, feature set, hyperparameters)

Ensure: Processed train/validation/test environments ready for algorithm-specific training

Load and validate experiment specification; check paths and split feasibility

Load L3 order book dataset; sort chronologically

Split data into train / validation / test subsets (chronological)

Initialise and validate feature pipeline on training split

Transform each split chronologically with causal session-aware normalization

Build training environment from processed training data

Algorithm 2 TD3 Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; TD3 hyperparameters

Ensure: Trained policy μ_ϕ, evaluation metrics

Initialise actor μ_ϕ, twin critics Q_{θ_1}, Q_{θ_2}, target networks $\mu_{\phi'}$, $Q_{\theta'_1}$, $Q_{\theta'_2}$, replay buffer $\mathcal{D}$

for each step $t = 1, \ldots,$ `max_steps` **do**

  $a_t \leftarrow \text{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big), \quad \epsilon \sim \mathcal{N}(0,\ \sigma^2_{\text{explore}})$

  Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

  Sample mini-batch $\mathcal{B} \sim \mathcal{D}$

  Compute smoothed target action: $\tilde{a}' \leftarrow \mu_{\phi'}(s') + \text{clip}\big(\mathcal{N}(0, \sigma^2_{\text{noise}}),\ \pm c\big)$

  Compute Bellman target: $y \leftarrow r + \gamma \cdot \min_{i=1,2} Q_{\theta'_i}(s',\ \tilde{a}')$

  Update both critics by minimising $\mathcal{L}_{\text{Smooth-L1}}\Big(y - Q_{\theta_i}(s, a)\Big)$

  if t mod `policy_delay` $= 0$ **then**

    Update actor: $\phi \leftarrow \phi + \alpha \nabla_\phi Q_{\theta_1}(s,\ \mu_\phi(s))$

    Soft-update targets: $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i;\ \phi' \leftarrow \tau\phi + (1-\tau)\phi'$

  end if

  Checkpoint and log metrics at configured intervals

end for

Evaluate deterministic policy μ_ϕ on test split

Compute financial performance metrics from realised return series

DDPG (Algorithm 3, Appendix Section C) follows the same loop as Algorithm 2 with

three simplifications. It uses a single critic in place of the twin critics, so the Bellman target uses $Q_{\theta'}(s', \mu_{\phi'}(s'))$ directly, without the minimum. The actor is updated every step rather than every `policy_delay` critic steps, and there is no target-action smoothing noise. PPO (Algorithm 4, same appendix section) instead follows the on-policy paradigm summarized above: batches are collected with the current stochastic policy, and GAE advantage estimates are computed. The clipped surrogate objective (Equation 7), together with a value loss and entropy bonus, is optimized jointly over multiple epochs per batch, with no replay buffer.

The notation follows the conventions established in Chapter 3:

- μ_ϕ is the deterministic actor; Q_{θ_1}, Q_{θ_2} are TD3's twin critics
- Primed symbols denote slowly updated target networks
- $\mathcal{D}$ is the replay buffer
- $\epsilon \sim \mathcal{N}(0, \sigma^2_{\text{explore}})$ is training-only exploration noise
- τ is the soft-update coefficient
- γ is the discount factor; α is the learning rate

Concrete values are listed in Appendix B; DDPG's and PPO's own symbols are defined alongside their algorithms in Appendix Section C.

5.4.4. Sanity Checks

Before running the empirical order-book experiments, the RL pipeline was tested on a synthetic sine-wave scenario with deliberately learnable structure.[16] The purpose of this check was not to evaluate trading performance, but to verify that the environment, reward computation, replay buffer, neural networks, and training loop were capable of recovering a known pattern when one was present. In this setting, the TD3 agent learned behavior consistent with the imposed sine-wave structure, providing a basic implementation sanity check before moving to noisy tick-level order-book data.

A complementary check was conducted using a random policy, one that selects actions uniformly at random with no learned component. This tests the opposite question: whether the environment, reward function, and accounting logic are free of structural bias that would generate

[16] The sanity-check run can be reproduced from the project root with: `uv run python src/cli.py train -c src/configs/scenarios/sine_wave/td3_hft_lob_future_oracle_combined/`.

returns even without a signal. The random policy produced approximately zero return, consistent with an unbiased implementation. Results for the random policy on the empirical dataset are reported in Chapter 6.

Beyond the ad-hoc checks above, a suite of automated tests guards the implementation against the failure modes most common in RL codebases. Learning health checks verify, after a single gradient update, that observations and rewards are finite and that every feature dimension varies across the batch (i.e., no collapsed or constant-valued feature). They also confirm that the loss is finite and that actor and critic parameters both change and remain finite. These four conditions are individually obvious but together rule out the silent errors most likely to produce plausible-looking training curves without genuine learning. Pipeline integration tests run a short end-to-end training loop to detect regressions in data loading, environment construction, or replay buffer interactions. These tests execute automatically as part of the development workflow, ensuring that changes to any component of the pipeline do not silently invalidate the experimental conditions that produced the reported results.

5.4.5. Evaluation

Final performance is reported on the held-out test split using a dedicated evaluation environment. Training reward and realized portfolio return are kept as two separate output streams: one for optimization diagnostics, one for the financial metrics computed from the deterministic rollout. This separation follows the reasons discussed in Section 4.3. Metrics derived from the test rollout are described in Chapter 6.

6. RESULTS

The thesis-facing experiment evaluates a TD3 agent trained on high-frequency order-book features from six Nasdaq equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO) using the state-space design defined in Chapters 4 and 5. The evidence covers repeated-run variability, robustness to nearby design changes, and final held-out performance.

6.1. Algorithm Comparison Results

Beyond benchmark performance, the key empirical question is how the algorithmic choices in Chapter 3 affect trading results. Comparing PPO, DDPG, and TD3 on identical data, features, and evaluation protocols isolates their practical effects.

6.1.1. Benchmark Strategies

Performance is assessed relative to five benchmark strategies. These benchmarks span passive exposure, execution-style references, and a stochastic baseline, together providing a comparison surface for the directional TD3 agent.

Buy-and-hold maintains a constant fully long position ($a_t = 1.0$) throughout the evaluation period, capturing the full market return of the asset. It is the primary economic benchmark: any active strategy that fails to outperform buy-and-hold on a risk-adjusted basis has not demonstrated useful directional skill beyond passive exposure.

TWAP (Time-Weighted Average Price) distributes position changes uniformly across time. **VWAP (Volume-Weighted Average Price)** distributes them in proportion to the volume actually traded during the evaluation window. Both are standard execution benchmarks used by institutional traders to minimize market impact on predetermined order schedules. It is important to note that TWAP and VWAP are execution algorithms rather than directional strategies. They describe how to work an order, not whether to be long or short. Their inclusion reflects standard practice in trading-system evaluation; however, outperforming them has a different interpretation than beating buy-and-hold, and they serve as lower-bar references rather than directional competitors.

VWAP is normalized by the evaluation window's total realized volume, making it a retrospective reference rather than a rule executable in real time; TWAP is the implementable

execution baseline.

Random selects actions uniformly from $[-1, 1]$ at each step without reference to market state. Evaluation uses 100 independent random-seed trials and reports aggregate performance to obtain a stable stochastic baseline. Failure to consistently outperform a random policy would indicate that the agent has not learned a meaningful relationship between observations and actions.

6.1.2. Comparison Methodology

The exported thesis snapshots report one completed evaluation run per algorithm for this comparison: it should be read as matched experimental evidence on the available exports, not as a multi-seed estimate of the population distribution of outcomes.

6.1.3. Performance Comparison

The table below reports test-split metrics for each algorithm, all trained on identical data, features, reward function, and evaluation protocol.

Table 4. Hypothesis 1 algorithm comparison: test-split metrics across TD3, DDPG, PPO, and Random, under frictionless (mid-price, zero-fee) execution.

Algorithm	Return	Ann. Vol	Max DD	Sharpe	Sortino	Win Rate	Lose Rate	PF	Turnover	% Long	% Short
TD3	304.94%	0.3617	-0.65%	5.035	0.774	20.44%	11.43%	3.413	0.4457	49.84%	50.16%
DDPG	312.78%	0.3530	-0.45%	4.962	88.721	25.10%	15.98%	3.641	0.4014	53.66%	46.34%
PPO	205.55%	0.2687	-0.47%	5.437	0.911	28.41%	19.60%	4.588	0.3634	51.85%	48.15%
Random	0.25%	0.1090	-0.59%	0.029	0.054	22.56%	22.52%	1.006	0.6668	49.96%	50.04%

Source: Author's own calculations.

Note: All figures use mid-price execution with zero transaction cost and represent frictionless gross performance, not net-of-cost performance; transaction-cost sensitivity is in Section 6.2 (Table 6). Every agent is evaluated over the same full per-symbol test window.

Return: cumulative simple return. Ann. Vol: annualized volatility. Max DD: maximum drawdown. Sharpe: mean excess return / standard deviation (not annualized). Sortino: mean excess return / downside deviation (not annualized). Win Rate: fraction of steps with positive return. PF: profit factor (gross wins / gross losses). Turnover: average absolute position change per step. % Long/Short: fraction of steps with positive/negative position.

6.1.4. Per-Instrument Decomposition

Each pooled scenario is evaluated one instrument at a time, so the aggregate above averages six independent evaluations. Reporting only that average would conceal how far the instruments diverge, and, more importantly, would misattribute the reason they do.

Table 5. Hypothesis 1 per-instrument test-split results for the TD3 agent, with the cumulative return restated as a mean edge per decision.

```
No per-instrument Hypothesis 1 results available. Run: uv run thesis-
experiments h1
```

The cumulative-return column spans roughly a factor of twenty, from $+33\%$ on META to $+675\%$ on AMZN, which invites the reading that the strategy works on some instruments and not others. The per-step column shows that reading is wrong. Expressed per decision the six instruments occupy a narrow band (roughly 0.016 to 0.036 basis points per step). The ordering does not match the cumulative one: META, the weakest instrument on cumulative return, carries a *higher* per-step edge (0.021 bp) than AMZN, the strongest (0.018 bp), and AVGO has the largest per-step edge of all (0.036 bp) on only a mid-ranking $+120\%$ cumulative return.

What separates them is the number of decisions the evaluation window afforded. AMZN records 1,128,425 environment steps against META's 136,676, an eight-fold difference in opportunity over the same wall-clock session, because message-level event counts differ across instruments. Compounding a similar per-step edge over eight times as many steps produces most of the gap in the cumulative column. Holding META's per-step edge fixed and applying AMZN's step count would imply a cumulative return near $+980\%$, well above AMZN's own $+675\%$. Cumulative return over a fixed session therefore measures event frequency at least as much as it measures skill, and the per-step figure is the quantity that should be compared across instruments and carried into the transaction-cost analysis.

6.1.5. Key Empirical Questions

The results below are read through the theoretical lens from Chapter 3: whether TD3's overestimation correction improves on DDPG, how PPO's stability trades off against TD3's

efficiency, and which algorithm best balances risk-adjusted return against practical deployment constraints.

6.1.6. Discussion of Algorithmic Trade-offs

The comparison reveals qualitatively distinct behavioral profiles across the three algorithms, even though all three were trained on identical data, features, and reward specification.

All three learners take balanced two-sided exposure. Long-position shares span a narrow band, 49.8% for TD3, 53.7% for DDPG, and 51.9% for PPO, against the random baseline's 50.0%. None commits to a persistent directional bet, so none of the returns reported here is explained by a standing long or short position in a drifting market. All three also turn over at broadly comparable rates, between 0.36 and 0.45 per step, well below the random policy's 0.67, which is what a policy that changes exposure deliberately rather than indiscriminately looks like.

DDPG records the highest cumulative return, 312.8%, with a profit factor of 3.64 and a Sharpe ratio of 4.96. **TD3** is close behind at 304.9%, with a profit factor of 3.41 and the highest per-bar Sharpe of the three, 5.04. **PPO** returns 205.6% on the highest profit factor, 4.59, at the lowest turnover, 0.36. On drawdown the order reverses: DDPG and PPO hold their worst loss to -0.45% and -0.47%, while TD3's -0.65% is the deepest of the learners, driven almost entirely by META (a -3.3% excursion on the instrument with the fewest events). The ordering on cumulative return is therefore not the ordering on risk-adjusted return or on drawdown, and no single arm dominates across the three.

The **random baseline** returns 0.2% with a profit factor of 1.01, which is what noise looks like: gross gains and gross losses almost exactly offset. It does so at the highest turnover, 0.67, and a drawdown of -0.59%. The separation from the learned arms is therefore large and consistent. Each learner clears the random policy by about three orders of magnitude on cumulative return and by a factor of three to five on profit factor, while trading less. That gap, rather than any difference among TD3, DDPG, and PPO, is what the algorithm comparison establishes.

The comparison does not separate the three learning algorithms. On AAPL, the common instrument for which all four agents have been evaluated, TD3 and DDPG are indistinguishable on cumulative return (3.548 versus 3.599), and PPO posts the highest per-bar Sharpe ratio (7.53) on the lowest return (2.522). Reading these three as a ranking would over-interpret a single instrument and a single evaluation window. What separates cleanly is the learned-versus-

unlearned boundary: all three learners return above 250% on AAPL while the random policy returns -0.9%. The algorithms differ mainly in how they reach that outcome. PPO changes position roughly three times as often as TD3 ($731{,}830$ versus $233{,}082$ steps with a position change) for a smaller gross result.

That convergence is more informative than a ranking would have been. TD3 and DDPG differ precisely in the twin-critic overestimation correction, and PPO is on-policy and stochastic rather than off-policy and deterministic. Three optimizers with materially different inductive biases arriving at the same order of magnitude indicates that the exploitable structure is a property of the feature set and the evaluation environment, not an artifact of one algorithm's bias. The remainder of this thesis therefore fixes TD3 as a representative continuous-control learner rather than as a demonstrated winner, and varies the experimental factors around it.

Read through the reworded Hypothesis 1, the algorithm comparison establishes the existence half of the claim. Under frictionless mid-price execution every learned agent extracts a directionally coherent, risk-controlled edge that the random policy does not, so a learnable microstructure signal is present in this window. What the comparison does not establish is that the edge is economically large. Expressed per decision rather than per window, the TD3 agent's edge is a mean log return of roughly 1.6 to 3.6×10^{-6} per step across the six instruments (approximately 0.016 to 0.036 basis points, itemised in Table 5). This edge holds over sessions in which buy-and-hold on the same instruments ranged from -0.68% (MSFT) to $+1.07\%$ (AVGO), that is, sessions with no material directional move to capture. The headline cumulative figures are the compounding of a per-step edge far smaller than one tick of spread, not evidence of a large exploitable inefficiency. Section 4.3 (Section 4.3.1) explains why this magnitude must be read against omitted execution costs. The algebraic half-spread bound on the queue-weighted-midpoint displacement does not bound the policy's return. The relevant evidence instead comes from the transaction-cost sensitivity analysis in Section 6.2 and the fixed-path bid/ask repricing in Section 7.2. The defensible Hypothesis 1 verdict is therefore that continuous-control agents learn a small edge under the frictionless simulator. Its economic magnitude is highly sensitive to execution assumptions and is not an estimate of deployable performance.

6.2. Hypothesis 2: Transaction-Cost Sensitivity

This section varies the proportional transaction cost while holding the algorithm, feature set, and reward fixed, on a baseline with the selected 10-feature set and Differential Sharpe Ratio reward. A separate policy is trained at each fee level. The comparison tests whether performance and learned behaviour are sensitive to the cost assumption. It does not isolate the mechanical effect of fees when a trained policy becomes degenerate.

Table 6. Hypothesis 2 transaction-cost sensitivity: test-split metrics for independently trained policies across fee levels.

Fees	Return	Ann. Vol	Max DD	Sharpe	Sortino	Win Rate	Lose Rate	PF	Turnover	% Long	% Short
0 bp	13.45%	0.1823	-0.04%	2.819	169.252	25.16%	15.92%	3.635	0.5281	44.82%	55.18%
0.01 bp	10.65%	0.1728	-0.04%	2.362	24.868	16.29%	39.97%	2.474	0.2522	59.08%	40.92%
0.1 bp	0.01%	0.2155	-0.34%	0.006	0.010	11.04%	83.89%	1.003	0.0000	0.00%	100.00%
1 bp	-0.03%	0.2155	-0.37%	-0.008	-0.011	10.31%	83.41%	0.996	0.0000	100.00%	0.00%

Source: Author's own calculations.

Legend: Return: cumulative simple return. Ann. Vol: annualized volatility. Max DD: maximum drawdown. Sharpe: mean excess return / standard deviation (not annualized). Sortino: mean excess return / downside deviation (not annualized). Win Rate / Lose Rate: fraction of steps with positive / negative return. PF: profit factor (gross wins / gross losses). Turnover: mean absolute position change per step. % Long / % Short: fraction of steps with positive / negative position.

The transaction-cost comparison determines how far the frictionless Hypothesis 1 result can be interpreted economically. The policies trade actively at 0 and 0.01 basis points. At 0.1 basis points, aggregate return remains slightly positive, but the policy is fully short and turnover rounds to zero. At 1 basis point, the policy is fully long and turnover also rounds to zero. Its small negative return cannot be read as an active strategy losing through recurring trading costs. It primarily reflects passive price drift. The comparison therefore shows that learned behaviour is highly sensitive to modest modeled costs. It does not show that an otherwise functioning policy becomes unprofitable at a precise fee. The evidence supports a positive result under frictionless execution, not a cost-independent or deployable edge.

For a price-taking agent, a constant half-spread has the same algebraic form as the proportional fee: both penalize each unit of position change. The retraining ladder nevertheless combines this cost with the optimization response at each fee level. It is therefore not equivalent to repricing one fixed policy. Explicit bid/ask execution is also needed to represent a spread that changes over time.

The break between 0.01 and 0.1 basis points is a change in learned behaviour, not a sign reversal in aggregate return. It is the point at which the policy stops trading altogether. The minimum price increment for these instruments is one cent, so the smallest possible move in the mid-price, when only one side of the book updates, is half a cent. At the mean price of each instrument over the evaluation window this gives a reference fee – the proportional cost equal to a one-sided minimum-tick move – ranging from roughly 0.08 basis points for the highest-priced instrument to roughly 0.24 basis points for the lowest-priced one (Table 14). The observed break sits below every one of these reference points. This reinforces that the learned margin is small relative to quote-scale movements, but it does not identify a theoretical maximum edge or the mechanism causing the policy to stop trading. The per-instrument breakdown also does not support a simple tick-size explanation. If the break-even fee were set independently per instrument, turnover should collapse first for the highest-priced instruments and last for the lowest-priced ones. Instead, turnover collapses to zero for all six instruments simultaneously, between the same two fee levels. This is consistent with how the policy is trained: one network is shared across all six instruments, and the input features are z-score normalized per symbol before the policy ever sees them, so the model has no access to an instrument's absolute price level and cannot condition its trading threshold on it. The tick-size arithmetic supplies a scale comparison. On

this evidence, it cannot explain why the six instruments break together rather than in sequence.

The evidence therefore partially supports Hypothesis 2. Performance and learned behaviour are materially sensitive to the transaction-cost assumption. However, the predicted sign threshold is not established. The 0.1-basis-point aggregate return is slightly positive, while the 0.1- and 1-basis-point policies are static and saturated. Because each arm was trained separately, the comparison cannot separate direct fee erosion from a degenerate-policy outcome. Fixed-path repricing in Section 7.2.1 provides complementary evidence about the economic cost of the Hypothesis 1 action path. It does not identify a break-even fee for successfully retrained policies.

6.3. Hypothesis 3: Feature Specification Sensitivity

This section compares three feature-specification levels under otherwise identical conditions (same algorithm, data, reward, and transaction cost) to assess whether richer microstructure-aware state representations improve out-of-sample performance.

Table 7. Hypothesis 3 feature sensitivity: test-split metrics for minimal (3), selected (10), and full (33) feature sets.

Feature Set	Return	Ann. Vol	Max DD	Sharpe	Sortino	Win Rate	Lose Rate	PF	Turnover	% Long	% Short
Minimal	-0.02%	0.2154	-0.37%	-0.006	-0.007	17.09%	17.15%	0.997	0.0000	100.00%	0.00%
	-13.47%	+0.0331	-0.33%	-2.825	-169.259	-8.07%	+1.23%	-2.638	-0.5281	+55.18%	-55.18%
Selected	13.45%	0.1823	-0.04%	2.819	169.252	25.16%	15.92%	3.635	0.5281	44.82%	55.18%
	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)
Full	0.02%	0.2154	-0.34%	0.006	0.010	29.67%	29.89%	1.004	0.0000	0.00%	100.00%
	-13.43%	+0.0331	-0.30%	-2.812	-169.242	+4.51%	+13.97%	-2.632	-0.5280	-44.82%	+44.82%

Source: Author's own calculations. *Feature sets: Minimal* (3 features): best-level spread in basis points, best-level book pressure, and the queue-weighted midpoint. *Selected* (10 features): book pressure, three-level order-book imbalance, order-count imbalance, the queue-weighted midpoint and its divergence, bid and ask depth slopes, instantaneous and 50-event rolling OFI, and 50-event signed trade flow. *Full* (33 features): the selected set plus deeper book-pressure levels, volume-weighted mid-price skew and VAMP, spread ratio, bid/ask convexity, depth ratio, multi-level and autocorrelation OFI, bid/ask queue depletion, trade-arrival rate, cancel-to-trade ratio, VPIN, large-trade and odd-lot ratios, inter-event time, mid-price acceleration, and cyclical hour encodings. All three also append the agent's runtime position. *Legend:* Return: cumulative simple return. Ann. Vol: annualized volatility. Max DD: maximum drawdown. Sharpe: mean excess return / standard deviation (not annualized). Sortino: mean excess return / downside deviation (not annualized). Win Rate / Lose Rate: fraction of steps with positive / negative return. PF: profit factor (gross wins / gross losses). Turnover: mean absolute position change per step. % Long / % Short: fraction of steps with positive / negative position.

Only one of the three arms produced a functioning policy, so the comparison does not isolate the effect of the feature set. The selected 10-feature representation trades: turnover of 0.53, $21{,}776$ position changes, a profit factor of 3.64, and a cumulative return of $+13.4\%$, with exposure split 44.8% long against 55.2% short. Neither the minimal nor the full representation does. The minimal three-feature arm records turnover of 5.3×10^{-6} with zero position changes and sits fully long for the entire evaluation; the full 33-feature arm records turnover of 4.7×10^{-5}, also with zero position changes, and sits fully short. Both are saturated at an action bound, and their near-zero returns (-0.02% and $+0.02\%$) are the drift of a held position rather than the outcome of any learned decision.

The failures are therefore not evidence that three features carry too little signal or that thirty-three carry too much noise. They are the same optimization failure documented for the log-return arm in Hypothesis 4 below, and the two arms saturate at *opposite* bounds, which is what unstable optimization landing arbitrarily looks like rather than a systematic response to feature count. A genuine feature-quality effect would be visible as a difference between trained policies, not as the difference between one trained policy and two that never left their initial bound.

Hypothesis 3 is therefore unresolved on this evidence. What the run supports is narrower: under an identical configuration and a matched 500,000-step budget, the selected 10-feature specification converged to a trading policy while the 3-feature and 33-feature specifications did not. Whether a broader feature set improves performance once all arms train, and whether higher-dimensional observations simply require a longer budget to converge, are open questions that a training-stability study would need to separate (Section 7.2).

6.4. Hypothesis 4: Reward Function Design

This section compares the log-return baseline against the Differential Sharpe Ratio reward on an otherwise identical configuration (selected 10-feature set, zero transaction cost), holding the algorithm and data fixed.

Table 8. Hypothesis 4 reward-design sensitivity: test-split metrics for log-return vs. Differential Sharpe Ratio.

Reward Function	Return	Ann. Vol	Max DD	Sharpe	Sortino	Win Rate	Lose Rate	PF	Turnover	% Long	% Short
Log Return	-0.02%	0.2154	-0.37%	-0.006	-0.007	19.76%	19.87%	0.997	0.0000	100.00%	0.00%
DSR	13.45%	0.1823	-0.04%	2.819	169.252	25.16%	15.92%	3.635	0.5281	44.82%	55.18%

Source: Author's own calculations.

Reward functions: Log Return: the per-step change in log portfolio value (the baseline objective), multiplied by a fixed scale factor. *DSR*: the Differential Sharpe Ratio, the online single-step contribution to the Sharpe ratio computed from exponentially decaying estimates of the first two return moments ($\eta = 0.01$); self-normalizing, so no scale factor is applied. The construction is given in Section 4.3. *Legend:* Return: cumulative simple return. Ann. Vol: annualized volatility. Max DD: maximum drawdown. Sharpe: mean excess return / standard deviation (not annualized). Sortino: mean excess return / downside deviation (not annualized). Win Rate / Lose Rate: fraction of steps with positive / negative return. PF: profit factor (gross wins / gross losses). Turnover: mean absolute position change per step. % Long / % Short: fraction of steps with positive / negative position.

The comparison cannot be read as a like-for-like performance result, because the log-return arm did not produce a functioning policy. Across all six instruments its turnover is between 7.8×10^{-6} and 2.1×10^{-5}, against 0.48 to 0.58 for the DSR arm, and it holds a fully long position (pct_long $= 1.0$) for the entire evaluation on every instrument. The policy is saturated at the upper bound of the action interval and does not respond to the state; its returns, between -0.22% and $+0.26\%$, are the passive drift of a held long position rather than the result of any learned decision.[17]

Two observations support reading this as an optimization failure rather than a learned preference for passive exposure. First, a saturated policy would be expected to sit at a neutral position if it had learned that trading is unprofitable, not at a bound. Second, the same degenerate behaviour appears in an independent scenario that also uses the log-return reward, the minimal-feature arm of Hypothesis 3, which saturates at the *opposite* bound (fully short) rather than converging to any intermediate exposure. A single frozen run could be attributed to seed or budget; the same collapse in two separate scenarios sharing one reward specification points at the reward configuration itself.

Hypothesis 4 is therefore not settled by this run. What can be said is narrower than a performance comparison: under an identical configuration and a matched 500,000-step budget, the Differential Sharpe reward produced a policy that trades and takes both sides of the market, while the log-return reward produced one that did not train, and the alternative scalings tried during development did not change that outcome. Establishing whether the log-return objective is genuinely unsuitable in this setting, or remains recoverable under a configuration not yet explored, requires a systematic sweep over reward scaling and learning rate that is left to future work (Section 7.2).

Overall, the robustness evidence is strongest for transaction-cost sensitivity and feature specification. Reward design is inconclusive on the present evidence, for the reason given above: the log-return arm did not train, so the comparison does not isolate the effect of the reward objective. The results are not yet strong evidence of robustness across market regimes, longer samples, alternative chronological splits, or repeated random seeds.

[17] The two arms differ only in `reward_type` and its scaling: the log-return arm applies `reward_scale: 10000.0` to per-step log returns of order 10^{-6}, while the Differential Sharpe arm uses `reward_scale: 1.0` with $\eta = 0.01$, the reward being self-normalizing by construction. Alternative reward scalings were tried for the log-return objective during development and did not resolve the saturation. A systematic scale sweep was not part of the reported experimental package, so the scaling remains a plausible but not fully isolated contributor.

Seed-level robustness was spot-checked outside the formal hypotheses: a small number of independent short-budget TD3 trials with different random seeds, using the baseline configuration, were run to confirm that the reported performance reflects systematic learning rather than a single favourable initialisation. A full multi-seed study at the training budget used in the main experiments was not carried out, so single-run variance remains a limitation (Section 6.1; Section 7.2.1).

6.5. Performance Evaluation

Returns, turnover, and path-dependent risk measures from the latest held-out run jointly test whether the TD3 agent behaves economically coherently.

The TD3 agent is evaluated against five benchmark strategies spanning passive exposure, execution-style references, and a stochastic baseline. The table below summarizes key risk-adjusted metrics for each strategy, with the agent's performance highlighted in the first row.

Table 9. Benchmark strategy performance comparison on the test split.
Source: Author's own calculations.

TR: Total Return. Volatility: annualized standard deviation of returns. SR: Sharpe Ratio (mean excess return divided by standard deviation, not annualized). Sortino: Sortino Ratio (mean excess return divided by downside deviation, not annualized). Max DD: Maximum Drawdown. Win Rate: fraction of steps with positive return. Turnover: average absolute position change per step.

Return aggregation for ratio metrics. Roughly 80–90% of consecutive LOB returns are zero. The mid-price does not move between most order-book events. This causes $\hat{\sigma}$ to collapse faster than $\bar{r}$, inflating the naive per-tick Sharpe by $\sqrt{N}$ without any real edge. The pipeline therefore compounds per-tick returns into the coarsest bar size with at least 50 observations before computing Sharpe and Sortino, which are then reported as μ/σ and μ/σ_{down} on the aggregated series with no annualization. Total return, max drawdown, win rate, profit factor, and turnover are computed on the raw per-step series.

6.5.1. Evaluation Plots

Equity curve plot data not available — re-run evaluation to generate parquet artifacts:

```
uv run python src/cli.py evaluate \
  -c pooled/td3_hft_lob_state_space_pooled_streaming_selected_dsr \
  --output-dir logs/<run> \
  --only metrics --only benchmarks --only plots
```

Source: Author's own calculations.

Reward plot data not available — re-run evaluation to generate parquet artifacts:

```
uv run python src/cli.py evaluate \
  -c pooled/td3_hft_lob_state_space_pooled_streaming_selected_dsr \
  --output-dir logs/<run> \
  --only metrics --only benchmarks --only plots
```

Source: Author's own calculations.

Action/position plot data not available — re-run evaluation to generate parquet artifacts:

```
uv run python src/cli.py evaluate \
  -c pooled/td3_hft_lob_state_space_pooled_streaming_selected_dsr \
  --output-dir logs/<run> \
  --only metrics --only benchmarks --only plots
```

Source: Author's own calculations.

6.5.2. Discussion: Reading Performance in the Context of State Design

The benchmark comparison places the TD3 agent's held-out behavior in direct relation to the reference strategies. Several features of the comparison are worth stating explicitly.

The test window shows very little net directional movement. Averaged across the six instruments, buy-and-hold returned -0.18%, TWAP and VWAP roughly $+0.06\%$ and 0.00%, and a random-action policy $+0.07\%$. Against these, the TD3 agent averaged $+305\%$, three to four orders of magnitude above every passive benchmark in absolute terms. Because the underlying instruments barely moved over the window, that difference cannot be attributed to directional exposure: a policy holding any fixed position, long or short, would have earned approximately what buy-and-hold earned.

The agent's behavioral profile is consistent with this. Exposure is close to balanced at 49.8% long against 50.2% short, and turnover is 0.45 per step, so the return is accumulated across many small two-sided position changes rather than held through a single directional bet. This is the opposite of the structural explanation available to a near-static policy: the outperformance is generated by trading, not by declining to trade.

The trade-level statistics support the same reading. The agent's profit factor is 3.41 against buy-and-hold's, TWAP's, and VWAP's, all within a fraction of a per cent of unity, and its win rate is 20.4% against approximately 10% for each passive benchmark. The passive profit factors sit right at unity, which is what a flat market produces mechanically. The agent's does not.

On drawdown, the agent's maximum drawdown of -0.65% is shallower than buy-and-hold's -1.11%, though not by the wide margin seen on the individual instruments: the aggregate is dominated by a single -3.3% excursion on META, the instrument with the fewest events, whereas on the other five the agent's worst loss stays inside -0.25%. Balanced two-sided exposure limits the loss any single directional move can inflict.

These figures are measured under frictionless mid-price execution and inherit the Hypothesis 1 caveat in full: the comparison establishes that the policy extracts a real signal the passive benchmarks do not, not that the resulting edge survives the cost of trading on it. Section 6.2 shows it does not.

Taken together, the benchmark comparison establishes that the agent's outperformance is not a positioning artefact. Win rate (20.4% against approximately 10%) and profit factor (3.41 against 1.00) both exceed every passive benchmark by a wide margin, so the return gap reflects the quality of individual position changes rather than a fortunate net exposure. The Hypothesis 1 verdict from the benchmark comparison is therefore two-sided. The agent clears the existence bar: it extracts a signal the passive benchmarks do not, with better trade-level economics and a shallower maximum drawdown. The comparison is, however, run under mid-price execution with zero fees. That is not a neutral choice: zero-cost execution is a subsidy proportional to turnover, so a comparison whose only asymmetry is the spread the active arm does not pay favours the higher-turnover reinforcement learning policy over the near-static passive benchmarks. Any spread-crossing cost both erases the agent's own frictionless return (Section 6.2) and narrows its measured advantage over buy-and-hold. The benchmark comparison thus supports the existence half of Hypothesis 1 (a coherent, risk-controlled policy) but not a claim of benchmark-dominating

or deployable performance. The economic magnitude is highly sensitive to transaction cost, as Sections 4.3 (Section 4.3.1) and 6.2 show.

7. CONCLUSIONS AND FUTURE WORK

This thesis examined whether a continuous-control deep RL agent using order-book-derived microstructure features can achieve economically meaningful held-out performance in a high-frequency single-asset setting. The answer, within the specified experimental design, is qualified but affirmative: the following sections detail the evidence (Section 7.1), the limitations that bound this claim (Section 7.2), and their implications for trading-system design and practice.

7.1. Summary of Findings

The thesis set out to answer four hypotheses concerning deep reinforcement learning for high-frequency single-asset trading using order-book-derived features. This section summarizes the empirical evidence for each.

Hypothesis 1: a TD3 agent learns an assumption-sensitive risk-adjusted edge from order-book microstructure. Under frictionless mid-price execution, the agent learns a directionally coherent, low-turnover, drawdown-controlled policy that beats passive benchmarks on this held-out window (Section 6.5). TD3, DDPG, and PPO also separate cleanly from a random policy while remaining indistinguishable from one another (Section 6.1). These results support a learnable relationship under the specified simulator. They do not establish deployable performance. In the transaction-cost sweep, the learned policies collapse at the higher fee levels (Section 6.2). Fixed-path bid/ask repricing reverses the result (Section 7.2). The strong frictionless figures are therefore conditional on favorable execution assumptions. Retraining and re-evaluating under explicit bid/ask execution with calibrated fees remains the outstanding test.

Hypothesis 2: empirical performance is materially sensitive to the transaction-cost assumption. The evidence partially supports this hypothesis. The policies trained at 0 and 0.01 basis points trade actively and produce positive aggregate returns. At 0.1 basis points, aggregate return is still slightly positive, but the policy is static and fully short. At 1 basis point, the policy is static and fully long, and its small negative return is passive drift. The sweep therefore shows cost-sensitive training outcomes. It does not establish the predicted sign threshold for a functioning policy. Fixed-path bid/ask repricing separately shows that the H1 action path is not robust to spread-crossing costs.

Hypothesis 3: a broader microstructure-aware feature set improves out-of-sample

performance relative to a simpler snapshot-based state representation. This hypothesis is unresolved. Of the three feature specifications, only the selected 10-feature representation converged to a trading policy (turnover 0.53, profit factor 3.64, cumulative return $+13.4\%$). The 3-feature and 33-feature arms both saturated at an action bound with zero position changes, at opposite bounds, so their near-zero returns reflect a held position rather than a learned decision. Because two of the three arms did not train, the comparison does not isolate the effect of the feature set, and no conclusion about feature quality can be drawn from it. Section 6.3 sets out the evidence in full.

Hypothesis 4: the reward objective materially shapes the learned policy. This hypothesis is also unresolved, for the same reason. The Differential Sharpe arm trained normally and traded both sides of the market; the log-return arm did not train, holding a fully long position across every instrument with zero position changes and turnover of order 10^{-5}. Its returns are passive drift. The two arms therefore differ in whether optimization succeeded, not demonstrably in what the reward objective rewards, and alternative reward scalings tried during development did not recover the log-return arm. Establishing whether the log-return objective is unsuitable in this setting or merely mis-configured requires the training-stability work identified in Section 7.2.

That two of the four hypotheses reduce to the same failure is itself the more substantial finding. Four separately configured arms across Hypotheses 3 and 4 collapsed to a constant, saturated position under an identical architecture, budget, and dataset, two pinned long and two pinned short. This locates the binding constraint on the present results in training stability rather than in feature or reward design, and it is the first thing a continuation of this work should address.

7.2. Limitations and Future Research

7.2.1. Execution Realism

The central limitation of the empirical setup is its execution model: mid-price-proxy valuation and reward computation, zero market impact, zero latency, and no baseline fee. These assumptions materially favor a microstructure strategy. The implemented queue-weighted midpoint lies between the best bid and ask, so its divergence from the ordinary midpoint is bounded by half the spread. This identity was verified across the 2.4-million-row AAPL sample. It describes the scale of one contemporaneous input feature. It does not bound future price changes, the

policy's combined signal, or strategy profit. The execution limitation must therefore be assessed through the empirical sensitivity tests, not through a theoretical signal-ceiling claim.

For the evaluated Hypothesis 1 action path, the estimated spread-crossing cost is roughly ten times the gross profit, and the breakeven proportional fee is on the order of 4×10^{-6}. The Hypothesis 2 sweep is not a direct estimate of this break-even point. Each fee arm uses a separately trained policy, and the two higher-fee arms collapse to static positions. Re-pricing the identical action path under bid/ask execution instead of the mid moves the total return from roughly $+2.3$ to roughly -10.6 in net-liquidation-value units: the same signal, the opposite sign. These figures are indicative rather than final: they come from re-pricing a fixed policy, not from retraining. A control in which random actions run through the same frictionless environment returns -0.033%, which shows that random action alone does not reproduce the learned result. It does not identify the source of the edge or establish deployability. The direct next step, and the most important single extension, is to **retrain** and re-evaluate the agents under an explicit bid/ask fill rule with calibrated maker/taker fees and a slippage or market-impact model. Because the training reward shares the mid-price assumption, re-scoring the existing checkpoints is not sufficient.

Two further assumptions in Section 5.2 bear on the same question and have not been quantified here. The first is zero latency: the agent acts on an observation with no transport, decision, or matching-engine delay. The median inter-event gap on the test split is roughly 33 microseconds, so a few-hundred-microsecond round trip spans multiple order-book updates before the corresponding action could reach the market. The queue-weighted-midpoint divergence is a contemporaneous imbalance measure, not a validated one-step forecast. Its useful horizon has not been estimated here. Zero latency nevertheless gives the policy access to the freshest possible state and may overstate how much short-lived information remains tradable. A latency sweep is required to quantify that effect.

The second is the absence of any effect of the agent's own trading on the book. Section 5.2 defends this on position-size grounds: at this scale, trades are small relative to displayed depth, so walking the book is not a concern. That defense addresses price impact from order size, but not a separate channel: the features the policy conditions on (order-flow imbalance, queue depletion, queue-weighted-midpoint divergence) are themselves signatures of recent trading in the book, and the agent trades on them at close to every step (turnover of roughly 0.4-0.5 across instruments in the Hypothesis 1 agent). A real venue would reflect each fill back into queue

depth and subsequent order flow, feeding into the very quantities the next observation is built from; the simulation cannot represent that loop by construction, since it treats every trade as leaving the book unchanged. This is a different question from the transaction-cost sensitivity tested in Hypothesis 2: that result shows the edge is fragile to a compensating fee; this limitation is about whether the edge would persist at all once the strategy's own order flow is allowed to feed back into the signal it exploits. Answering it needs either an explicit impact model (for example, a price-impact term keyed to trade size in the spirit of Kyle's lambda, already used as a feature-construction concept in Chapter 2) or a multi-agent analysis of what happens once the same signal is traded by more than one participant; both are left for future work.

7.2.2. Evaluation Scope

The most important constraint on interpreting the results is the width of the evaluation window. The agent trains on three consecutive sessions (February 25-27, 2026) and is evaluated on a single held-out session (March 2, 2026), divided chronologically into a validation half and a test half. The test split is therefore genuinely out-of-sample with respect to training, drawn from a day the agent never saw. It is not independent of the validation split, which precedes it within the same session, and a single session supplies only one volatility regime, spread environment, and intraday seasonality. The agent has never encountered a different week, market condition, or instrument. This is a necessary but not sufficient condition for practical utility: the results establish that the learning problem is tractable in this window, not that the policy generalizes beyond it. The comparisons are also based on a single completed run per configuration on a single machine with a modest compute budget, so multi-seed variance and the ceiling reachable under larger-scale infrastructure both remain open. Extending the window across volatility and liquidity regimes, walk-forward validation, multi-seed reporting, and cross-asset testing (including foreign exchange, futures, and cryptocurrency) would be the most direct ways to strengthen the generalization claim and determine whether asset-class-specific tuning is necessary.

7.2.3. Partial Observability

A more fundamental constraint, shared with most academic RL trading research, is that the agent sees only the spot limit order book of a single instrument at a single venue. Price-relevant information that never reaches that book (derivatives and options-implied signals, unstructured

data such as earnings and macro releases, and proprietary order flow visible only to systematic internalizers, market makers, and prime brokers) appears to the agent as unexplained noise. This noise both limits value-estimate precision and raises reward variance. This creates a structural information asymmetry between academic research, confined to public data, and well-resourced industry participants. The results here are best read as a lower bound on what such participants could achieve with the same technique, not as a statement about the technique's ceiling. The reward signal inherits this noise directly, since tick-level returns are dominated by bid-ask bounce and transient order-flow effects the agent cannot attribute to a cause. This constraint on learning exists independently of algorithm or reward choice, and action-frequency sub-sampling (acting every N ticks rather than every tick) could partially diagnose it at low cost. If several of the engineered features carry predictive content over horizons longer than the next event, a lower-frequency action interval might reveal signal the current tick-level setup discards as noise; whether they do is an open question this thesis does not test.

7.2.4. Training Workflow

Action saturation is the most consequential unresolved issue in the experimental package, and it is not confined to a few discarded runs. Four separately configured arms reported in Chapter 6 converged to a constant, saturated position: the 3-feature and 33-feature arms of Hypothesis 3, and the log-return arm of Hypothesis 4, together with an earlier log-return baseline that preceded it. Each records turnover of order 10^{-5} with zero position changes and sits at 100% long or 100% short for the entire evaluation. Two saturate at the upper bound and two at the lower, which is characteristic of an optimization that lands arbitrarily rather than of a systematic response to the factor under test. Because the affected arms span both the feature-set and reward-design comparisons, two of the four hypotheses are unresolved on present evidence, and the constraint is a property of training rather than of the design choices those hypotheses examine.

Two properties of the setup make this diagnosable rather than mysterious. The arms that failed share the log-return objective or a small observation space, while the Differential Sharpe arms at the same budget trained normally, and alternative reward scalings tried during development did not recover the log-return arm. A systematic sweep over reward scaling, actor learning rate, and exploration noise, with the saturation rate logged as a first-class training diagnostic, would establish whether these configurations are unsuitable in this setting or merely mis-parameterized.

Validation-based checkpoint selection and early stopping on validation Sharpe, action saturation rate, and critic-value magnitude would additionally prevent a saturated policy from reaching evaluation at all, replacing the manual exclusion used here with an automated and reproducible criterion.

An earlier revision of the Hypothesis 1 comparison scored TD3 over a ten-percent window, after a configuration collision required it to be re-evaluated, while DDPG, PPO, and the random baseline used the full test split; because cumulative return compounds with window length, that table carried a per-step-edge rescaling of the TD3 figure. TD3 has since been re-evaluated from its three-million-step checkpoint over the identical full per-symbol test window, so the comparison in Section 6.1 is now a matched direct measurement with no projection. It places TD3 at $+305\%$ aggregate return against DDPG's $+313\%$ and PPO's $+206\%$, confirming that the three learners are indistinguishable and that the separation that matters is the one from the random policy.

TD3's stability corrections are also not complete. Target policy smoothing regularizes the critic to be locally consistent around actions seen in the replay buffer, but it does not eliminate extrapolation error. Once the actor begins producing actions in regions the critic has seen rarely or never, the critic's estimates there are extrapolations from the training distribution rather than reliable values. The actor receives no signal that it has left the buffer's support. This is distinct from the overestimation-bias correction discussed in Chapter 3 (Section 3.3). The twin-critic minimum addresses optimistic bias at the target computation, not exposure to out-of-distribution actions at the actor update. Stochastic-policy methods such as SAC are structurally less exposed to this failure mode, since the entropy bonus discourages the policy from collapsing onto a narrow region of action space.

7.2.5. Future Research Directions

Beyond re-evaluating under realistic execution and broadening the evaluation window (above), several extensions follow directly from the design choices made here:

- **Richer agent state.** The observation includes only current exposure; unrealized P&L, holding duration, rolling turnover, and episode drawdown would add inventory-aware self-state without changing the market observation.
- **Temporal context.** The observation is a single snapshot, which aliases different price

trajectories into the same state. A shallow lag stack (5-10 steps) of key features is a low-cost first step before the more substantial redesign implied by a sequence encoder.

- **Cross-asset and multi-asset extensions.** The pooled model trains across six instruments but manages one position at a time. Synchronized cross-asset signals (e.g. VIX futures, sector-ETF flow), genuine multi-asset portfolio optimization, and testing on instruments with intermediate liquidity are natural extensions. Intermediate liquidity means active enough to generate a rich order book but not so deep that every pattern is immediately arbitraged, unlike the highly liquid large-caps used here. A more tractable intermediate step is a pair with an economically well-defined price anchor, such as dual-class shares representing identical economic exposure to the same underlying business. For such a pair, deviations from parity are theoretically bounded and the learning target becomes the spread rather than a directional price level.
- **Algorithm and reward comparison.** Replacing TD3 with SAC or PPO, and the Differential Sharpe Ratio with a Sortino-based or drawdown-penalized reward, would test how much of the observed performance is attributable to the specific algorithm and reward choice rather than the feature design. Within the MLP family, a structured search over learning rate, hidden-layer width and depth, and activation function would separate algorithmic contribution from parameterization. Activation functions with different saturation properties, such as GELU or ELU in place of ReLU, may interact differently with the bounded, skewed distribution of order-book features. Layer normalization inside the critic (untested here) could stabilize Q-value scale across reward magnitudes.
- **Alternative architectures.** Transformer-based sequence encoders and hybrid gradient-boosted-tree approaches such as GBRL (Fuhrer et al., 2025) are promising directions given the tabular, engineered nature of the feature set. However, a linear-policy baseline should be evaluated first: if a linear actor matches the MLP's out-of-sample performance, that is stronger evidence about the problem's structure than any higher-capacity model result would be.

7.3. Implications for Trading-System Design

Four findings from the experiments generalise beyond the specific configurations tested, in the sense that they concern the design of the learning problem rather than the particular agent

trained here.

State specification dominated algorithm choice. Feature specification determined whether training succeeded at all. Of three feature sets run under an identical budget and architecture, only the curated ten-feature set converged to a trading policy; both a three-feature subset and a thirty-three-feature superset saturated at a fixed position and never trained (Section 6.3). Because two of the three arms did not optimise, this does not establish that a richer state improves returns. It does show that a poorly chosen state can block learning outright, which no amount of algorithm tuning recovers. The microstructure literature reviewed in Chapter 2 motivates which quantities belong in the state: order-flow imbalance, spread regime, book shape, and temporal encoding.

The choice among continuous-control algorithms mattered less than the selection rationale implied. TD3 was chosen for its three stability mechanisms, namely clipped double Q-learning, delayed policy updates, and target policy smoothing (Section 3.3). On this data those mechanisms produced no measurable advantage. TD3 and DDPG, which has none of them, are indistinguishable on cumulative return, and PPO separates from neither (Section 6.1). All three separate cleanly from a random policy. The comparison therefore locates the learnable signal in the state representation and the reward rather than in the choice among continuous-control algorithms, at least at this training budget and on this window. The continuous action space itself remains motivated on design grounds, since the economically correct position magnitude varies with signal strength (Section 4.1), but no discrete-action baseline was trained here to test that argument empirically.

Chronological evaluation discipline is a precondition, not a refinement. Any lookahead, whether features computed from future data, normalisation fit on the full dataset, or non-sequential splits, produces a materially optimistic out-of-sample assessment. In high-frequency settings, where signal decay is measured in seconds, even mild lookahead can manufacture the appearance of a viable strategy from a leakage artefact. Two constructions avoid it: fitting the normalisation on the training split alone and applying it frozen, or computing it as a strictly causal online transformation. This thesis uses the second, with statistics maintained per symbol and reset at session boundaries, and verifies the property directly rather than asserting it (Section 5.1).

Reward optimisation and financial evaluation measure different things. A shaped reward such as the Differential Sharpe Ratio is a training objective, not a performance report

(Section 4.3). Treating a high training DSR as evidence of held-out Sharpe performance conflates the two, and the separation is maintained throughout the evaluation reported in Chapter 6.

Transaction-cost sensitivity bounds what the result can support. The execution model used here is a mid-price proxy and is optimistic relative to executable bid and ask prices. Spread-crossing costs are of similar magnitude to the price moves the agent targets. The higher-fee retraining arms collapse to static positions (Section 6.2). The sweep establishes sensitivity, but not a precise break-even fee for a functioning policy. Fixed-path bid/ask repricing does reverse the baseline result. The frictionless finding should therefore be read as evidence within the simulator, not as evidence of tradability.

7.4. Conclusion

The central question of this thesis was: can a LOB-informed deep RL agent achieve meaningful out-of-sample trading performance in a high-frequency setting? The answer, under the specified experimental conditions, is affirmative but bounded. Under frictionless mid-price execution, the TD3 agent trained on microstructure-aware limit order book features learned a directionally coherent, risk-controlled policy that outperformed the evaluated benchmark strategies on this held-out window. That outperformance is clear within the simulator but highly sensitive to its execution assumptions. The fee sweep shows that higher-fee policies collapse to static positions. It does not identify a precise break-even fee. Fixed-path bid/ask repricing reverses the result (Section 7.2). The qualitative character of the learned policy is consistent with a deliberate strategy rather than noise: turnover well below that of a random policy, responsive rather than static positioning, and drawdown an order of magnitude shallower than passive exposure.

The contribution of the feature set itself is left open. Of the three feature specifications compared under identical training and evaluation conditions, only the curated ten-feature configuration converged to a trading policy; the three-feature and thirty-three-feature variants both saturated at a fixed position and never trained (Sections 6.2 and 7.1). Because two of the three arms did not optimize, the comparison cannot isolate the effect of feature choice, and Hypothesis 3 is unresolved. The pattern is nonetheless suggestive: a compact, theoretically motivated set may give the policy a higher signal-to-noise ratio to work with than either a state too sparse to be informative or one padded with collinear and noisy channels. Whether that intuition survives

once every arm is trained to convergence, or whether the failures instead reflect optimization budget and observation dimensionality, is a question for the training-stability work identified in Section 7.2.

The contribution of the thesis is methodological as much as empirical. The end-to-end pipeline developed here provides a replicable framework for this class of research problem. It covers feature engineering from 10-level order book data, structured experiment management, TD3 training with Differential Sharpe Ratio reward, and a rigorous separation between training diagnostics and held-out financial evaluation. The emphasis throughout on design transparency, chronological data integrity, and explicit simulation assumptions reflects a view that the quality of the experimental setup is inseparable from the soundness of its conclusions.

At the same time, the thesis is explicit about what it does not claim. Within the constraints of the controlled setup, the observed edge cannot be straightforwardly mapped to a deployable trading strategy. Whether the microstructure signal captured by the learned policy survives realistic spread-crossing costs, maker/taker fees, queue uncertainty, and latency remains an open and important empirical question. The limitations and future research section identifies it as the immediate extension of the current work.

What the thesis does establish is that the combination of limit order book data, LOB-informed feature engineering, and continuous-action actor-critic learning is a productive research direction. The informational content of the order book, well-established in the theoretical microstructure literature, can be translated into state features that a continuous-control agent learns to act on, producing held-out behavior that is measurable and consistent with economic intuition. That conclusion is the foundation on which more operationally realistic extensions (wider data windows, multi-seed validation, explicit bid/ask execution, and cross-asset generalization) can now be built.

BIBLIOGRAPHY

Abergel, F., Anane, M., Chakraborti, A., Jedidi, A., & Muni Toke, I. (2016). *Limit Order Books*. Cambridge University Press. https://doi.org/10.1017/CBO9781316683040

Bellman, R. (1957). *Dynamic programming*. Princeton University Press.

Biais, B., Hillion, P., & Spatt, C. (1995). An empirical analysis of the limit order book and the order flow in the Paris Bourse. *Journal of Finance*, *50*(5), 1655–1689. https://doi.org/10.1111/j.1540-6261.1995.tb05192.x

Boehmer, E., Jones, C. M., Zhang, X., & Zhang, X. (2021). Tracking retail investor activity. *Journal of Finance*, *76*(5), 2249–2305. https://doi.org/10.1111/jofi.13033

Bouchaud, J.-P., Mézard, M., & Potters, M. (2002). Statistical properties of stock order books: Empirical results and models. *Quantitative Finance*, *2*(4), 251–256. https://doi.org/10.1088/1469-7688/2/4/301

Cao, C., Hansch, O., & Wang, X. (2009). The information content of an open limit-order book. *Journal of Futures Markets*, *29*(1), 16–41. https://doi.org/10.1002/fut.20334

Cont, R., Kukanov, A., & Stoikov, S. (2014). The price impact of order book events. *Journal of Financial Econometrics*, *12*(1), 47–88. https://doi.org/10.1093/jjfinec/nbt002

D'Avolio, G. (2002). The market for borrowing stock. *Journal of Financial Economics*, *66*(2–3), 271–306. https://doi.org/10.1016/S0304-405X(02)00206-4

Dempster, M. A. H., & Romahi, Y. (2002). Intraday FX trading: An evolutionary reinforcement learning approach. *Intelligent Data Engineering and Automated Learning (IDEAL 2002), Lecture Notes in Computer Science*, *2412*, 347–358. https://doi.org/10.1007/3-540-45675-9_52

Deng, Y., Bao, F., Kong, Y., Ren, Z., & Dai, Q. (2017). Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, *28*(3), 653–664. https://doi.org/10.1109/TNNLS.2016.2522401

Easley, D., López de Prado, M. M., & O'Hara, M. (2012). Flow toxicity and liquidity in a high-frequency world. *Review of Financial Studies*, *25*(5), 1457–1493. https://doi.org/10.1093/rfs/hhs053

Engle, R. F. (2000). The econometrics of ultra-high-frequency data. *Econometrica*, *68*(1), 1–22. https://doi.org/10.1111/1468-0262.00091

Foucault, T., Kadan, O., & Kandel, E. (2005). Limit Order Book as a Market for Liquidity. *The Review of Financial Studies*, *18*(4), 1171–1217. https://doi.org/10.1093/rfs/hhi029

Fuhrer, B., Tessler, C., & Dalal, G. (2025). Gradient boosting reinforcement learning. *Proceedings of the Forty-Second International Conference on Machine Learning*, *267*, 17960–17985. https://arxiv.org/abs/2407.08250

Fujimoto, S., Hoof, H. van, & Meger, D. (2018). Addressing function approximation error in actor-critic methods. *ICML*, 1587–1596.

Glosten, L. R. (1994). Is the electronic open limit order book inevitable? *Journal of Finance*, *49*(4), 1127–1161. https://doi.org/10.1111/j.1540-6261.1994.tb02450.x

Glosten, L. R., & Milgrom, P. R. (1985). Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, *14*(1), 71–100. https://doi.org/10.1016/0304-405X(85)90044-3

Gold, C. (2003). FX trading via recurrent reinforcement learning. *Proceedings of the 2003 IEEE International Conference on Computational Intelligence for Financial Engineering*, 363–370. https://doi.org/10.1109/CIFER.2003.1196283

Gort, B. J. D., Liu, X.-Y., Sun, X., Gao, J., Chen, S., & Wang, C. D. (2022). *Deep reinforcement learning for cryptocurrency trading: Practical approach to address backtest overfitting*. https://arxiv.org/abs/2209.05559

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *ICML*, 1861–1870.

Harris, L. (1986). A transaction data study of weekly and intradaily patterns in stock returns. *Journal of Financial Economics*, *16*(1), 99–117. https://doi.org/10.1016/0304-405X(86)90044-9

Harris, L. (1991). Stock price clustering and discreteness. *Review of Financial Studies*, *4*(3), 389–415. https://doi.org/10.1093/rfs/4.3.389

Hasselt, H. van. (2010). Double Q-learning. *Advances in Neural Information Processing Systems*, *23*, 2613–2621.

Hasselt, H. van, Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *30*. https://doi.org/10.1609/aaai.v30i1.10295

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep Reinforcement Learning That Matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, *32*, 3207–3214. https://doi.org/10.1609/aaai.v32i1.11694

Jiang, Z., & Liang, J. (2017). Cryptocurrency portfolio management with deep reinforcement learning. *arXiv Preprint arXiv:1612.01277*. https://arxiv.org/abs/1612.01277

Jiang, Z., Xu, D., & Liang, J. (2017). *A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem*. 1–31. http://arxiv.org/abs/1706.10059

Kabbani, T., & Duman, E. (2022). Deep reinforcement learning approach for trading automation in the stock market. *IEEE Access*, *10*, 93564–93574. https://doi.org/10.1109/ACCESS.2022.3203697

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv Preprint arXiv:1412.6980*. https://arxiv.org/abs/1412.6980

Krauss, C., Do, X. A., & Huck, N. (2017). Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the s&p 500. *European Journal of Operational Research*, *259*(2), 689–702. https://doi.org/10.1016/j.ejor.2016.10.031

Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica*, *53*(6), 1315–1335. https://doi.org/10.2307/1913210

Lee, C. M. C., & Ready, M. J. (1991). Inferring trade direction from intraday data. *Journal of Finance*, *46*(2), 733–746. https://doi.org/10.1111/j.1540-6261.1991.tb02683.x

Lee, J. W., Park, J., O, J., Lee, J., & Hong, E. (2007). A multiagent approach to Q-learning for daily stock trading. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, *37*(6), 864–877. https://doi.org/10.1109/TSMCA.2007.904825

Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J., Jordan, M., & Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. *Proceedings of the 35th International Conference on Machine Learning, Proceedings of Machine Learning Research*, *80*, 3053–3062.

Liang, Z., Chen, H., Zhu, J., Jiang, K., Li, Y., & Zhu, W. (2018). Adversarial deep reinforcement learning in portfolio management. *arXiv Preprint arXiv:1808.09940*. https://arxiv.org/abs/1808.09940

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. *International Conference on Learning Representations*.

Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, *8*(3–4), 293–321. https://doi.org/10.1007/BF00992699

Liu, X.-Y., Xiong, Z., Zhong, S., Yang, H., & Walid, A. (2018). Practical deep reinforcement learning approach for stock trading. *arXiv Preprint arXiv:1811.07522*. https://arxiv.org/abs/1811.07522

Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., & Wang, C. D. (2020). FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv*

Preprint arXiv:2011.09607. https://arxiv.org/abs/2011.09607

López de Prado, M. (2018). *Advances in financial machine learning*. Wiley.

Majidi, N., Shamsi, M., & Marvasti, F. (2024). Algorithmic trading using continuous action space deep reinforcement learning. *Expert Systems with Applications*, *237*, 121292. https://doi.org/10.1016/j.eswa.2023.121292

Millea, A. (2021). Deep reinforcement learning for trading—a critical survey. *Data*, *6*(11), 119. https://doi.org/10.3390/data6110119

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*(7540), 529–533. https://doi.org/10.1038/nature14236

Mohammadshafie, A., Mirzaeinia, A., Jumakhan, H., & Mirzaeinia, A. (2024). *Deep reinforcement learning strategies in finance: Insights into asset holding, trading behavior, and purchase diversity*. https://arxiv.org/abs/2407.09557

Moody, J., & Saffell, M. (1998). *Reinforcement learning for trading*.

Moody, J., & Saffell, M. (2001). Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, *12*(4), 875–889. https://doi.org/10.1109/72.935097

Neuneier, R. (1998). Enhancing q-learning for optimal asset allocation. *Advances in Neural Information Processing Systems*, *10*, 936–942.

Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. *ICML*, 278–287.

Noonan, L. (2017). *JPMorgan to Take AI Trading Global*. https://www.ft.com/content/16b8ffb6-7161-11e7-aca6-c6bd07df1a3c?syn-25a6b1a6=1

Puterman, M. L. (1994). *Markov decision processes: Discrete stochastic dynamic programming*. John Wiley & Sons.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, *22*(268), 1–8. http://jmlr.org/papers/v22/20-1364.html

Roll, R. (1984). A simple implicit measure of the effective bid-ask spread in an efficient market. *Journal of Finance*, *39*(4), 1127–1139.

Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015). Trust region policy

optimization. *ICML*, 1889–1897.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv Preprint arXiv:1707.06347*. https://arxiv.org/abs/1707.06347

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 387–395.

Stoikov, S. (2018). The micro-price: A high-frequency estimator of future prices. *Quantitative Finance*, *18*(12), 1959–1966. https://doi.org/10.1080/14697688.2018.1489139

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, *12*, 1057–1063.

Tsitsiklis, J. N., & Van Roy, B. (1997). An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, *42*(5), 674–690. https://doi.org/10.1109/9.580874

Tulchinsky, I. (Ed.). (2019). *Finding alphas: A quantitative approach to building trading strategies* (2nd ed.). Wiley.

Wang, J., Zhang, Y., Tang, K., Wu, J., & Xiong, Z. (2019). AlphaStock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1900–1908. https://doi.org/10.1145/3292500.3330647

Watkins, C. J. C. H. (1989). *Learning from delayed rewards*. King's College, University of Cambridge.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, *8*(3–4), 279–292. https://doi.org/10.1007/BF00992698

Welford, B. P. (1962). Note on a method for calculating corrected sums of squares and products. *Technometrics*, *4*(3), 419–420. https://doi.org/10.1080/00401706.1962.10490022

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, *8*(3–4), 229–256. https://doi.org/10.1007/BF00992696

Wojdalski, K. (2026). *Deep reinforcement learning for high-frequency trading: Master's thesis codebase*. https://github.com/kwojdalski/masters_thesis

Wood, R. A., McInish, T. H., & Ord, J. K. (1985). An investigation of transactions data for NYSE stocks. *Journal of Finance*, *40*(3), 723–739.

XAI Asset Management. (2024). *TradingEnv: An event-driven market simulator for backtesting and reinforcement learning*. https://github.com/xaiassetmanagement/tradingenv

Yang, H., Liu, X.-Y., Zhong, S., & Walid, A. (2020). *Deep reinforcement learning for automated stock trading: An ensemble strategy* (pp. 1–8). Proceedings of the First ACM International Conference on AI in Finance. https://doi.org/10.1145/3383455.3422540

Zhang, Z., Zohren, S., & Roberts, S. (2019). DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, *67*(11), 3001–3012. https://doi.org/10.1109/TSP.2019.2907260

Zhang, Z., Zohren, S., & Roberts, S. (2020). Deep reinforcement learning for trading. *The Journal of Financial Data Science*, *2*(2), 25–40. https://doi.org/10.3905/jfds.2020.1.030

GLOSSARY OF ABBREVIATIONS AND KEY TERMS

This glossary defines abbreviations and domain-specific terms used throughout the thesis. Entries are restricted to terms whose meaning in this context differs from everyday usage or whose precise definition is necessary to interpret the empirical results; widely understood concepts are omitted. Terms are grouped by domain — market microstructure and trading, reinforcement learning and machine learning, and other — and listed alphabetically within each group. For mathematical notation and feature formulas, see Appendix A. The glossary is intentionally selective. Not every technical term that appears in the text receives an entry; terms with widely accepted, unambiguous definitions — standard statistical measures, common financial concepts, and general machine-learning terminology — are omitted where their inclusion would expand the section beyond its practical purpose.

Market Microstructure and Trading

Term	Definition
Adverse selection	The risk that a counterparty in a trade possesses superior private information, causing uninformed participants (e.g., market makers) to trade at a disadvantage.
Ask slope	Price elasticity of the ask side of the order book across multiple depth levels; a proxy for market impact per unit of sell-side volume.
Backtest	Historical simulation of a trading strategy using past market data to assess performance before live deployment. All performance figures reported in this thesis are based on backtests conducted on held-out out-of-sample data.
Basis points (bps)	One hundredth of a percentage point (0.01%). Used to express spreads and returns at fine granularity.
Bid-ask spread	The difference between the lowest ask price and the highest bid price in the order book; the primary observable measure of transaction cost and adverse selection risk.

Term	Definition
Bid slope	Price elasticity of the bid side of the order book across multiple depth levels; a proxy for market impact per unit of buy-side volume.
Book convexity	A measure of curvature in the price schedule of the order book, capturing non-linear spacing between price levels on each side.
CumDelta (Cumulative Delta)	Signed trade flow aggregated over a rolling window; the net difference between buyer-initiated and seller-initiated executed volume.
Depth imbalance	Asymmetry in resting volume between the bid and ask sides of the order book at one or more price levels.
ETF	Exchange-Traded Fund. A publicly traded fund that tracks an index, sector, or asset class.
HFT	High-Frequency Trading. Algorithmic trading operating at tick or sub-second timescales, relying on order-book data and microstructure signals rather than price bars.
Inter-event time	The elapsed time between consecutive limit order book updates; a proxy for market activity intensity.
Kyle's lambda (λ)	A measure of price impact (Kyle, 1985): the sensitivity of price changes to net order flow. Higher λ implies lower market liquidity.
LOB	Limit Order Book. The electronic record of all outstanding buy (bid) and sell (ask) limit orders at each price level, maintained by an exchange.
Market maker	A liquidity provider that continuously posts limit orders on both sides of the book, profiting from the bid-ask spread while bearing inventory and adverse selection risk.
MFT	Medium-Frequency Trading. Trading strategies operating at intraday bar (minute to hour) timescales, as distinct from high-frequency tick-level strategies.

Term	Definition
Micro-price (Stoikov)	A model-based estimate of a future midpoint conditional on the current midpoint, queue imbalance, and spread (Stoikov, 2018). It is distinct from the static weighted mid-price.
Microprice / MicroDiv (legacy feature names)	Implementation names retained for result provenance. In this thesis they denote the static queue-weighted midpoint and its divergence from the ordinary midpoint, not Stoikov's full micro-price model.
Mid-price	The arithmetic average of the best bid and best ask prices; commonly used as a reference fair value when the spread is symmetric.
NLV (Net Liquidation Value)	The current market value of all open positions plus available cash in a trading account. Used in this thesis as the basis for computing true portfolio returns during evaluation, as opposed to per-step reward signals.
OBI (Order Book Imbalance)	Normalized difference in resting volume between bid and ask sides, aggregated across multiple depth levels.
OCI (Order Count Imbalance)	Normalized difference in the number of resting orders between bid and ask sides at a given price level; sensitive to order fragmentation masked by volume-based measures.
OFI (Order Flow Imbalance)	Net directional pressure exerted on the best bid and ask queues by limit order arrivals, cancellations, and executions (Cont et al., 2014). The empirically dominant contemporaneous predictor of short-term price changes.
OHLCV	Open, High, Low, Close, Volume. The standard aggregated price bar representation used in low- and medium-frequency trading; does not capture intra-bar queue dynamics.
Odd-lot	A trade smaller than the standard round lot size (typically below 100 shares). Predominantly reflects retail and algorithmic small-lot activity.

Term	Definition
Queue-weighted midpoint	A static weighted average of the best bid and ask using the displayed opposing queue sizes (Stoikov, 2018). It summarizes current top-of-book imbalance and remains inside the quoted spread.
Queue depletion	The rate at which the best-bid or best-ask queue is consumed by incoming market orders across consecutive snapshots; a leading indicator of imminent price movement.
RSI	Relative Strength Index. A momentum oscillator measuring the speed and magnitude of price changes; used in rule-based and technical trading strategies.
Spread ratio	The current bid-ask spread divided by its recent rolling mean; a regime signal separating absolute spread level from elevated adverse selection conditions.
TWAP	Time-Weighted Average Price. An execution benchmark computed as the mean transaction price over a fixed time window, weighting each trade equally in time. Reported alongside VWAP in the performance comparison tables.
VIX	CBOE Volatility Index. A measure of expected 30-day S&P 500 volatility derived from options prices; a common proxy for broad market fear or uncertainty.
VWAP	Volume-Weighted Average Price. The average price of an asset weighted by traded volume over a period; a common execution benchmark. Computed retrospectively over the evaluation window, so the reported VWAP rows are reference values rather than attainable returns (see Section 6.1.1).

Reinforcement Learning and Machine Learning

Term	Definition
Actor-critic	A class of reinforcement learning architectures combining a policy network (actor) that selects actions with a value network (critic) that evaluates them. Reduces gradient variance relative to pure policy gradient methods.
Bellman equation	The recursive identity expressing the value of a state as immediate reward plus the discounted value of the successor state: $V(s) = r + \gamma V(s')$. The theoretical foundation of temporal difference learning and Q-learning.
Bootstrapping	Updating a value estimate using another estimate as the target, rather than waiting for a complete episode return. Introduces bias but reduces variance; the mechanism underlying all temporal difference algorithms.
DDPG	Deep Deterministic Policy Gradient (Lillicrap et al., 2016). An off-policy actor-critic algorithm for continuous action spaces using deterministic policies and experience replay. Predecessor to TD3.
DQN	Deep Q-Network (Mnih et al., 2015). A value-based deep RL algorithm combining Q-learning with a deep neural network, experience replay, and a frozen target network. Restricted to discrete action spaces.
DSR	Differential Sharpe Ratio (Moody & Saffell, 1998). A reward formulation that approximates the gradient of the Sharpe Ratio with respect to returns, aligning the optimization objective with risk-adjusted performance.
Exploration-exploitation tradeoff	The tension in reinforcement learning between taking actions that yield known rewards (exploitation) and trying new actions to discover potentially better outcomes (exploration). Particularly costly in trading due to transaction costs.

Term	Definition
MDP	Markov Decision Process. The formal framework for sequential decision-making under uncertainty, consisting of states, actions, a transition function, and a reward function. Assumes that future states depend only on the current state and action, not on history.
Off-policy	A class of RL algorithms that learn a target policy from data collected by a different behavior policy. Enables experience replay and reuse of historical data. TD3 is off-policy.
On-policy	A class of RL algorithms that must learn from data collected by the same policy being improved. More stable but less sample-efficient than off-policy methods.
PCA	Principal Component Analysis. A dimensionality reduction technique that projects data onto orthogonal axes of maximum variance.
Policy gradient	A family of RL algorithms that directly optimize the policy parameters through gradient ascent on expected cumulative reward. Handles continuous action spaces naturally.
PPO	Proximal Policy Optimization. An on-policy actor-critic algorithm that constrains policy updates to a trust region to improve stability.
Q-learning	A model-free, off-policy value-based RL algorithm that learns the action-value function $Q(s, a)$ through temporal difference updates (Watkins & Dayan, 1992).
Replay buffer	A data structure storing past environment transitions (s_t, a_t, r_t, s_{t+1}) for reuse in off-policy training. Decorrelates training samples and improves sample efficiency.

Term	Definition
REINFORCE	A Monte Carlo policy gradient algorithm (Williams, 1992) that updates policy parameters using full-episode return estimates. High variance; foundational to modern policy gradient methods.
RL	Reinforcement Learning. A machine learning paradigm in which an agent learns to make sequential decisions by interacting with an environment and maximizing cumulative reward.
SAC	Soft Actor-Critic (Haarnoja et al., 2018). An off-policy actor-critic algorithm incorporating a maximum-entropy objective to encourage exploration through stochastic policies.
SARSA	State-Action-Reward-State-Action. An on-policy temporal difference algorithm that updates $Q(s, a)$ using the action actually taken by the current policy.
Target network	A periodically or slowly updated copy of the Q-network or actor network used to generate stable training targets, preventing the oscillations that arise when both prediction and target change simultaneously. Used in DQN, DDPG, and TD3.
TD3	Twin Delayed Deep Deterministic Policy Gradient (Fujimoto et al., 2018). An off-policy actor-critic algorithm for continuous control that extends DDPG with clipped double Q-learning, delayed policy updates, and target policy smoothing to reduce overestimation bias. The primary algorithm used in this thesis.
TD error	The difference between the Bellman target $y_t = r_t + \gamma Q(s_{t+1}, a_{t+1})$ and the current value estimate $Q(s_t, a_t)$. Minimising the mean squared TD error trains the critic network in all actor-critic algorithms.

Term	Definition
Temporal difference (TD) learning	A class of RL methods that update value estimates based on the difference between successive predictions, without requiring full episode trajectories.
Value function	A function estimating the expected cumulative discounted reward from a given state (or state-action pair) under a given policy. The critic network in actor-critic algorithms approximates this function.

Other

Term	Definition
AAPL	The NASDAQ ticker symbol for Apple Inc. One of six Nasdaq-listed equities — alongside MSFT, TSLA, META, AMZN, and AVGO — used as trading instruments in this thesis.
Maximum drawdown	The largest peak-to-trough decline in portfolio value over an evaluation period, expressed as a percentage. A standard tail-risk measure reported alongside the Sharpe and Sortino ratios in the performance tables.
Out-of-sample evaluation	Assessment of model performance on data not used during training or hyperparameter selection. A central validation principle of this thesis: all reported performance figures are computed on a held-out test period.
Sharpe ratio	A risk-adjusted performance measure defined as the ratio of mean excess return to return volatility. Higher values indicate better return per unit of risk.

Term	Definition
Sortino ratio	A risk-adjusted return measure analogous to the Sharpe ratio but using downside deviation — the volatility of negative returns only — rather than total volatility in the denominator. Rewards strategies that limit losses without penalising upside volatility.

List of Tables

List of Figures

List of Appendices

APPENDICES

A. FEATURE INVENTORY

The tables below catalogue every feature type implemented in the feature registry. The subset used in the main state-space experiment is marked in the **In model** column: ✓ indicates all variants are used, while a specific parameter value indicates only that variant is used. Feature names match the keys produced by the feature pipeline at runtime.

The ten static LOB features used in the main HFT experiment are z-score normalized with causal running statistics that are updated chronologically and reset at detected session boundaries. The runtime position feature is appended by the environment and remains in its raw bounded scale $[-1, 1]$. The cyclical time encodings listed in the registry are inherently bounded but are not part of the selected TD3 specification.

A.1. LOB Microstructure Features

Notation. P_i^{bid}, P_i^{ask}: bid/ask price at book level i; V_i^{bid}, V_i^{ask}: resting volume at level i; C_i^{bid}, C_i^{ask}: order count at level i; $M_t = \frac{1}{2}(P_0^{bid} + P_0^{ask})$: mid-price; $S_t = P_0^{ask} - P_0^{bid}$: spread; $QWM_t = \frac{V_0^{ask} P_0^{bid} + V_0^{bid} P_0^{ask}}{V_0^{bid} + V_0^{ask}}$: queue-weighted midpoint (legacy feature name `microprice`); $\bar{P}_N^{vw} = \frac{\sum_{i<N}(P_i^{bid} V_i^{bid} + P_i^{ask} V_i^{ask})}{\sum_{i<N}(V_i^{bid} + V_i^{ask})}$: volume-weighted mid-price over the top N levels; $\Delta V_t^{bid} = V_{0,t}^{bid} - V_{0,t-1}^{bid}$, $\Delta V_t^{ask} = V_{0,t}^{ask} - V_{0,t-1}^{ask}$: tick-to-tick volume change; W: rolling window in events; $\sum_W x_t = \sum_{\tau=t-W+1}^{t} x_\tau$: rolling sum; $\bar{x}_W$: rolling mean over W events; V_W^B, V_W^S: rolling buy/sell volume over W events; L: odd-lot size threshold (100 shares); Q: notional or size threshold; h_t: hour of day (0–23); Δt_{ns}: inter-event time in nanoseconds; $\mathbf{1}[\cdot]$: indicator function; $\hat{\rho}(\cdot, \text{lag} = 1)$: sample autocorrelation at lag 1.

Table 10. Full LOB feature registry with formulas and citations.

Feature	Group	Formula	Param	In model	Citation
`hft_book_pressure`	Imbalance	$(V_i^{bid} - V_i^{ask})/(V_i^{bid} + V_i^{ask})$	$i \in \{0, 1, 2\}$	level 0	Cao et al. (2009); Biais et al. (1995)
`hft_order_book_imbalance`	Imbalance	$\frac{\sum_{i<N}(V_i^{bid} - V_i^{ask})}{\sum_{i<N}(V_i^{bid} + V_i^{ask})}$	$N = 3$	✓	Cont et al. (2014)
`hft_order_count_imbalance`	Imbalance	$(C_i^{bid} - C_i^{ask})/(C_i^{bid} + C_i^{ask})$	$i = 0$	level 0	Biais et al. (1995)
`hft_microprice`	Fair value	QWM_t	—	✓	Stoikov (2018)
`hft_microprice_divergence`	Fair value	$QWM_t - M_t$	—	✓	—
`hft_vwmp_skew`	Fair value	$(\bar{P}_N^{vw} - M_t)/S_t$	$N = 3$		—
`hft_price_vamp`	Fair value	Avg. execution price for fixed-notional order walked through N levels	$Q \in \{1k, 5k\}$		—
`hft_depth_ratio`	Depth	$(V_0^{bid} + V_0^{ask}) \,/\, \sum_{i=1}^{N}(V_i^{bid} + V_i^{ask})$	$N = 4$		Bouchaud et al. (2002)
`hft_spread_bps`	Spread/cost	$(P_0^{ask} - P_0^{bid})/M_t \times 10{,}000$	—		Kyle (1985); Glosten & Milgrom (1985)
`hft_spread_ratio`	Spread/cost	$\text{SpreadBps}_t \,/\, \overline{\text{SpreadBps}}_W$	$W = 50$		Easley et al. (2012)
`hft_bid_convexity`	Spread/cost	$(P_0^{bid} - P_1^{bid}) - (P_1^{bid} - P_2^{bid})$	bid		Bouchaud et al. (2002)
`hft_ask_convexity`	Spread/cost	$(P_2^{ask} - P_1^{ask}) - (P_1^{ask} - P_0^{ask})$	ask		Bouchaud et al. (2002)
`hft_bid_slope`	Spread/cost	$(P_0^{bid} - P_4^{bid}) \,/\, \sum_{i<5} V_i^{bid}$	5 levels	✓	Bouchaud et al. (2002); Kyle (1985)
`hft_ask_slope`	Spread/cost	$(P_4^{ask} - P_0^{ask}) \,/\, \sum_{i<5} V_i^{ask}$	5 levels	✓	Bouchaud et al. (2002); Kyle (1985)

Table 10 continued.

Feature	Group	Formula	Param	In model	Citation
`hft_ofi`	Order flow	$\Delta V_t^{bid} \cdot \mathbf{1}[P_{0,t}^{bid} \geq P_{0,t-1}^{bid}] - \Delta V_t^{ask} \cdot \mathbf{1}[P_{0,t}^{ask} \leq P_{0,t-1}^{ask}]$	—	✓	Cont et al. (2014)
`hft_ofi_rolling`	Order flow	$\sum_W \text{OFI}_t$	$W = 50$	$W = 50$	Cont et al. (2014)
`hft_ofi_multilevel`	Order flow	$\sum_{i<N} \text{OFI}_t^{(i)}$ (event classification at each level i)	$N \in \{3, 5\}$		Cont et al. (2014)
`hft_bid_queue_depletion`	Order flow	$\max(V_{0,t-1}^{bid} - V_{0,t}^{bid},\, 0) \,/\, V_{0,t-1}^{bid}$	bid		Foucault et al. (2005)
`hft_ask_queue_depletion`	Order flow	$\max(V_{0,t-1}^{ask} - V_{0,t}^{ask},\, 0) \,/\, V_{0,t-1}^{ask}$	ask		Foucault et al. (2005)
`hft_signed_trade_flow`	Order flow	$\sum_W \text{size}_\tau \cdot (\mathbf{1}[\text{side}_\tau = \text{B}] - \mathbf{1}[\text{side}_\tau = \text{A}])$	$W = 50$	$W = 50$	Lee & Ready (1991)
`hft_odd_lot_trade_ratio`	Order flow	$\sum_W \mathbf{1}[\text{trade} \wedge \text{size} < L] \,/\, \sum_W \mathbf{1}[\text{trade}]$	$W = 200$		Boehmer et al. (2021)
`hft_odd_lot_imbalance`	Order flow	$(V_W^{B,\text{odd}} - V_W^{S,\text{odd}})/(V_W^{B,\text{odd}} + V_W^{S,\text{odd}})$; size $< L$	$W = 200$		Boehmer et al. (2021)
`hft_vpin`	Order flow	$\lvert V_W^B - V_W^S \rvert \,/\, (V_W^B + V_W^S)$	$W \in \{100, 500\}$		Easley et al. (2012)
`hft_cancel_to_trade`	Order flow	$\sum_W \mathbf{1}[\text{action} = \text{C}] \,/\, \sum_W \mathbf{1}[\text{action} = \text{T}]$	$W \in \{200, 1000\}$		Foucault et al. (2005)
`hft_trade_arrival_rate`	Order flow	$\sum_W \mathbf{1}[\text{action} = \text{T}]$	$W \in \{100, 500\}$		Engle (2000)
`hft_large_trade_ratio`	Order flow	$\sum_W \mathbf{1}[\text{trade} \wedge \text{size} \geq Q] \,/\, \sum_W \mathbf{1}[\text{trade}]$	$W = 200$		—
`hft_ofi_autocorr`	Order flow	$\hat{\rho}(\text{OFI}_{t-W:t},\, \text{lag} = 1)$	$W \in \{50, 200\}$		Cont et al. (2014)
`hft_inter_event_time`	Regime/temp.	$\log(1 + \Delta t_{\text{ns}})$	—		Engle (2000)
`hft_mid_price_acceleration`	Regime/temp.	$M_t - 2M_{t-1} + M_{t-2}$	—		Abergel et al. (2016)
`hft_hour_sin`	Regime/temp.	$\sin(2\pi h_t/24)$	—		Wood et al. (1985)
`hft_hour_cos`	Regime/temp.	$\cos(2\pi h_t/24)$	—		Wood et al. (1985)

Source: *author's own compilation. Formulas are implementations or adaptations; citations identify conceptual sources where applicable. A dash denotes an author's construction or no direct originating formula.*

B. MAIN EXPERIMENT SPECIFICATION

The table below condenses the main experiment into a self-contained research specification. It records the empirical choices needed to reproduce the experiment. Numerical hyperparameters are loaded from the exported thesis snapshot so that the table stays in sync with the scenario configuration without manual updates.

Table 11. Main TD3 experiment specification.

Component	Specification
Asset and data	AAPL, MSFT, TSLA, META, AMZN, and AVGO; ten-level limit order book observations during regular U.S. trading hours
Training data	18 pooled symbol-day files: AAPL, AMZN, AVGO, META, MSFT, and TSLA over February 25–27, 2026
Validation/test data	March 2, 2026 symbol-day files, capped at 50,000 rows per split
Feature set	Selected causal LOB microstructure set: book pressure L0, three-level OBI, order-count imbalance L0, queue-weighted midpoint and its divergence (legacy microprice feature names), bid/ask slope, OFI, 50-event rolling OFI, 50-event signed trade flow, and runtime position
Environment backend	Continuous single-asset trading environment
Episode length	10,000 event-time steps (streaming)
Action	Target portfolio exposure in [-1, 1]
Reward	Differential Sharpe Ratio with eta = 0.01
Transaction fee	0.000 in the baseline setting
Algorithm	TD3
Network widths	Actor and critic hidden layers [128, 64]
Learning rates	Actor and critic 0.0001
Weight decay	Actor 0.0; critic 2×10^{-6}
Discount factor	gamma = 0.9
Target update	tau = 0.005
Policy delay	2 critic updates per actor update
Target policy noise	sigma = 0.2, clipped at ±0.3
Exploration noise	sigma = 0.3 during training

Table 11 continued.

Component	Specification
Total training steps	3,000,000
Initial random steps	5,000
Frames per batch	200
Optimisation steps per batch	5
Mini-batch size	128
Replay buffer size	100,000
Loss function	Smooth L1 (Huber)
Evaluation random seed	42
Compute environment	Apple M3 MacBook (November 2023) with 18 GB unified memory

Source: Author's own compilation.

C. BASELINE ALGORITHM PSEUDOCODE

Chapter 5.2 gives TD3's full training loop (Algorithm 2) since TD3 is this thesis's primary method. DDPG and PPO are baseline comparisons used in Chapter 6's algorithm comparison; their training loops are recorded here for reproducibility.

Algorithm 3 DDPG Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; DDPG hyperparameters

Ensure: Trained policy μ_ϕ, evaluation metrics

Initialise actor μ_ϕ, critic Q_θ, target networks $\mu_{\phi'}$, $Q_{\theta'}$, replay buffer $\mathcal{D}$
for each step $t = 1, \ldots,$ `max_steps` **do**
    $a_t \leftarrow \text{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big), \quad \epsilon \sim \mathcal{N}(0,\ \sigma^2_{\text{explore}})$
    Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
    Sample mini-batch $\mathcal{B} \sim \mathcal{D}$
    Compute Bellman target: $y \leftarrow r + \gamma \cdot Q_{\theta'}(s',\ \mu_{\phi'}(s'))$
    Update critic by minimising $\mathcal{L}_{\text{Smooth-L1}}(y - Q_\theta(s, a))$
    Update actor: $\phi \leftarrow \phi + \alpha \nabla_\phi Q_\theta(s,\ \mu_\phi(s))$
    Soft-update targets: $\theta' \leftarrow \tau\theta + (1-\tau)\theta';\ \phi' \leftarrow \tau\phi + (1-\tau)\phi'$
    Checkpoint and log metrics at configured intervals
end for
Evaluate deterministic policy μ_ϕ on test split
Compute financial performance metrics from realised return series

Algorithm 4 PPO Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; PPO hyperparameters

Ensure: Trained policy π_θ, evaluation metrics

Initialise stochastic actor π_θ (TanhNormal), value network V_ϕ
for each batch iteration $k = 1, \ldots,$ `max_steps/frames_per_batch` **do**
    Collect trajectory $\mathcal{T}_k$ of `frames_per_batch` steps using π_θ
    Compute advantage estimates $\hat{A}_t$ via GAE using V_ϕ on $\mathcal{T}_k$
    for each PPO epoch $j = 1, \ldots, K$ **do**
        Sample mini-batch $\mathcal{B} \sim \mathcal{T}_k$
        Compute probability ratio: $\rho_t(\theta) \leftarrow \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$
        Compute clipped objective: $\mathcal{L}^{\text{CLIP}} \leftarrow \mathbb{E}\big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1 \pm \epsilon)\hat{A}_t\big)\big]$
        Compute value loss: $\mathcal{L}^V \leftarrow \mathcal{L}_{\text{L2}}\big(V_\phi(s) - V^{\text{target}}\big)$
        Compute entropy bonus: $\mathcal{L}^H \leftarrow c_1 \mathcal{H}(\pi_\theta(\cdot \mid s))$
        Update θ, ϕ jointly: minimise $-\mathcal{L}^{\text{CLIP}} + c_2 \mathcal{L}^V - \mathcal{L}^H$
    end for
    Checkpoint and log metrics at configured intervals
end for
Evaluate policy mode $\tanh(\mu_\theta(s))$ on test split
Compute financial performance metrics from realised return series

For DDPG (beyond the symbols already defined in Chapter 5.2 for TD3):

- μ_ϕ is the deterministic actor; Q_θ is DDPG's single critic
- Primed symbols denote slowly updated target networks
- $\mathcal{D}$ is the replay buffer; τ is the soft-update coefficient

For PPO:

- π_θ is the stochastic actor (TanhNormal distribution)
- V_ϕ is the state value network
- $\hat{A}_t$ are GAE advantage estimates
- $\rho_t(\theta)$ is the probability ratio between new and old policies
- ϵ is the clipping threshold

- c_1, c_2 are the entropy and value loss coefficients
- K is the number of PPO epochs per batch

D. RAW DATA INVENTORY

Table 12 reports raw MBP-10 event counts and inter-event timing per symbol and split, before the level-0-4 filter described in Section 5.3.7 is applied. Training aggregates all three training days per symbol (February 25-27, 2026); validation and test each cover one chronological half of the March 2 session.

Table 12. Raw MBP-10 event counts and inter-event timing per symbol and split.

Symbol	Split	Events	Trades	Mean Δt	Median Δt
AAPL	Train	4,575,844	402,135	16.57 ms	39.1 µs
AMZN	Train	7,101,442	384,315	10.36 ms	64.7 µs
AVGO	Train	1,636,373	285,066	49.38 ms	145.9 µs
META	Train	1,105,240	161,099	71.22 ms	242.2 µs
MSFT	Train	3,590,446	518,469	22.02 ms	50.4 µs
TSLA	Train	5,611,429	692,119	13.89 ms	38.9 µs
AAPL	Val	1,207,098	66,269	7.51 ms	28.1 µs
AAPL	Test	1,207,098	92,876	13.07 ms	38.8 µs
AMZN	Val	1,354,730	53,610	6.44 ms	60.1 µs
AMZN	Test	1,354,730	69,455	11.53 ms	82.5 µs
AVGO	Val	296,905	37,745	34.62 ms	136.1 µs
AVGO	Test	296,906	45,813	53.21 ms	148.7 µs
META	Val	188,074	24,474	58.99 ms	318.2 µs
META	Test	188,075	26,375	78.95 ms	304.6 µs
MSFT	Val	490,610	70,938	21.87 ms	55.4 µs
MSFT	Test	490,610	65,798	31.04 ms	65.4 µs
TSLA	Val	999,867	117,916	9.68 ms	31.5 µs
TSLA	Test	999,868	115,563	16.10 ms	42.2 µs

Source: Author's own calculations based on DataBento Nasdaq MBP-10 data.

Note: Mean and median Δt computed from positive within-session inter-event deltas only (overnight gaps excluded for the training split). Training aggregates all three days per symbol (February 25–27, 2026); validation and test each cover one chronological half of the March 2 session.

E. TRANSFORMED FEATURE EXAMPLE

Table 13 shows twelve consecutive events from the AAPL feed, illustrating how the raw order-book state is transformed into the input features used by the RL agent. Each row is one

order-book event; the left columns display the raw state (best bid/ask prices and sizes), the right columns show the corresponding normalized microstructure features computed from that state via the causal running normalization in Equation 28. The table is wide (nine columns), so in the PDF it is placed on its own landscape-oriented page; the HTML version scrolls horizontally instead.

Table 13. Twelve consecutive AAPL LOB events and their selected microstructure features.

Timestamp	Best Bid	Best Ask	Bid Size	Ask Size	Book Pressure	Order Imbalance	QWM Deviation	OFI
16:53:18.611	265.71	265.74	240	124	0.408	0.284	0.213	0.004
16:53:19.027	265.71	265.74	242	124	0.415	0.288	0.216	0.05
16:53:19.044	265.72	265.74	25	124	-1.418	-0.288	-0.489	0.578
16:53:19.044	265.72	265.74	71	124	-0.689	-0.157	-0.248	1.061
16:53:19.044	265.72	265.74	96	124	-0.42	-0.09	-0.159	0.578
16:53:19.044	265.72	265.74	50	124	-0.974	-0.216	-0.342	-1.052
16:53:19.044	265.72	265.74	25	124	-1.418	-0.288	-0.489	-0.57
16:53:19.044	265.71	265.74	242	124	0.415	0.288	0.216	-0.57
16:53:19.612	265.71	265.74	242	99	0.595	0.358	0.305	0.578
16:53:19.612	265.72	265.74	10	99	-1.701	-0.261	-0.582	0.234
16:53:19.612	265.72	265.74	10	90	-1.67	-0.234	-0.572	0.211
16:53:19.612	265.72	265.74	35	90	-1.001	-0.16	-0.351	0.578

Source: DataBento Nasdaq MBP-10 feed, AAPL, 2 March 2026.

Note: Twelve consecutive order-book events selected from the test split to include multiple bid/ask price changes. Book Pressure, Order Imbalance, queue-weighted-midpoint (QWM) Deviation, and OFI are z-score normalized using causal running statistics. A vertical rule separates the raw order-book state (left columns) from the normalized features (right columns).

F. TICK-SIZE BREAK-EVEN FEE BY INSTRUMENT

Table 14 supports the transaction-cost discussion in Section 6.2. The minimum price increment for these instruments is one cent, so the smallest possible one-sided move in the mid-price is half a cent. At each instrument's mean price over the evaluation window (2 March 2026), this sets a break-even fee: the proportional trading cost at which a single minimum-tick move stops covering the fee.

Table 14. Per-instrument break-even transaction fee by mean price.

Symbol	Mean Price	Break-Even Fee
META	652.89	0.077
TSLA	400.10	0.125
MSFT	398.48	0.125
AVGO	315.49	0.158
AAPL	264.37	0.189
AMZN	207.97	0.240

Source: Author's own calculations based on DataBento Nasdaq MBP-10 data.

Note: Break-even fee, in basis points, is the proportional trading cost at which a one-sided minimum-tick move stops covering the fee: (half of the one-cent minimum price increment / mean mid-price) x 10,000. Mean price is in US dollars, over the 2 March 2026 evaluation session.

G. FEATURE DISTRIBUTION STATISTICS

Table 15 reports distributional statistics for all engineered microstructure features computed over the training split, after skipping the first 500 events to allow rolling-window features to stabilise. Means close to zero and standard deviations near one confirm that causal running normalisation is operating as intended. Q1, Q2, and Q3 show the interquartile spread; Skew and Kurt (excess kurtosis) characterise the shape of each marginal distribution.

Several features — most notably `ofi` and `ofi_rolling_50` — exhibit extreme minimum and maximum values despite their near-unit standard deviations. This is a structural property of order-flow imbalance, not a data-quality issue. Most tick events represent small queue adjustments that produce z-scores close to zero, while occasional aggressive orders consume multiple price levels in rapid succession. Because the running standard deviation is dominated by the quiet majority, these rare spikes emerge as z-scores of ± 290 or more and produce the large positive excess kurtosis visible for OFI features.

The values in Table 15 are post-normalisation but pre-clipping. The environment applies a hard observation clip of $[-5, 5]$ through `obs_clip`, so the policy network never receives the extreme tails shown in the table.

Table 15. Distributional statistics of engineered microstructure features.

Feature	Mean	Std	Skew	Kurt	Median	Min	Max
book_pressure_l0	0.0105	0.9695	-0.062	-1.018	0.0269	-2.1249	2.3985
book_pressure_l1	0.0167	0.9565	-0.043	-0.839	0.0208	-2.3712	2.3173
book_pressure_l2	0.0126	0.9642	-0.021	-0.713	0.0167	-2.7692	2.8761
order_book_imbalance_3l	0.0127	0.9521	-0.042	-0.327	0.0222	-3.6806	3.1602
order_count_imbalance_l0	0.0157	1.0256	-0.012	-0.677	-0.0205	-4.8446	3.9749
microprice	-0.2207	1.1677	0.173	-0.629	-0.2427	-3.9217	4.5490
microprice_divergence	0.0054	0.6207	4.179	306.521	0.0108	-35.9938*	35.1759*
vwmp_skew	-0.0160	0.9841	-0.057	49.225	-0.0201	-115.2118*	81.6524*
price_vamp	-0.2207	1.1680	0.174	-0.629	-0.2431	-4.0689	4.4282
spread_bps	-0.4428	0.7457	13.581	575.179	-0.5041	-2.8230	70.3718*
spread_ratio_50	-0.0035	0.9807	0.841	3.319	-0.0760	-3.4316	41.7445*
bid_convexity	-0.0231	0.4495	0.954	215.217	-0.0327	-27.5972*	52.6868*
ask_convexity	0.0224	0.4676	-1.582	338.717	0.0334	-67.1721*	39.6020*
bid_slope	-0.0075	0.0329	24.334	2564.642	-0.0032	-0.5273	9.7874*
ask_slope	-0.0069	0.0289	18.460	1343.546	-0.0032	-0.3072	6.4803*
depth_ratio	-0.0196	0.9781	27.783	1594.771	-0.1422	-1.2729	219.7064*
ofi	0.0012	1.1514	19.480	19260.628	0.0003	-564.2619*	513.1559*
ofi_rolling_50	0.0066	1.0842	3.685	371.010	0.0088	-61.0772*	93.9005*
ofi_multilevel_3l	0.0007	1.1154	0.415	4041.658	-0.0009	-329.4502*	305.5992*
ofi_autocorrelation_50	0.0498	0.9933	-0.443	1.516	0.0645	-6.9517*	5.4193*
bid_queue_depletion	0.0087	0.9931	3.103	8.889	-0.3315	-0.4249	6.0522*
ask_queue_depletion	0.0073	0.9896	3.010	8.261	-0.3372	-0.4651	5.6298*
signed_trade_flow_50	-0.0114	1.0649	6.393	615.608	0.0064	-61.9163*	103.4519*
vpin_100	0.0169	0.9752	-0.743	-1.011	0.6497	-4.4186	1.2639
large_trade_ratio_100	-0.0078	0.8814	16.095	367.247	-0.0956	-0.8435	48.9647*
cancel_to_trade_ratio_200	-0.0934	0.9235	2.684	10.572	-0.3728	-2.0828	21.3408*
trade_arrival_rate_100	0.0835	1.0040	7.193	100.654	-0.1379	-1.6466	30.0658*
odd_lot_trade_ratio_200	0.0570	0.9130	-2.120	7.899	0.3837	-13.8982*	2.1290
odd_lot_imbalance_200	-0.0070	0.9780	0.082	-1.311	0.0335	-7.1696*	4.0563
inter_event_time	0.0651	1.0469	-0.271	0.930	-0.1174	-4.4738	3.6724
mid_price_acceleration	-0.0000	0.4108	-0.086	409.535	0.0000	-60.7336*	68.7368*
hour_sin	-0.8086	0.1641	0.731	-0.657	-0.8660	-1.0000	-0.5000
hour_cos	-0.2771	0.4924	0.409	-1.308	-0.5000	-0.8660	0.5000

Computed over the full pooled training set streamed to the agent – six instruments over three sessions (25-27 February 2026), roughly 18.7 million events – with the first 500 events of each session skipped so the rolling-window features reach steady state. Values are the normalized features as fed to the policy (causal running z-score, reset at session boundaries), before the environment's observation clip. Skew and Kurt are the bias-corrected Fisher skewness and excess kurtosis. Median is the 50th percentile. * *the RL environment clips observations to* ± 5*; starred values exceed this bound.*

H. FEATURE–RETURN CORRELATIONS

Table 16 reports Pearson and Spearman rank correlations between each engineered feature and the one-step-ahead log mid-price return, computed over the full streamed training set (six instruments, three sessions); see Section 5.3.5 for the interpretation of this result.

Table 16. Pearson and Spearman rank correlations between each engineered feature and the one-step-ahead log mid-price return, over the streamed training set.

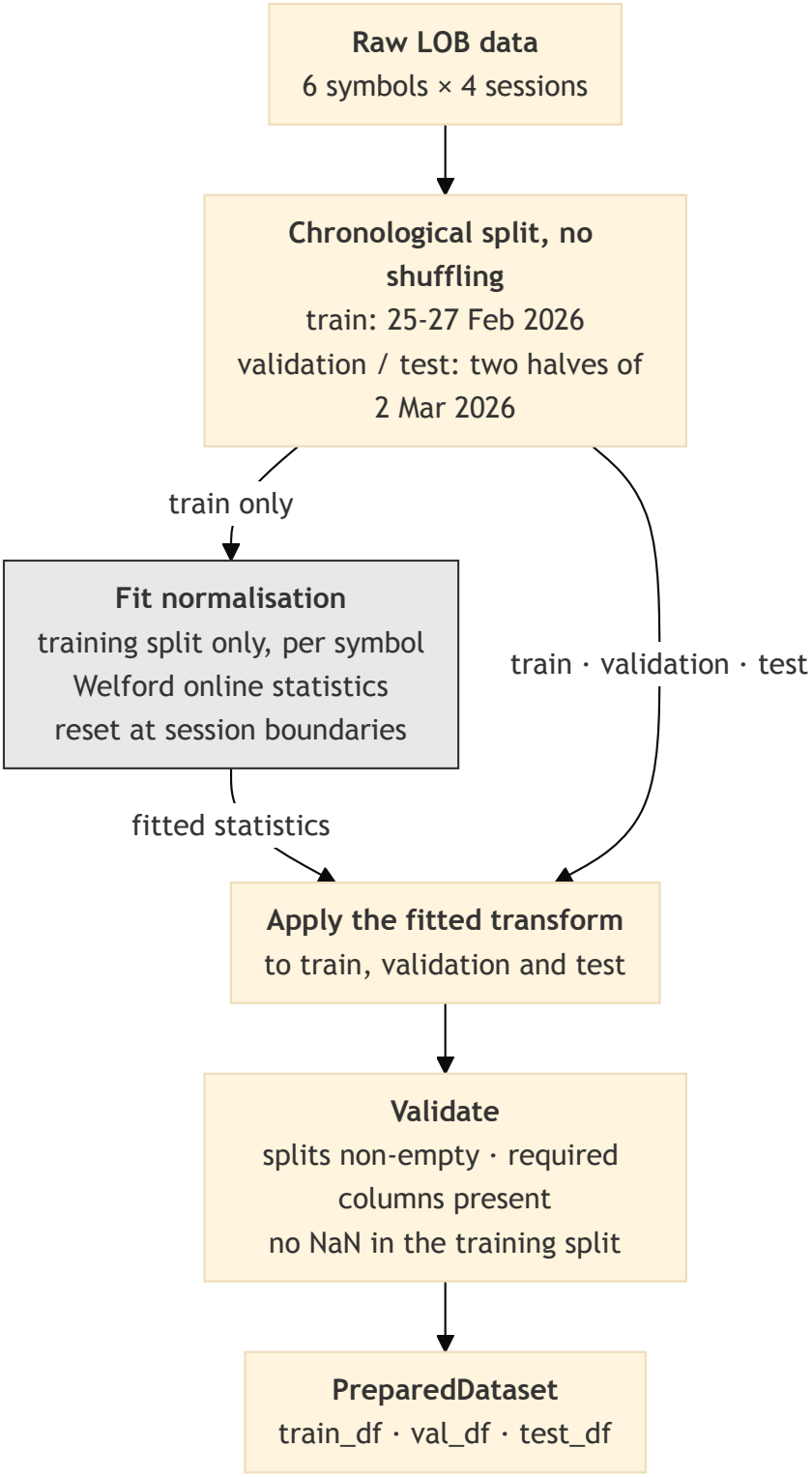
Feature	Pearson	Spearman	Feature	Pearson	Spearman
order_count_imbalance_l0	+0.1608	+0.2597	vwmp_skew	+0.0152	-0.0488
book_pressure_l0	+0.1090	+0.1828	spread_ratio_50	-0.0070	-0.0037
microprice_divergence	+0.0880	+0.1715	inter_event_time	+0.0061	+0.0069
bid_queue_depletion	-0.0788	-0.0933	book_pressure_l2	+0.0057	+0.0121
ask_queue_depletion	+0.0764	+0.0930	spread_bps	-0.0040	-0.0011
ask_convexity	-0.0728	-0.0147	vpin_100	-0.0025	-0.0031
bid_convexity	-0.0680	-0.0124	trade_arrival_rate_100	-0.0023	-0.0053
order_book_imbalance_3l	+0.0576	+0.1004	ofi_autocorrelation_50	-0.0015	-0.0023
mid_price_acceleration	-0.0448	+0.0090	microprice	+0.0014	+0.0042
ofi	+0.0393	+0.1681	cancel_to_trade_ratio_200	+0.0013	+0.0023
ask_slope	+0.0389	+0.0443	depth_ratio	+0.0008	+0.0020
bid_slope	-0.0355	-0.0392	odd_lot_trade_ratio_200	-0.0007	+0.0001
ofi_rolling_50	+0.0352	+0.0843	hour_cos	+0.0004	+0.0009
signed_trade_flow_50	+0.0309	+0.0866	price_vamp	+0.0004	+0.0029
odd_lot_imbalance_200	+0.0268	+0.0422	hour_sin	+0.0001	-0.0004
ofi_multilevel_3l	+0.0240	+0.1459	large_trade_ratio_100	+0.0000	+0.0012
book_pressure_l1	+0.0187	+0.0333			

Each event's feature value is paired with the log mid-price return to the next event, pooled over six instruments and three sessions (roughly 18.7 million pairs), with the first 500 events of each session skipped. Pearson measures linear association, Spearman any monotone association. The largest magnitude is a Spearman correlation of 0.26 for best-level order-count imbalance; the imbalance and order-flow features sit near 0.15-0.26 on Spearman and below 0.16 on Pearson, and every other feature is below 0.02 on both. Section 5.3.5 interprets this.

I. DATA PREPARATION PIPELINE

The diagram below shows the feature pipeline design that ensures no information from the validation or test splits leaks into the fitted normalization parameters. Only the training split reaches the fitting step; all three splits are then transformed using the statistics estimated from it.

Fig. 3. Data preparation pipeline: fitting normalization on the training split only, then transforming train/val/test to prevent leakage.



Source: Author's own elaboration.

The figure shows the leakage-prevention path only. The full pipeline – including the prepared-data and feature caches and their compatibility keys, the per-day and chronological branches, per-symbol memory-mapped output, HFT post-processing, and the timestamp de-duplication applied after concatenation across symbols – is maintained as a Mermaid source file at `docs/diagrams/data_pipeline.mmd` in the project repository (Wojdalski, 2026). It is too information-dense to reproduce at print resolution.

J. OFFLINE FEATURE SELECTION PIPELINE

The feature selection procedure described in Section 4.2 operates in two stages: theory constrains the candidate pool to mechanisms grounded in market microstructure literature; IC/ICIR ranking selects among the available implementations of those mechanisms and removes redundancy via a conditional IC filter. This appendix describes the operational pipeline that implements Stage 2 of that procedure. It takes a theory-defined candidate pool as input, scores each feature against a forward return proxy, and stores the selected feature specification used by the training pipeline.

The procedure scores each candidate feature against a forward return proxy and applies a greedy redundancy filter to produce the reduced set used by the training pipeline.

Proxy target. Features are scored against the forward log-return at horizon h:

$$y_t = \log \frac{P_{t+h}}{P_t} \tag{29}$$

Scoring. The Information Coefficient (IC) of feature f_i is its Spearman rank correlation with the forward return:

$$\mathrm{IC}_{i,t} = \rho_{\mathrm{Sp}}\big(f_{i,t},\, y_t\big) \tag{30}$$

computed over rolling windows on the validation split. Features are ranked by the IC Information Ratio (ICIR):

$$\mathrm{ICIR}_i = \frac{\overline{\mathrm{IC}}_i}{\sigma(\mathrm{IC}_i)} \tag{31}$$

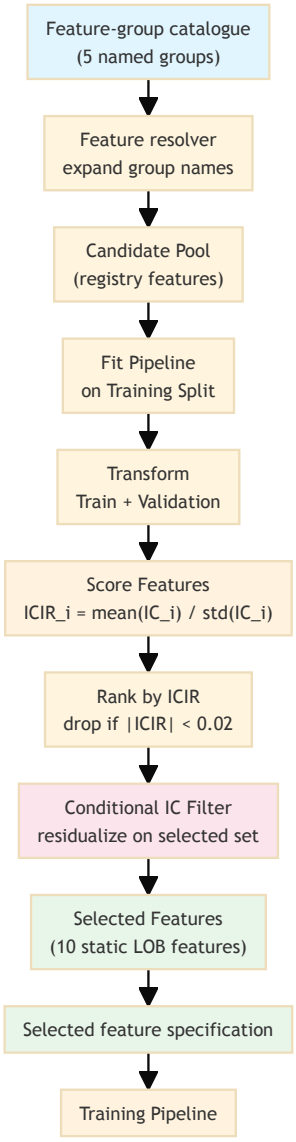
which jointly rewards signal strength and consistency; features with $|\mathrm{ICIR}_i| < 0.02$ are dropped.

Redundancy filter. Features are selected greedily in ICIR order. Each candidate is included only if its ICIR on residuals — after regressing out already-selected features — remains above the threshold. This conditional IC criterion is more principled than pairwise correlation thresholding: two correlated features can both be retained if each explains a distinct component of the forward return.

Output. The selected feature specification is stored and read by the training pipeline at runtime; no selection logic runs during training.

The diagram below summarizes the pipeline.

Fig. 4. Offline feature selection pipeline: score candidates by ICIR, apply a conditional-IC redundancy filter, store the selected specification for training.



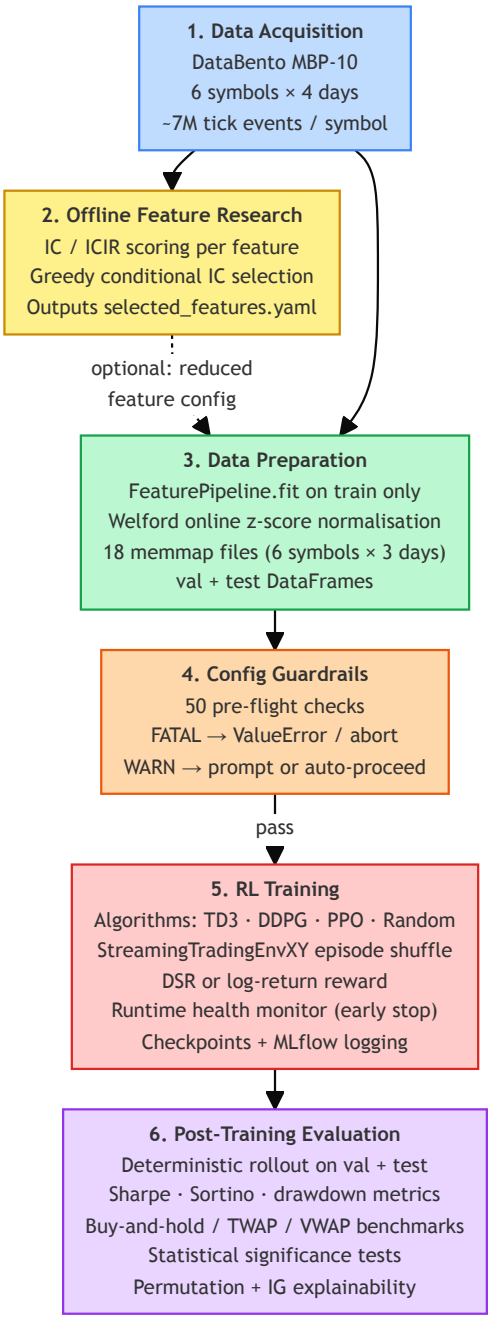
Source: Author's own elaboration.

The offline nature of this pipeline is deliberate: the proxy target and scoring metrics are not the reward function that the agent optimizes during training. Running selection as a separate step ensures that the agent's training loop remains uncontaminated by any pre-training statistical analysis that could introduce look-ahead bias or circular reasoning.

K. EXPERIMENTAL PIPELINE ARCHITECTURE

The figure below shows the six major stages of the experimental pipeline at a glance. Each stage is described in the corresponding chapter; this overview is intended as a navigation aid.

Fig. 5. The six major stages of the experimental pipeline, from data acquisition through post-training evaluation.



Source: Author's own elaboration.

The full machine-readable diagram — including CLI entry points, per-step data flows, all configuration guardrail checks, environment variants, MLflow logging, and asset provenance metadata — is maintained as a D2 source file at `docs/workflow.d2` in the project repository (Wojdalski, 2026). It is too information-dense to reproduce at print resolution; the SVG rendering is available at `docs/workflow.svg`.